조선대학교 CHOSUN UNIVERSITY

AVR 마이크로 프로세서

차례

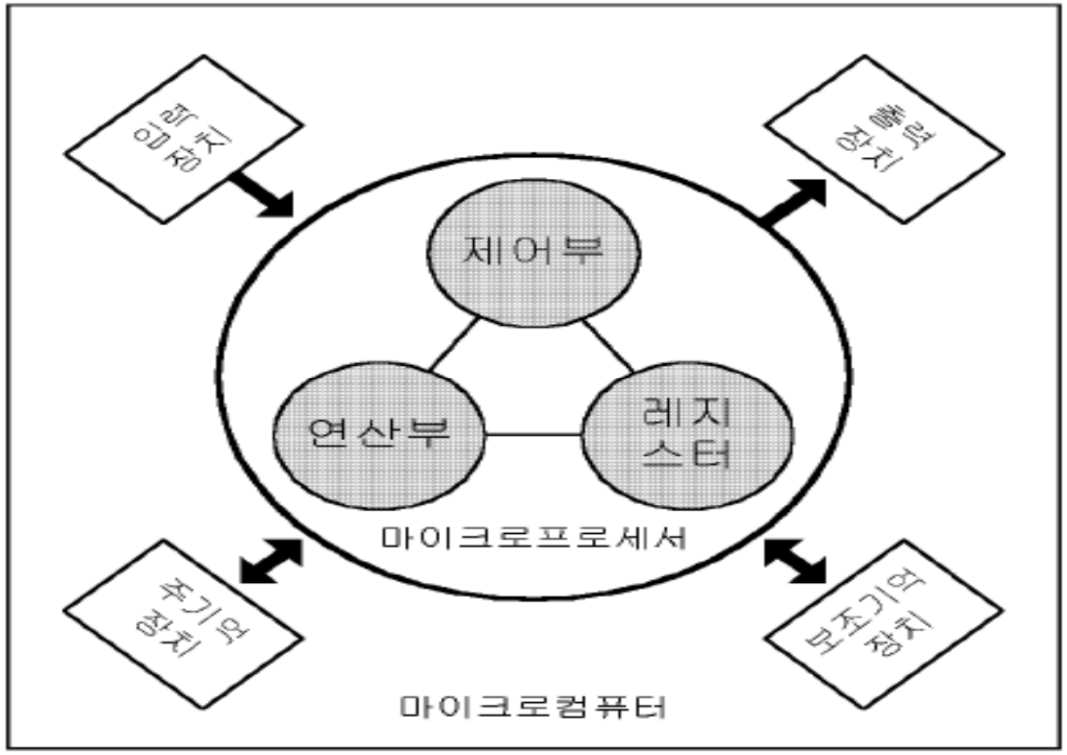
- 기본 개념
- AVR 마이크로 컨트롤러의 개요
- ATmega128의 기본구조와 기능
- ATmega128의 명령 세트
- ATmega128의 기본 하드웨어

기본 개념 – 마이크로 프로세서

- 마이크로 프로세서란?
  - 컴퓨터의 중앙처리장치(Central Processor Unit ; CPU)를 단일 IC칩에 집적시켜 만든 반도체 소자
  - 1971년 미국 Intel 사에 의해서 세계 최초로 만들어 졌으며 오늘날은 이를 흔히 MPU(MicroProcessor Unit)이라고 함
  - 마이크로프로세서 = MPU < CPU

기본 개념 – 마이크로 컴퓨터

- 마이크로 컴퓨터
  - 마이크로프로세서를 CPU로 사용하여 만든 소형 컴퓨터
  - 소형경량화, 저 소비전력, 저가, 부품 수 감소로 인한 신뢰성 향상

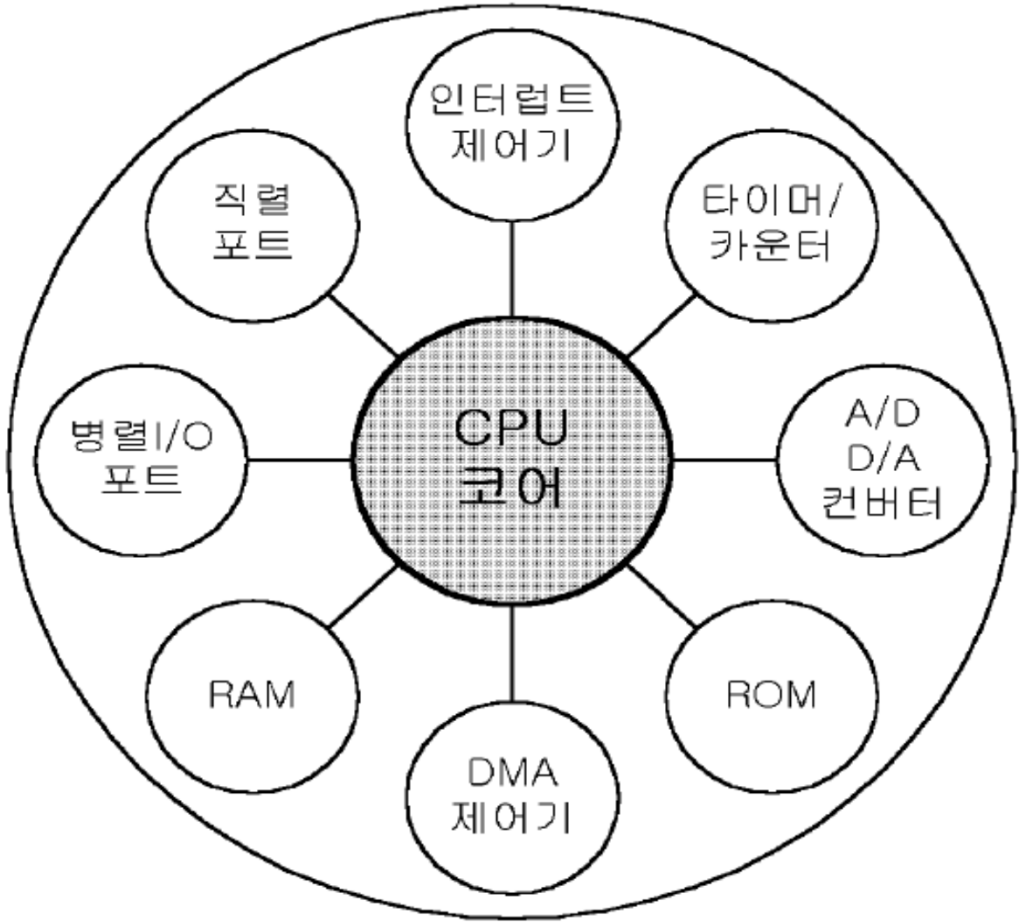


기본 개념 – 마이크로 컨트롤러

- 마이크로 컨트롤러란?
  - 한 개의 칩 내에 CPU기능은 물론이고 일정한 용량의 메모리 (ROM,RAM 등)와 입출력 제어 인터페이스 회로까지 내장한 것
  - 범용의 목적보다는 기기 제어용에 주로 사용되므로 붙여진 이름
  - Single-chip Microcomputer 라고도 함
  - MPU와 구분하기 위하여, MicroController Unit (MCU) 또는 MicroComputer Unit (MCU) 라고 부르기도 함
  - 넓은 의미에서 마이크로프로세서는 MCU나 DSP등 CPU의 기능을 포함하는 단일 반도체 소자를 모두 지칭하는 포괄적인 개념

기본 개념 – 마이크로 컨트롤러

- 마이크로 컨트롤러



기본 개념 – 마이크로 프로세서 vs 마이크로 컨트롤러

- 마이크로 프로세서
  - 프로세서를 한 개의 칩으로 구현
  - 데이터부와 제어부가 들어있는 코어를 의미

- 마이크로 컨트롤러
  - 코어 + 여러 가지 크기와 다양한 종류의 메모리 + 여러 종류의 주변장치 + 여러 종류의 입출력 포트
  - 여러 응용분야에 필요로 하는 주변기기들을 한 칩 내에서 모두 제공
  - 시스템보드의 칩의 개수를 줄임
    - 경박단소 (經博短小) : 가볍고 얇고 짧고 작은 특징
    - 시스템을 성능 및 크기, 그리고 가격을 최적화
    - 제품을 제작 시 제품의 개발기간 및 가격 최소화
    - 제품의 용도에 맞게 프로그램의 변경만으로 적용이 가능하여 적용성 우수
    - 부품수가 적고 시스템 구성이 간단하여 신뢰성이 향상

기본 개념 – 마이크로 프로세서의 분류

- 데이터 버스의 크기, 산술처리기(Arithmetic Logic Unit) 및 레지스터들의 크기에 따른 분류
  - 8비트 프로세서 : 응용목적이 간단한 경우
  - 16비트 프로세서 : 중간 정도의 복잡도를 가지는 응용분야
  - 32비트 프로세서 : 내장형 시스템 시장에서 주류
  - 64비트 프로세서 : 몇몇 업체에서만 제공

- 명령어 구조에 따른 분류
  - Complex Instruction Set Computer (CISC)
  - Reduced Instruction Set Computer (RISC)
  - Extended Instruction Set Computer (EISC)

기본 개념 – 마이크로 프로세서의 분류

- 기능에 따른 분류
  - MPU – Micro Processor Unit
    - 중앙처리장치에서 주기억장치를 제외한 연산장치, 제어장치 및 각종 레지스터들을 단지 1개의 IC 소자에 집적시킨 것
    - 용도 : PC
  - MCU – Micro Controller Unit
    - 마이크로프로세서 중에 1개의 칩 내에 CPU 기능은 물론이고 일정한 용량의 메모리(ROM, RAM 등)와 입출력 제어 인터페이스 회로까지 내장한 것
    - MCU, 단일 칩 마이크로컴퓨터, 마이컴 등으로 불린다.
  - DSP – Digital Signal Processor
    - 디지털 신호를 하드웨어적으로 처리할 수 있는 집적회로
    - 용도 : 영상, 음성 등 신호처리

기본 개념 – 마이크로 프로세서의 분류

- 프로세서가 한번에 처리할 수 있는 비트 수로 분류
  - 8비트 프로세서
    - 1970년대 마이크로 프로세서
    - 현재는 소용량 마이크로 컨트롤러가 8-Bit 프로세서
    - 8051, AVR, PIC, SAM88
  - 16-Bit 프로세서
    - 1980년대 초 마이크로 프로세서
    - 현재는 고속의 마이크로 컨트롤러로 사용 8-Bit 와 32-Bit 의 발달로 많이 사용하지 않음
    - AM188, MSP430
  - 32-Bit 프로세서
    - 1980년대 중반 이후 현재까지 마이크로 프로세서
    - PC, 또는 대용량의 데이터 처리를 하는 산업용 장치에 쓰임

기본 개념 – 마이크로 프로세서의 분류

- 프로세서의 타입
  - 프로세서가 제공하는 명령어 세트 (Instruction Set)의 복잡도
  - Complex Instruction Set Computer(CISC)
    - 복잡한 명령어를 지원하는 프로세서
    - 1980년대 이후 마이크로프로세서 시장에 처음 선을 보임
    - 인텔, 모토롤러 프로세서
  - Reduced Instruction Set Computer(RISC)
    - 단순한 명령어 세트를 지원하는 프로세서
    - MIPS와 ARM의 프로세서

기본 개념 – 마이크로 프로세서의 분류

- RISC와 CISC 두 종류의 프로세서의 차이점
  - CISC : 복잡한 많은 수의 명령어를 해독하는데 많은 실리콘을 사용
  - RISC : 작은 명령어 세트로 인하여 남는(CISC에 비해서) 실리콘을 다른 유용한 목적으로 사용
    - 이전 : RISC와 CISC의 관계는 서로의 우월성을 주장하던 관계
    - 최근 : post RISC : 성숙한 단계로 접어들어서 상호간의 장•단점을 CPU 구조에 적절하게 반영하여 최적의 성능 제공

기본 개념 – 마이크로 프로세서의 분류

❑ CISC, RISC, EISC의 특징

구분	CISC	RISC	EISC
CPU instruction	명령어 개수가 많고, 그 길이가 다양하며 실행사이클도 명령어마다 다름	명령어길이는 고정적이며, 워드와 데이터 버스 크기가 모두 동일, 실행사이클도 모두 동일	16비트 명령을 사용하여 32비트 데이터를 처리
회로 구성	복잡	단순	단순
메모리 사용	높은 밀도 메모리 사용이 효율적	낮은 밀도의 명령어사용으로 메모리 사용이 비효율적	코드밀도가 높다(임베디드 시스템에 유리)
프로그램측면	명령어를 적게 사용	상대적으로 많은 명령어가 필요, 파이프라인 사용	RISC보다 더 깊은 파이프 라인 스테이지
컴파일러	다양한 명령을 사용하므로 컴파일러가 복잡해짐	명령어 개수가 적어서 단순한 컴파일러 구현 가능	국산 기술에 의해 개발 (ADC corp.)

기본 개념 – 마이크로 프로세서의 성능지표(Word)

- 각 마이크로프로세서는 프로세서 설계를 단순화 하기 위하여 고정된 길이의 데이터를 처리
- 바이트 (1byte=8bit) 많이 사용, 워드는 마이크로 프로세서마다 크기가 다르지만, 바이트는 일정
- 4 bit 마이크로프로세서
  - 가장 저가
  - BCD 사용가능
  - 계산기, 간단한 가전기기, 장난감
- 8 bit 마이크로프로세서
  - 가장 널리 쓰임
  - 4bit의 두배, BCD 숫자 두개 다룸
  - 숫자포함 문자 및 특별기호 표현 가능 (ASCII)
- 워드길이가 두 배로 되면 성능은 두 배 이상으로 강력해짐

기본 개념 – 마이크로 프로세서의 성능지표(메모리 용량)

- 메모리의 데이터 워드 길이는 마이크로 프로세서에서 사용하는 워드길이와 같음
- 메모리 주소의 크기가 클수록 마이크로 프로세서의 성능도 커짐
- 한 개 이상의 워드를 사용하여 메모리 주소를 형성할 수 있으므로, 메모리 주소범위는 마이크로 프로세서의 워드 길이에 의해서 제한되지 않음
- 8비트 마이크로 컴퓨터에서 65,536워드 메모리 = 16비트
  마이크로 프로세서에서 32,768워드 메모리 = 65,536바이트

기본 개념 – 마이크로 프로세서의 성능지표(명령어 실행 속도)

- 하나의 명령어의 인출/실행 주기를 완수하는데 걸리는 시간

- 속도를 측정하는 두 가지 방법
  - 클럭 속도 : 클럭의 주파수로 MHz 단위
  - 1초에 수행할 수 있는 명령어의 수 : MIPS (Millions of Instructions Per Second)로 나타냄

- MIPS율은 클럭 주파수와 워드 크기에 관련
  - 동일 워드, 주파수 2배 -> 2배 MIPS
  - 워드 2배, 주파수 동일 -> 2배 MIPS
  - 평균 명령어의 크기는 1바이트 이상이고, 보통 8bit 마이크로 프로세서가 명령어 1개를 인출하기 위해서는 두 번 메모리에 접근해야 함

기본 개념 – 마이크로 프로세서의 성능지표(속도를 높이기 위한 기술)

- 병렬처리
  - 두개의 프로세서 이용
- 코프로세싱
  - 전용계산을 위한 보조프로세서 이용
- 캐시 메모리 기술
  - 메모리의 접근성을 높이기 위한 중간 메모리
- 파이프라인 기법
  - 다수의 명령어들을 조립공정과 같은 방식으로 조금씩 처리

기본 개념 – 마이크로 프로세서의 내부

- 블록도

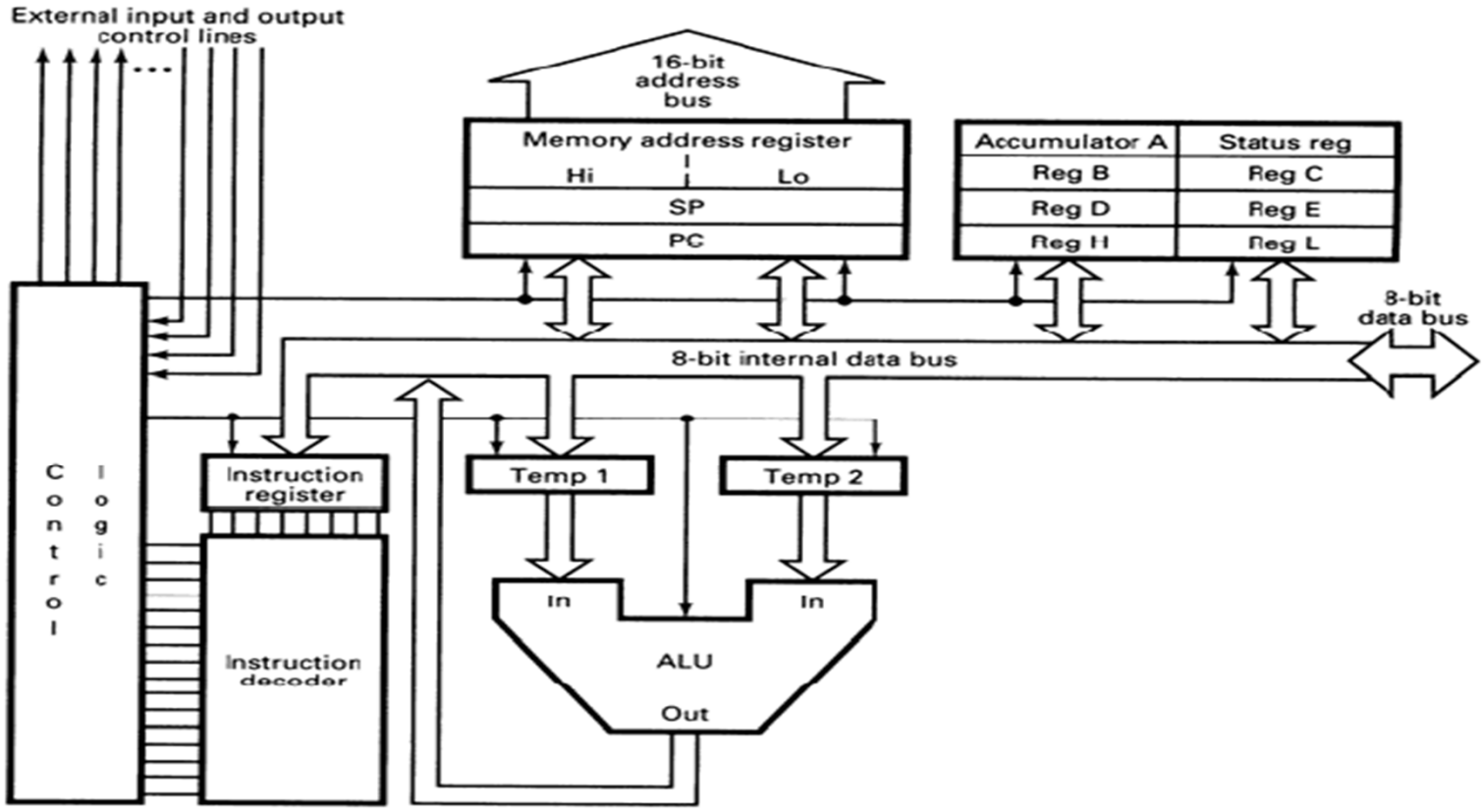


Fig. 5-1 A block diagram for the microprocessor used as a model in this chapter. This 8-bit microprocessor is used in this chapter to illustrate basic microprocessor features. These features will be found in many different forms as the more complex actual microprocessors are examined in later chapters.

기본 개념 – 마이크로 프로세서의 레지스터

- 일시적으로 데이터 워드를 저장하기 위해 충분한 비트를 가진 데이터 래치

- 누산기, 프로그램 카운터, 스택 포인터, 상태 레지스터, 범용 레지스터, 메모리 어드레스 레지스터, 관련 논리, 명령 레지스터 및 임시 레지스터

기본 개념 – 마이크로 프로세서의 레지스터

- ALU (Arithmetic Logic Unit)
  - 중앙처리장치의 일부로서 컴퓨터 명령어 내에 있는 연산자들에 대해 연산과 논리동작을 담당
- 누산기 (Accumulator)
  - 연산기의 입출력 데이터를 임시적으로 기억하는 연산용 레지스터
- 프로그램 카운터 (Program Counter)
  - 프로그램의 수행 순서를 제어하는 레지스터로, 다음에 실행할 명령의 번지 기억
- 스택 포인터
  - 프로그램이 돌아갈 장소를 기억하는 장소
- 상태 레지스터
  - CPU에서 사용되는 연산에 관련된 여러 가지 상태 정보를 기억하기 위해 사용되는 레지스터

기본 개념 – 마이크로 프로세서의 레지스터

- 범용 레지스터
  - 여러 목적으로 사용될 수 있는 레지스터 즉 필요에 따라 누산기, 기억 레지스터, 주소 레지스터, 색인 레지스터 등의 기능을 수행하는 레지스터
- 메모리 주소 레지스터
  - 명령어를 가져올 메모리 주소를 지정
- 명령 레지스터
  - 마이크로프로세서가 현재 실행하고 있는 명령어를 기억
- 임시 데이터 레지스터
  - 누산기의 저장장소를 제공

AVR 마이크로 컨트롤러의 개요 - 탄생

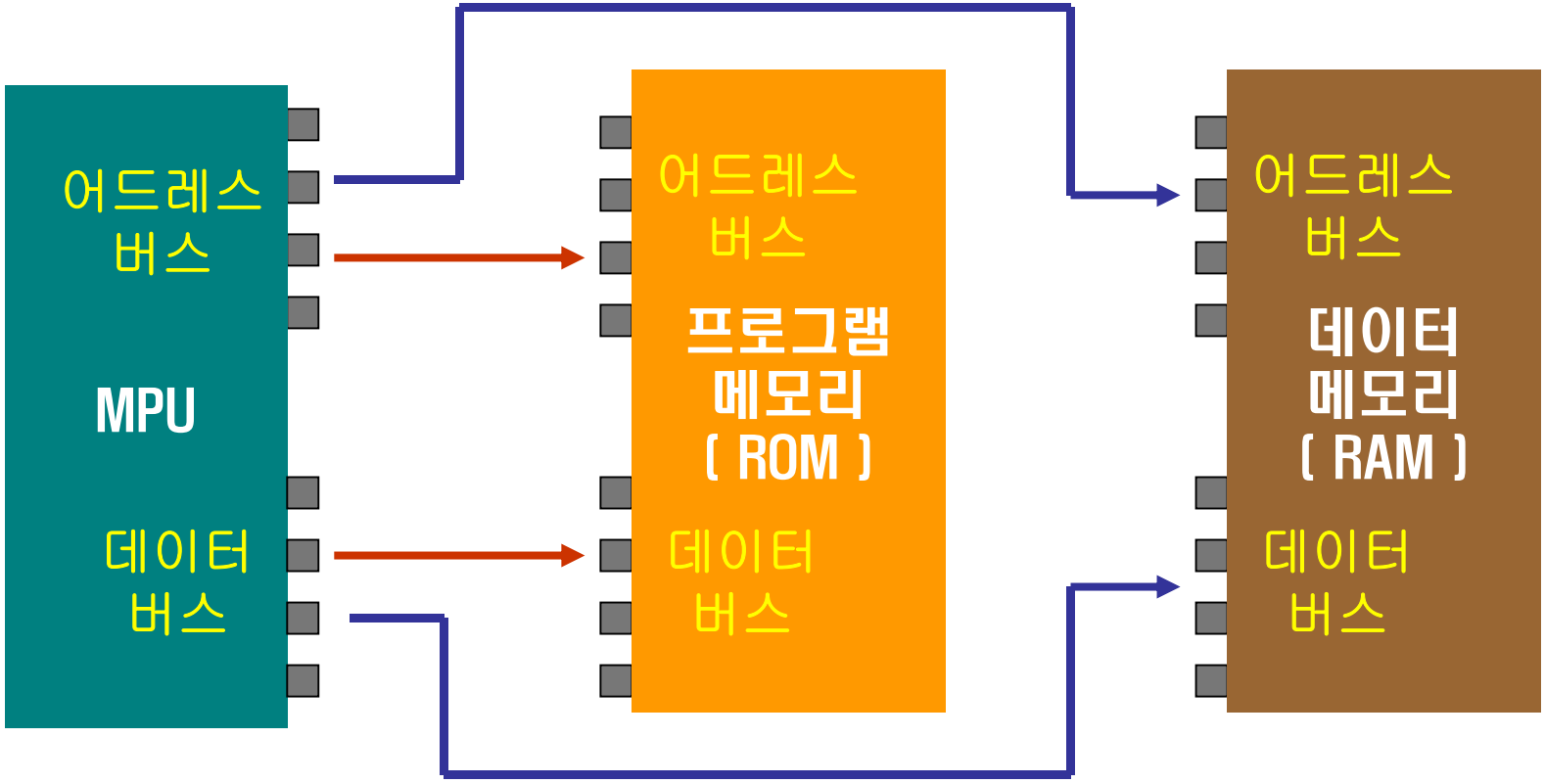
- Atmel사에서 1997년 처음 발표한 8bit의 제어용 마이크로프로세서
- Alf-Egil Bogen과 Vegard Wollan의 진보된 RISC 기술을 기반으로 설계 (AVR)
  - 혹자는 Advanced Virtual RISC의 약어라고 말함
- 하버드 구조와 파이프라인 처리 방식을 기반으로 하는 RISC기술을 적용하여 매우 빠른 처리 속도를 자랑
- 칩 내에 프로그램 코드용으로 플래시 메모리를 내장
- 사용자 프로그램을 쉽게 다운로드 할 수 있는 ISP(In-System Programming)방식을 적용

AVR 마이크로 컨트롤러의 개요 – 탄생

- 용어설명(하버드 구조)
  - 프로그램 메모리와 데이터 메모리가 분리된 형태를 의미
  - PIC, AVR이나 DSP와 같은 마이크로 컨트롤러에 사용
  - 프로그램 메모리와 데이터 메모리에서 주소 버스 및 데이터 버스가 완전히 분리된 것
  - 프로그램 메모리와 데이터 메모리의 주소 크기, 데이터 크기 등이 완전히 서로 다름
  - 명령어들은 프로그램 메모리에 저장하고 데이터들은 데이터 메모리에 저장한 후 프로그램 수행

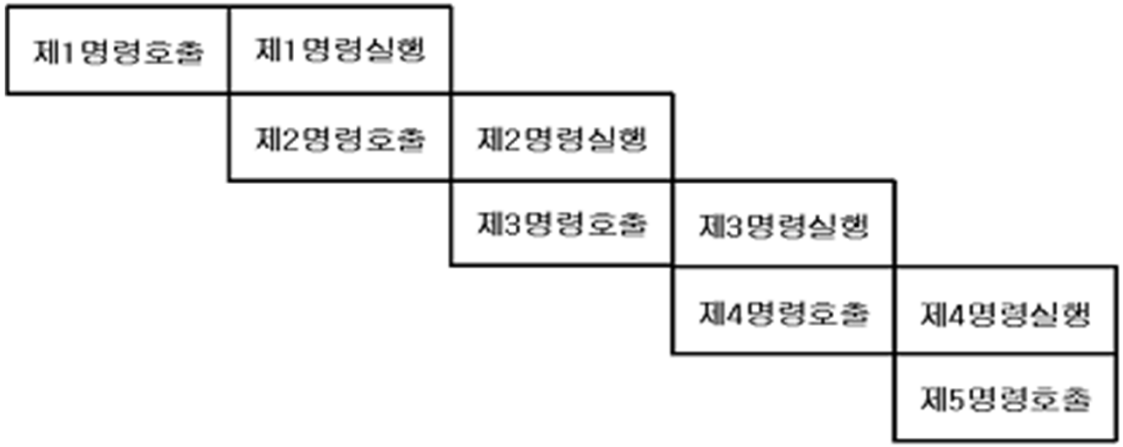
AVR 마이크로 컨트롤러의 개요 – 탄생

- 용어설명(하버드 구조)



AVR 마이크로 컨트롤러의 개요 – 탄생

- 용어설명(파이프라인)
  - ATmega128 RISC(Reduced Instruction Set Code)구조
  - 복합 명령어를 쓰지 않고, 극히 간단한 명령어만을 사용함으로써 명령어의 수를 필요한 최소한으로 줄임
  - 주소지정방식 또한 필요한 최소한에 국한하는 컴퓨터 구조
  - 2단 명령 파이프라인은 명령어 수와 주소 지정방식의 수가 적기 때문에 제 1명령어의 실행이 이루어지는 동시에 제 2명령어의 호출이 시작



<파이프 라인 구조>

AVR 마이크로 컨트롤러의 개요 – 특징

- 명령어는 16bit 버스 폭의 하드웨어로 처리
- 데이터는 8bit 워드 길이를 기반으로 하는 특이한 8bit 마이크로 컨트롤러
- 진보된 RISC 구조와 파이프라인 처리 방식을 사용
- 소비전력이 매우 적으며 대부분의 명령을 단일 클럭 사이클에 실행
- 32개의 범용 레지스터를 가지는 레지스터 중심형 구조

AVR 마이크로 컨트롤러의 개요 – 특징

- 단순한 어드레싱 모드와 간결한 프로그램 코드 사이즈를 가지도록 하는 등 C언어 프로그램에 매우 유리하도록 설계
- 하버드 구조를 사용하여 프로그램 메모리와 데이터 메모리를 동시에 액세스 할 수 있으므로 성능이 향상
- 프로그램 메모리는 모델에 따라 1KB에서 256KB를 가짐
- 프로그램 메모리는 MCU 소자에 내장되므로 외부에는 프로그램 메모리를 접속 할 수 없음

AVR 마이크로 컨트롤러의 개요 – 특징

- 직렬통신 방식의 ISP 기능을 이용 사용자가 쉽게 프로그램을 다운로드
- 플래시 메모리는 10,000번까지 반복하여 다시 프로그래밍 가능
- 64바이트에서 4KB까지 EEPROM을 내장, 데이터 백업이 가능
- EEPROM은 100,000번까지 반복하여 프로그래밍 가능
- 데이터 메모리로서 64바이트에서 8KB까지 용량이 큰 SRAM을 내장
- 병렬 I/O 포트, 내장 시스템 클럭 발진기, 8비트 타이머, 16비트 타이머, 워치독 타이머, UART 또는 USART, SPI, TWI, 아날로그 비교기, 10비트 A/D 컨버터, PWM 출력, RTC(Real Time Counter), BOD(Brown Out Detector) 등 매우 다양한 I/O 기능

AVR 마이크로 컨트롤러의 개요 – 특징

- 외부에 버스를 가지는 모델에서는 이 외부 시스템 버스를 이용하여 데이터 메모리 또는 I/O 디바이스를 확장할 수 있으나, 프로그램 메모리는 확장하지 못함
- 다양한 인터럽트 소스와 인터럽트 처리 기능
- 약 90~135개 정도의 명령을 가짐
- 고성능인 상위 모델에서는 하드웨어 곱셈기를 내장하고 있어 2클럭 사이클에 실행되는 곱셈 명령을 사용할 수 있음
- 슬립 모드라고 불리는 여러 가지의 파워절약 모드를 가짐
- 다양한 응용 분야에 적합한 모델을 선택적으로 사용할 수 있도록 기능 및 성능에 따라 ATtiny, AT90, ATmega 등 3가지 패밀리가 있음

AVR 마이크로 컨트롤러의 개요 - 종류

- ATtiny 패밀리
  - tiny 패밀리라고도 하며, 모든 모델명이 ATtiny로 시작
  - 내부에 1KB~2KB의 플래시 메모리
  - 대량으로 생산되는 간단한 응용분야에 사용
  - 가격이 낮고 성능도 낮음
  - 핀수가 8핀에서 28핀 정도이며, 외형이 아주 작음
  - 외부에 시스템 버스를 가지고 있지 않은 경우가 대부분
  - 외부에서 수정 발진자를 접속하는 것도 부담이 되므로 외부에는 클럭 입력 단자가 없음
  - 시스템 클럭을 발생하는 전체 발진회로를 소자에 내장하는 경우도 있음

AVR 마이크로 컨트롤러의 개요 – 종류

- ATtiny 패밀리

Part	FLASH (KB)	EEPROM(Bytes)	RAM(Bytes)	Instructions	I/O Pins	Interrupts	Ext. Interrupts	SPI	UART	8-bit Timer	16-bit timer	PWM	Watchdog Timer	RTC Timer	Analog Comp	10-bit A/D Channels	On Chip Oscillator	Brown Out Detector	In System Programming	Vcc(V)	Clock speed(MHz)	Packages	Part
ATtiny11L	1	-	-	90	6	4	1[1]	-	-	1	-	-	Y	-	Y	-	Y	-	Y[3]	2.7-5.5	0-2	8-Pin DIP 8-Pin SOIC	ATtiny11L
ATtiny11	1	-	-	90	6	4	1[1]	-	-	1	-	-	Y	-	Y	-	Y	-	Y[3]	4.0-5.5	0-6	8-Pin DIP 8-Pin SOIC	ATtiny11
ATtiny12V[4]	1	64	-	90	6	5	1[1]	-	-	1	-	-	Y	-	Y	-	Y[2]	Y	Y	1.8-5.5	0-1	8-Pin DIP 8-Pin SOIC	ATtiny12V[4]
ATtiny12L[4]	1	64	-	90	6	5	1[1]	-	-	1	-	-	Y	-	Y	-	Y[2]	Y	Y	2.7-5.5	0-4	8-Pin DIP 8-Pin SOIC	ATtiny12L[4]
ATtiny12[4]	1	64	-	90	6	5	1[1]	-	-	1	-	-	Y	-	Y	-	Y[2]	Y	Y	4.0-5.5	0-8	8-Pin DIP 8-Pin SOIC	ATtiny12[4]
ATtiny15L	1	64	-	90	6	8	1[1]	-	-	2	-	1	Y	-	Y	4	Y[2]	Y	Y	2.7-5.5	1.6	8-Pin DIP 8-Pin SOIC	ATtiny15L
ATtiny22L	2	128	128	120	5	2	1	-	-	1	-	-	Y	-	-	-	Y	-	Y	2.7-6.0	0-4	8-Pin DIP 8-Pin SOIC	ATtiny22L
ATtiny22	2	128	128	120	5	2	1	-	-	1	-	-	Y	-	-	-	Y	-	Y	4.0-6.0	0-8	8-Pin DIP 8-Pin SOIC	ATtiny22
ATtiny28V	2	-	-	90	19	5	2	-	-	1	-	-	Y	-	-	-	Y[2]	-	Y	1.8-3.6	0-1	28-Pin DIP 32-Pin TQFP	ATtiny28V
ATtiny28L	2	-	-	90	19	5	2	-	-	1	-	-	Y	-	-	-	Y[2]	-	Y	2.7-3.6	0-4	28-Pin DIP 32-Pin TQFP	ATtiny28L

AVR 마이크로 컨트롤러의 개요 - 종류

- AT90 패밀리
  - classic 패밀리라고도 하며, 모든 모델명이 AT90으로 시작
  - 내부에 1KB~8KB의 플래시 메모리를 가지고 있으므로 고급언어로도 프로그램이 가능
  - 비교적 큰 512바이트 이하 정도의 EEPROM과 SRAM을 내장
  - ATtiny 패밀리에 비하여 규모가 큰 시스템에 적용하는 것을 겨냥한 모델
  - 외형이 대체로 20핀에서 40핀 정도
  - 두 패밀리에 비하여 특징이나 장점이 뚜렷하지 않고 시장성에서 우위를 차지하지 못해 대부분의 모델이 단종

AVR 마이크로 컨트롤러의 개요 – 종류

- AT90 패밀리

Part	FLASH (KB)	EEPROM(Bytes)	RAM(Bytes)	Instructions	I/O Pins	Interrupts	Ext. Interrupts	SPI	UART	8-bit Timer	16-bit timer	PWM	Watchdog Timer	RTC Timer	Analog Comp	10-bit A/D Channels	On Chip Oscillator	Brown Out Detector	In System Programming	Vcc(V)	Clock speed(MHz)	Packages	Part
AT90S1200	1	64	-	89	15	3	1	-	-	1	-	-	Y	-	Y	-	Y	-	Y	2.7-6.0	0-12	20-Pin DIP 20-Pin SOIC 20-Pin SSOP	AT90S1200
AT90S2313	2	128	128	120	15	10	2	-	1	1	1	1	Y	-	Y	-	-	-	Y	2.7-6.0	0-10	20-Pin DIP 20-Pin SOIC	AT90S2313
AT90LS2323	2	128	128	120	3	2	1	-	-	1	-	-	Y	-	-	-	-	-	Y	2.7-6.0	0-4	8-Pin DIP 8-Pin SOIC	AT90LS2323
AT90S2323	2	128	128	120	3	2	1	-	-	1	-	-	Y	-	-	-	-	-	Y	4.0-6.0	0-10	8-Pin DIP 8-Pin SOIC	AT90S2323
AT90LS2343	2	128	128	120	5	2	1	-	-	1	-	-	Y	-	-	-	Y	-	Y	2.7-6.0	0-4	8-Pin DIP 8-Pin SOIC	AT90LS2343
AT90S2343	2	128	128	120	5	2	1	-	-	1	-	-	Y	-	-	-	Y	-	Y	4.0-6.0	0-10	8-Pin DIP 8-Pin SOIC	AT90S2343
AT90LS2333	2	128	128	120	20	14	2	1	1	1	1	2	Y	-	Y	6	-	Y	Y	2.7-6.0	0-4	28-Pin DIP 32-Pin TQFP	AT90LS2333
AT90S2333	2	128	128	120	20	14	2	1	1	1	1	2	Y	-	Y	6	-	Y	Y	4.0-6.0	0-8	28-Pin DIP 32-Pin TQFP	AT90S2333
AT90S4414	4	256	256	120	32	11	2	1	1	1	1	2	Y	-	Y	-	-	-	Y	2.7-6.0	0-8	40-Pin DIP 44-Pin PLCC 44-Pin TQFP	AT90S4414
AT90LS4433	4	256	128	120	20	14	2	1	1	1	1	2	Y	-	Y	6	-	Y	Y	2.7-6.0	0-4	28-Pin DIP 32-Pin TQFP	AT90LS4433
AT90S4433	4	256	128	120	20	14	2	1	1	1	1	2	Y	-	Y	6	-	Y	Y	4.0-6.0	0-8	28-Pin DIP 32-Pin TQFP	AT90S4433
AT90LS4434	4	256	256	120	32	15	2	1	1	2	1	3	Y	Y	Y	8	-	-	Y	2.7-6.0	0-4	40-Pin DIP 44-Pin PLCC 44-Pin TQFP	AT90LS4434
AT90S4434	4	256	256	120	32	15	2	1	1	2	1	3	Y	Y	Y	8	-	-	Y	4.0-6.0	0-8	40-Pin DIP 44-Pin PLCC 44-Pin TQFP	AT90S4434
AT90S8515	8	512	512	120	32	11	2	1	1	1	1	2	Y	-	Y	-	-	-	Y	2.7-6.0	0-8	40-Pin DIP 44-Pin PLCC 44-Pin TQFP	AT90S8515
AT90C8534	8	512	256	120	15	7	2	-	-	1	1	-	Y	-	-	6	-	-	-	3.3-6.0	0-1.5	48-Pin VQFP	AT90C8534
AT90LS8535	8	512	512	120	32	15	2	1	1	2	1	3	Y	Y	Y	8	-	-	Y	2.7-6.0	0-4	40-Pin DIP 44-Pin PLCC 44-Pin TQFP	AT90LS8535
AT90S8535	8	512	512	120	32	15	2	1	1	2	1	3	Y	Y	Y	8	-	-	Y	4.0-6.0	0-8	40-Pin DIP 44-Pin PLCC 44-Pin TQFP	AT90S8535
ATmega83L	8	512	512	128	32	17	2	1	1	2	1	4	Y	Y	Y	8	Y	Y	Y[6]	2.7-3.6	0-4	40-Pin DIP 44-Pin PLCC 44-Pin TQFP	ATmega83L
ATmega83	8	512	512	128	32	17	2	1	1	2	1	4	Y	Y	Y	8	Y	Y	Y[5]	4.0-5.5	0-6	40-Pin DIP 44-Pin PLCC 44-Pin TQFP	ATmega83

AVR 마이크로 컨트롤러의 개요 – 종류

- ATmega 패밀리
  - mega 패밀리라고도 하며, 모델명이 Atmega로 시작
  - 내부에 8KB~256KB의 플래시 메모리
  - 가장 규모가 크고 성능이 높은 응용분야에 사용
  - 가격이 높고 기능이나 성능도 높음
  - 패키지도 28~100핀 정도로 핀 수가 상당히 많음
  - 성능이 우수한 모델들은 20MHz 클럭에서 20MIPS의 명령 처리 속도를 갖음
  - 내장하고 있는 플래시 메모리의 용량에 따라 기본 모델이 ATmega8, ATmega16, ATmega32, ATmega64, ATmega128, ATmega256 등으로 정착되어감

AVR 마이크로 컨트롤러의 개요 – 종류

❑ ATmega 패밀리

ATmega8L	8	512	1K	130	23	16	2	1	1[5]	1	Y	2	1	3	Y	Y	Y	8	Y[2]	Y	Y	Y	2.7-5.5	0-8	28-Pin DIP 32-Pin MLF 32-Pin TQFP	Q1'02
ATmega8	8	512	1K	130	23	16	2	1	1[5]	1	Y	2	1	3	Y	Y	Y	8	Y[2]	Y	Y	Y	4.0-5.5	0-16	28-Pin DIP 32-Pin MLF 32-Pin TQFP	Q1'02
ATmega161L	16	512	1K	130	35	20	3	1	2	-	Y	2	1	4	Y	Y	Y	-	-	Y	Y	Y	2.7-5.5	0-4	40-Pin DIP 44-Pin TQFP	now
ATmega161	16	512	1K	130	35	20	3	1	2	-	Y	2	1	4	Y	Y	Y	-	-	Y	Y	Y	4.0-5.5	0-8	40-Pin DIP 44-Pin TQFP	now
ATmega163L	16	512	1K	130	32	17	2	1	1	1	Y	2	1	3	Y	Y	Y	8	Y[2]	Y	Y	Y	2.7-5.5	0-4	40-Pin DIP 44-Pin TQFP	now
ATmega163	16	512	1K	130	32	17	2	1	1	1	Y	2	1	3	Y	Y	Y	8	Y[2]	Y	Y	Y	4.0-5.5	0-8	40-Pin DIP 44-Pin TQFP	now
ATmega16L	16	512	1K	130	32	17	3	1	1[5]	1	Y	2	1	3	Y	Y	Y	8	Y[2]	Y	Y	Y	2.7-5.5	0-8	40-Pin DIP 44-Pin TQFP	Q1'02
ATmega16	16	512	1K	130	32	17	3	1	1[5]	1	Y	2	1	3	Y	Y	Y	8	Y[2]	Y	Y	Y	4.0-5.5	0-16	40-Pin DIP 44-Pin TQFP	Q1'02
ATmega323L	32	1K	2K	130	32	19	3	1	1	1	Y	2	1	4	Y	Y	Y	8	Y[2]	Y	Y	Y	2.7-5.5	0-4	40-Pin DIP 44-Pin TQFP	now
ATmega323	32	1K	2K	130	32	19	3	1	1	1	Y	2	1	4	Y	Y	Y	8	Y[2]	Y	Y	Y	4.0-5.5	0-8	40-Pin DIP 44-Pin TQFP	now
ATmega103L	128	4K	4K	121	48	16	6	1	1	-	-	2	1	4	Y	Y	Y	8	-	-	Y	-	2.7-3.6	0-4	64-Pin TQFP	now
ATmega103	128	4K	4K	121	48	16	8	1	1	-	-	2	1	4	Y	Y	Y	8	-	-	Y	-	4.0-5.5	0-6	64-Pin TQFP	now
ATmega128L	128	4K	4K	133	48	27	8	1	2[5]	1	Y	2	2	6+2	Y	Y	Y	8	Y[2]	Y	Y	Y	2.7-5.5	0-8	64-Pin TQFP	Eng. Samples Q4
ATmega128	128	4K	4K	133	48	27	8	1	2[5]	1	Y	2	2	6+2	Y	Y	Y	8	Y[2]	Y	Y	Y	4.0-5.5	0-16	64-Pin TQFP	Eng. Samples Q4

[1] One external interrupt + interrupt and wake-up on pin change(all I/O pins)
[2] High accuracy (5%) internal RC oscillator with programmable speed.
[3] Requires 12V signal on RESET pin during programming
[4] Compatible to I2C
[5] Programmable Serial USART
[6] interrupt on pin low on 8 pins

Atmega128의 기본구조와 기능 - 특징

- 고성능이면서 저전력의 8비트 마이크로 컨트롤러

- 진보된 RISC구조를 사용
  - 133개의 명령세트를 가지며, 대부분이 1클럭 사이클에 실행
  - 32개의 8비트 범용 작업레지스터
  - 완전 정적인 기능
  - 16MHz에서 평균적으로 16MIPS의 명령처리 속도
  - 2싸이클에 실행되는 곱셈기

Atmega128의 기본구조와 기능 - 특징

- 비휘발성 프로그램-데이터 메모리
  - 128KB의 ISP방식 프로그램용 플래시 메모리
  - 4K Bytes EEPROM
  - 4K Bytes Internal SRAM
  - 쓰기/삭제 가능: 플래시 메모리 10,000번 EEPROM 100,000번
  - 독립적인 lock bit을 가지는 부수적인 Boot code 분할
- one chip boot program에 의한 내부 system programming
- True read-while-write 기능
  - 64K Bytes 이상의 임의의 외부 메모리공간
  - software 안전을 위한 programming lock 기능
  - ISP를 위한 SPI (Serial Peripheral Interface) 핀을 가짐

Atmega128의 기본구조와 기능 - 특징

- 용어설명 : SPI (Serial Peripheral Interface) 직렬통신
  - MOTOROLA사에 의하여 개발된 근거리용 직렬통신 규격
  - MOSI, MISO, SCK, SS등 4개의 통신선을 사용
  - 고속 동기식 직렬 통신 방식

Atmega128 특징

- JTAG (IEEE std. 1149.1 Compliant) Interface
  - Boundary-scan Capabilities According to the JTAG Standard
  - 온칩 디버그 기능 수행
    장점 : 정확한 전기적, 시간적 특성을 얻게 됨
  - Flash, EEPROM, Fuses and Lock Bits의 프로그래밍을 위해 JTAG Interface 기능 가짐

Atmega128의 기본구조와 기능 - 특징

❑ 주변장치 특징

- 2개의 8bit Timer/Counter
- 2개의 16bit Timer/Counter
- 분리된 Oscillator를 가지는 Real-Time Counter
- 2개의 8-bit PWM채널
- 2~16bit 프로그램 분석 가능한 6개의 PWM채널
- 출력비교 단자와 관련되어 동작
- 8개 채널, 10-bit A/D 컨버터
- Byte-oriented Two-wire Serial Interface
- 2개의 프로그램이 가능한 serial USART
- Master/Slave SPI Serial Interface
- 독립적인 내장 발진기에 의한 워치독 타이머
- on-chip 아날로그 비교기

Atmega128의 기본구조와 기능 - 특징

- 특별한 Microcontroller의 특징
  - Power-on Reset 과 프로그램 가능한 Brown-out Detection(BOD : EEPROM을 안전하게 보호)
  - 내부 측정된 RC Oscillator
  - 외부, 내부 인터럽트 소스
  - 6가지 슬립모드 : idle, Power-Down, Standby and Extended Standby, ADC-noise 제거기, Power- save
  - Software 선택 가능한 Clock 주파수
  - ATmega103 호환 모드
  - Global pull-up 금지

Atmega128의 기본구조와 기능 – 특징

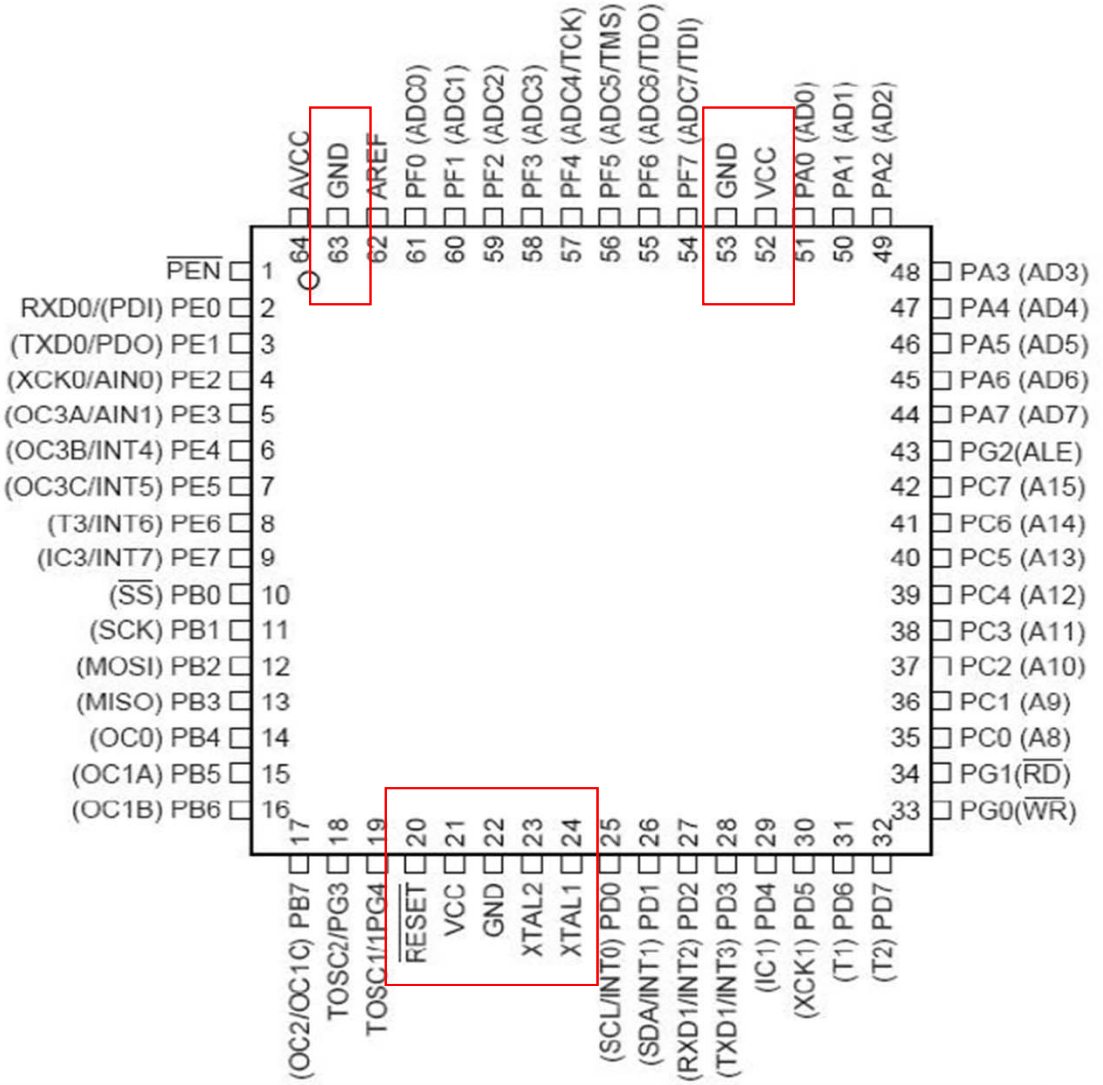
- I/O and Packages
  - 53 프로그램 가능한 I/O line
  - 64-lead TQFP and 64-pad QFN/MLF
- 동작 Voltages
  - 2.7 – 5.5V for ATmega128L
  - 4.5 – 5.5V for ATmega128
- Speed 등급
  - 0 – 8 MHz for ATmega128L
  - 0 – 16 MHz for ATmega128

Thin Quad Flat Pack,
a type of IC packaging designed for use
in space-conscious applications such as PC cards.
TQFP-packaged chips are thinner than
PQFP-packaged chips,
with a thickness of either 1.0mm or 1.4mm



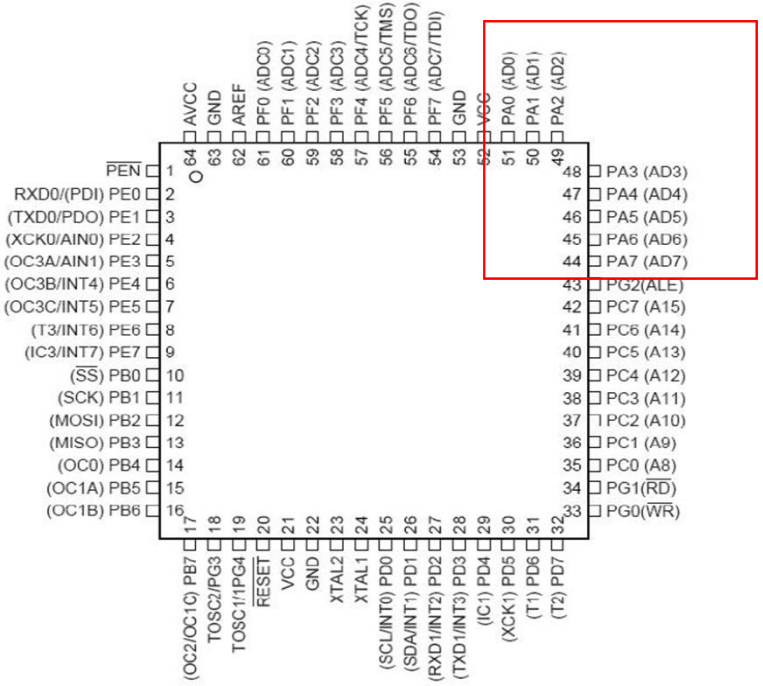
ATmega128 외부 구조

- VCC (21, 52)
  - Digital supply voltage
- GND (22, 53, 63)
  - Ground
- XTAL1, XTAL2 (23, 24)
  - 크리스탈 오실레이터 입/출력 Pin
- RESET (20)
  - 리셋 입력 Pin
  - 50ns 이상 Low level 입력이 지속되면 리셋 발생



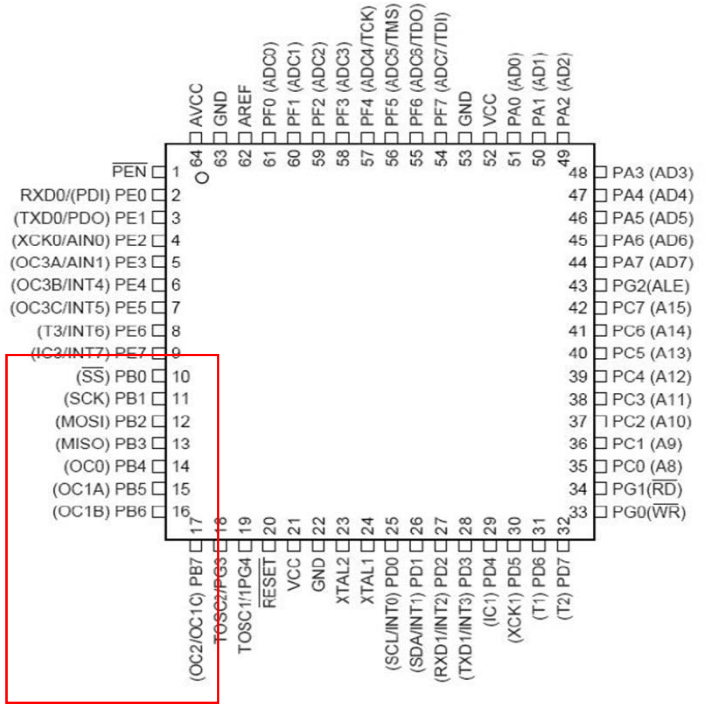
ATmega128 외부 구조(Cont)

- Port A(PA7.. PA0/AD7..AD0)
  - 내부적으로 풀업(pull-up)된 8bit 양방향 병렬 포트
  - DDRA 레지스터
    입/출력 방향을 결정
  - PINA 레지스터
    데이터 입력
  - PORTA 레지스터
    데이터를 출력
  - 외부 데이터 메모리 액세스를 위한 16비트 어드레스의 하위 8bit로 사용



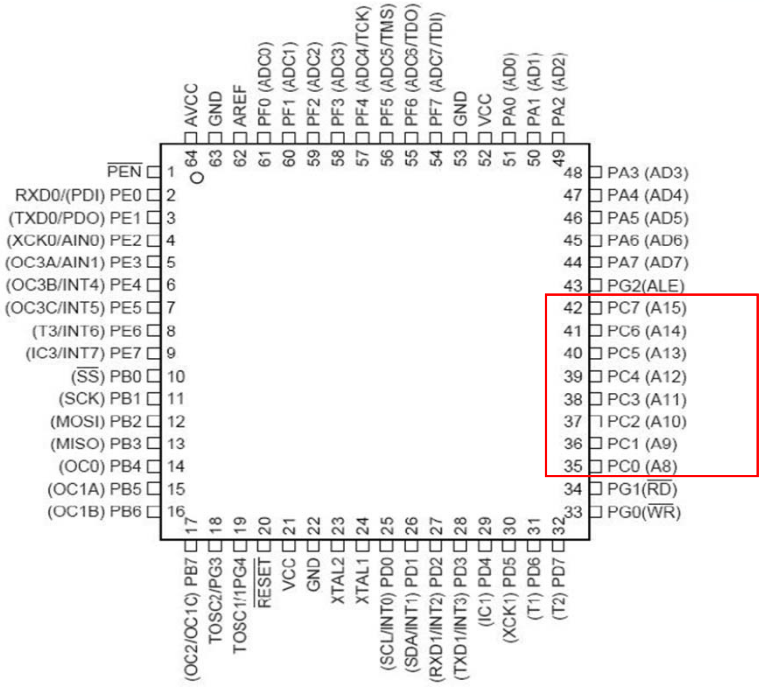
ATmega128 외부 구조(Cont)

- Port B(PB7.. PB0)
  - 내부적으로 풀업(pull-up)된 8bit 양방향 병렬 포트
  - DDRB 레지스터
    입/출력 방향을 결정
  - PINB 레지스터
    데이터 입력
  - PORTB 레지스터
    데이터를 출력



ATmega128 외부 구조(Cont)

- Port C(PA7.. PA0)
  - 내부적으로 풀업(pull-up)된 8bit 양방향 병렬 포트
  - DDRC 레지스터
    입/출력 방향을 결정
  - PINC 레지스터
    데이터 입력
  - PORTC 레지스터
    데이터를 출력
  - 외부 데이터 메모리 액세스를 위한 16비트 어드레스의 상위 8bit로 사용
  - 외부 메모리 영역은 65536개까지 지정 가능(내부에서 사용되는 일부 제외)



ATmega128 외부 구조(Cont)

- Port C(PC7.. PC0)를 이용한 예제

```
 #include <mega128.h>

void Delay(unsigned int);                              // 시간 지연 함수

void main(void)
{
        unsigned char led = 0xFE;       // LED 초기 출력값

        DDRC = 0xFF;                                                    // 포트 C 출력 설정

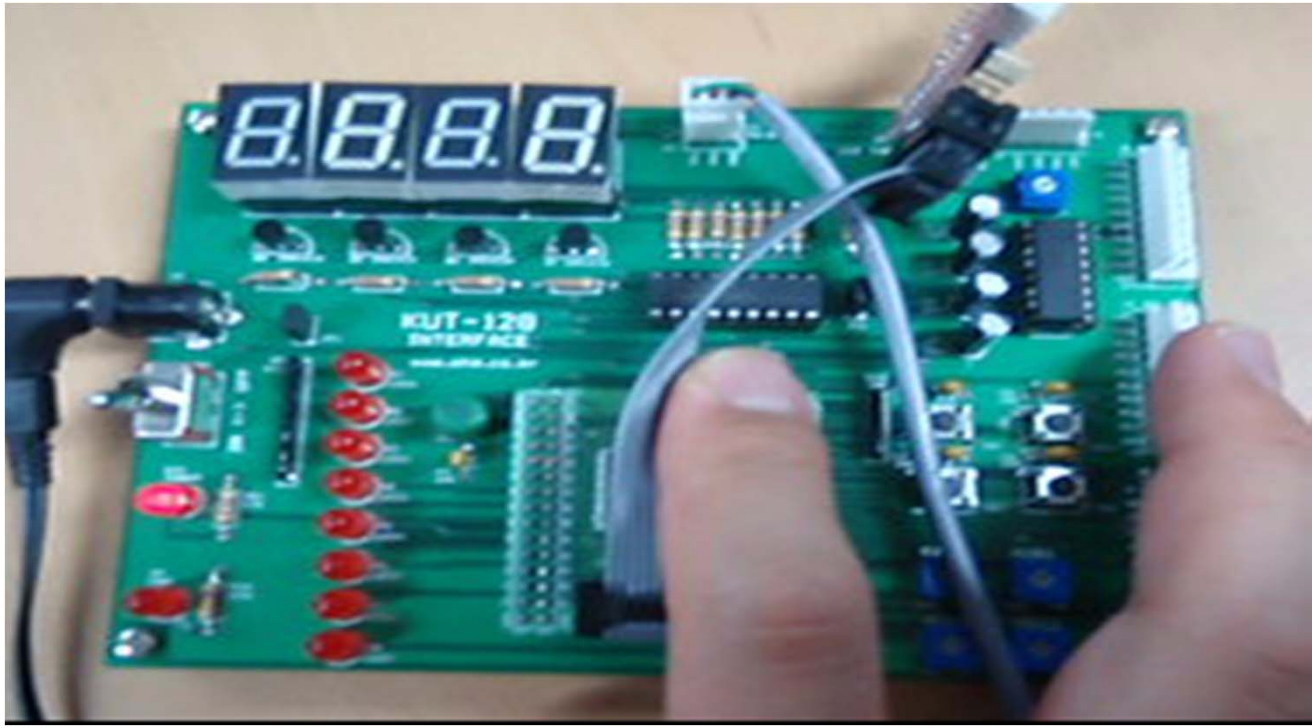
        while(1){
                PORTC = led;                           // 포트 C에 출력
                Delay(5);                              // 시간 지연
                led <<= 1;                             // led 변수의 좌 쉬프트
                led |= 0x01;                           // 쉬프트 후에 0 비트에는 '0'이 입력되므로 '1'로 셋
                if(led == 0xFF) led = 0xFE; // led 변수 초기화
        }
}

// 시간 지연 함수
void Delay(unsigned int cnt)
{
        unsigned int i, j;

        for(i = 0;i < cnt;i++){
                j = 50000;
                while(--j);
        }
}
```

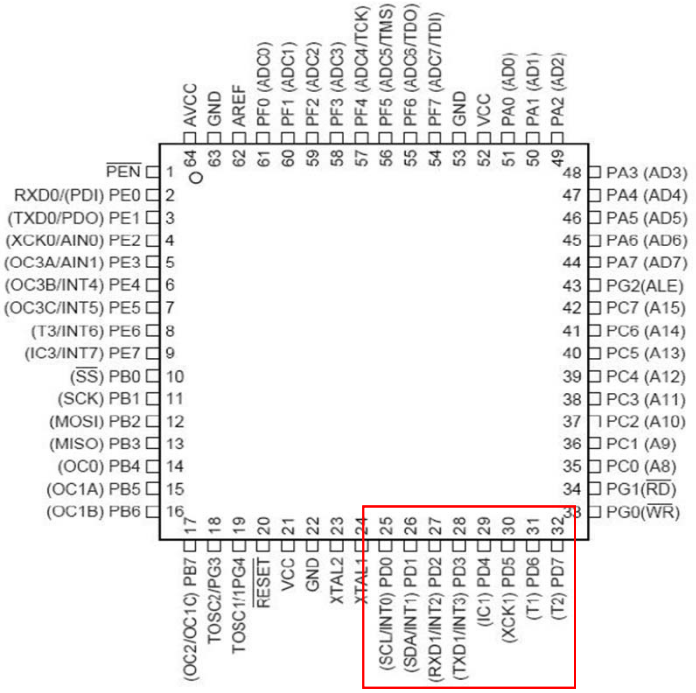
ATmega128 외부 구조(Cont)

- Port C(PC7.. PC0)를 이용한 예제



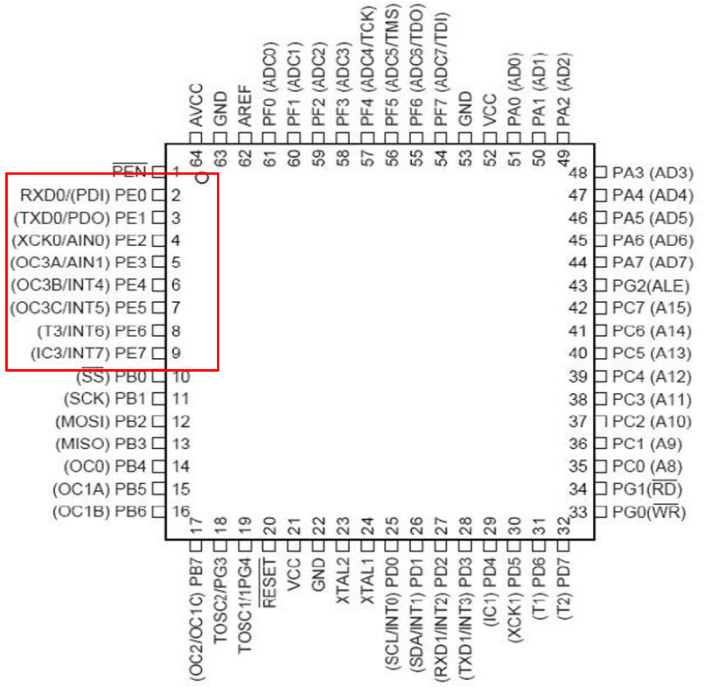
ATmega128 외부 구조(Cont)

- Port D(PD7.. PD0)
  - 내부적으로 풀업(pull-up)된 8bit 양방향 병렬 포트
  - DDRD 레지스터
    입/출력 방향을 결정
  - PIND 레지스터
    데이터 입력
  - PORTD 레지스터
    데이터를 출력



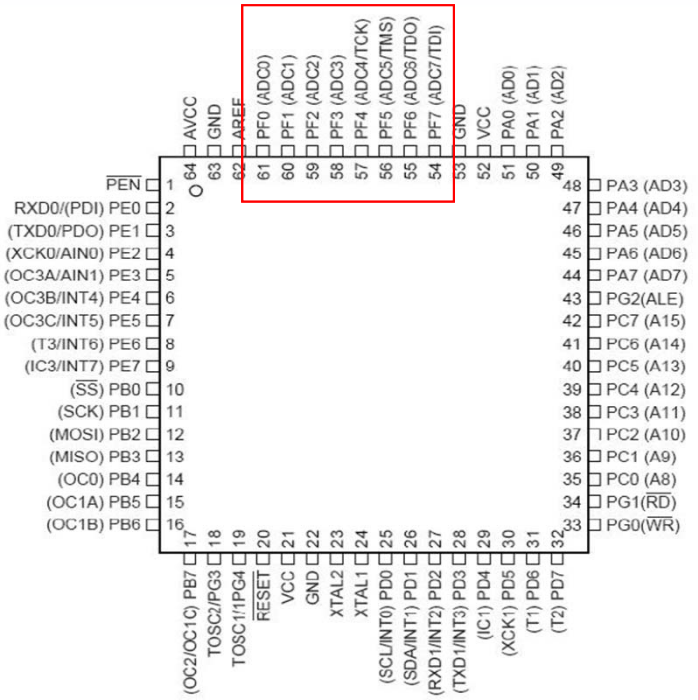
ATmega128 외부 구조(Cont)

- Port E(PE7.. PE0)
  - 내부적으로 풀업(pull-up)된 8bit 양방향 병렬 포트
  - DDRE 레지스터
    입/출력 방향을 결정
  - PINE 레지스터
    데이터 입력
  - PORTE 레지스터
    데이터를 출력



ATmega128 외부 구조(Cont)

- Port F(PF7.. PF0)
  - 내부적으로 풀업(pull-up)된 8bit 양방향 병렬 포트
  - DDRF 레지스터
    입/출력 방향을 결정
  - PINF 레지스터
    데이터 입력
  - PORTF 레지스터
    데이터를 출력



ATmega128 외부 구조(Cont)

- Port A(PA7.. PA0) ~ Port F(PF7.. PF0)

Bit	7	6	5	4	3	2	1	0	
	PORTA7	PORTA6	PORTA5	PORTA4	PORTA3	PORTA2	PORTA1	PORTA0	PORTA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

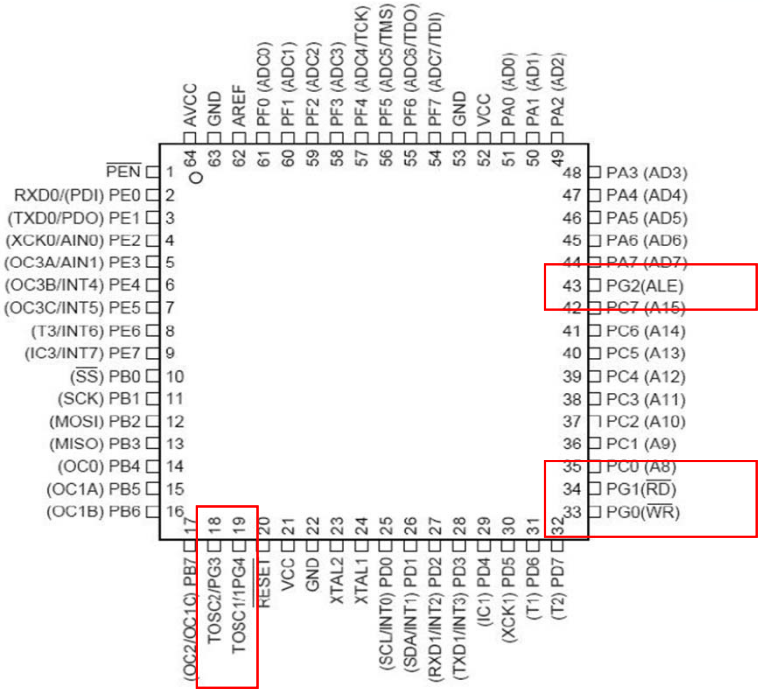
Bit	7	6	5	4	3	2	1	0	
	DDA7	DDA6	DDA5	DDA4	DDA3	DDA2	DDA1	DDA0	DDRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	PINA7	PINA6	PINA5	PINA4	PINA3	PINA2	PINA1	PINA0	PINA
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

<Register description for I/O Port(A~F는 동일)>

ATmega128 외부 구조(Cont)

- Port G(PG4.. PG0)
  - 내부적으로 풀업(pull-up)된 4bit 양방향 병렬 포트
  - DDRG 레지스터
    입/출력 방향을 결정
  - PING 레지스터
    데이터 입력
  - PORTG 레지스터
    데이터를 출력



ATmega128 외부 구조(Cont)

- Port G(PG4.. PG0)

Bit	7	6	5	4	3	2	1	0	
	–	–	–	PORTG4	PORTG3	PORTG2	PORTG1	PORTG0	PORTG
Read/Write	R	R	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	–	–	–	DDG4	DDG3	DDG2	DDG1	DDG0	DDRG
Read/Write	R	R	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	–	–	–	PING4	PING3	PING2	PING1	PING0	PING
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	0	0	0	N/A	N/A	N/A	N/A	N/A	

<Register description for I/O PortG>

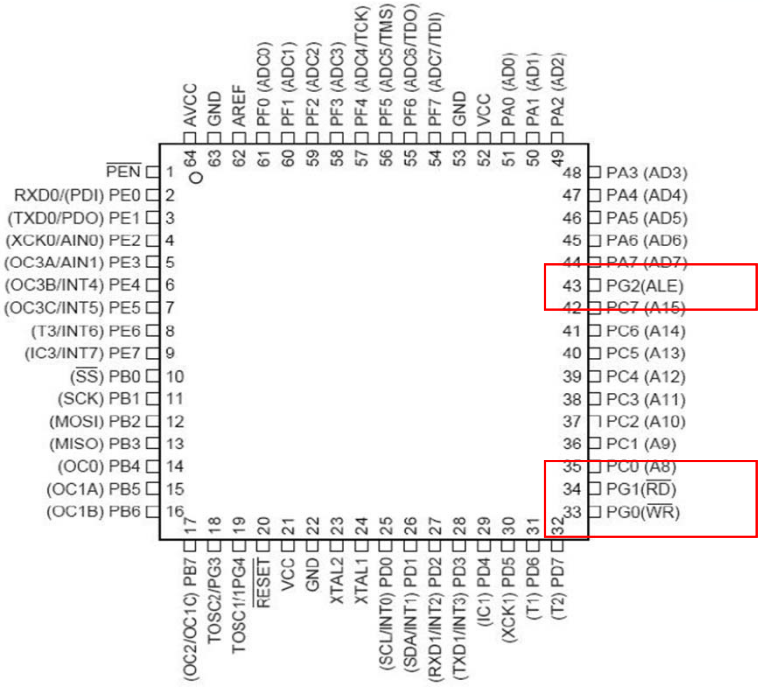
ATmega128 외부 구조(Cont)

- ALE
  - Address Latch Enable

    AD0~AD7에 외부 메모리를 액세스하기 위한 하위 어드레스가 출력되고 있음을 나타내는 신호로서 이를 외부에서 래치시키는데 사용
- RD'
  - Read Strobe

    현재 CPU가 외부 데이터 메모리 영역에서 데이터 read동작을 수행 함을 나타내는 신호
- WR'
  - Write Strobe

    현재 CPU가 외부 데이터 메모리 영역에서 데이터 write동작을 수행 함을 나타내는 신호



ATmega128 외부 구조(Cont)

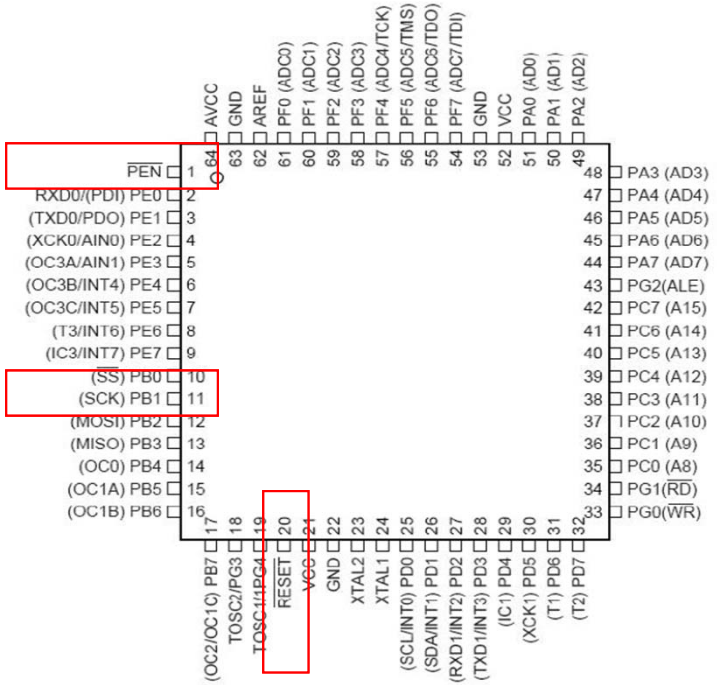
- RESET'
  - CPU 리셋 용도
  - 올바른 동작을 위하여 최소 1.5us이상의 신호폭 필요
- PEN'
  - Programming Enable

    CPU가 SPI 직렬통신에 의한 프로그래밍 모드로 들어가도록 허용하는 기능

    전원 투입에 의한 리셋시에 low상태를 유지함으로써 허용하게 됨

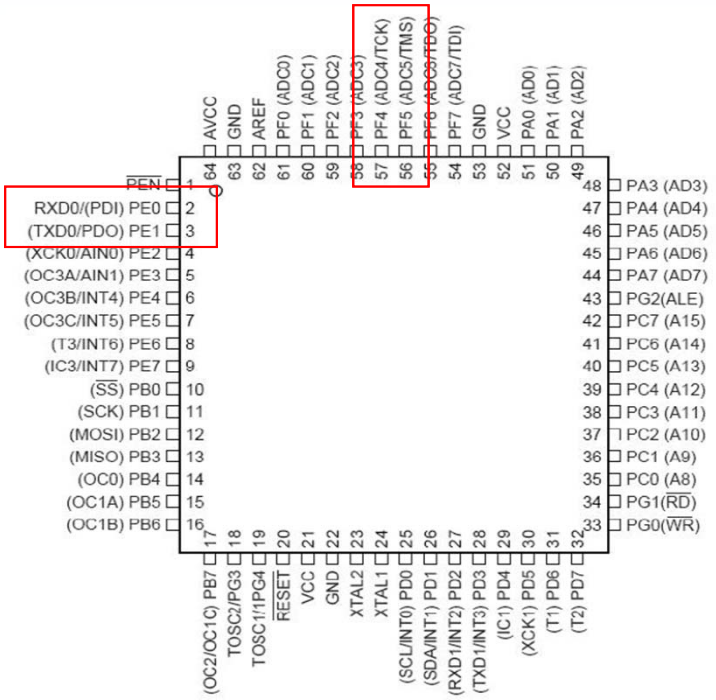
    보통 Vcc에 접속시켜 둠
- SCK
  - Serial Clock

    ISP 프로그래머(master)가 ATmega128(slave)에게 보내는 프로그래밍 신호



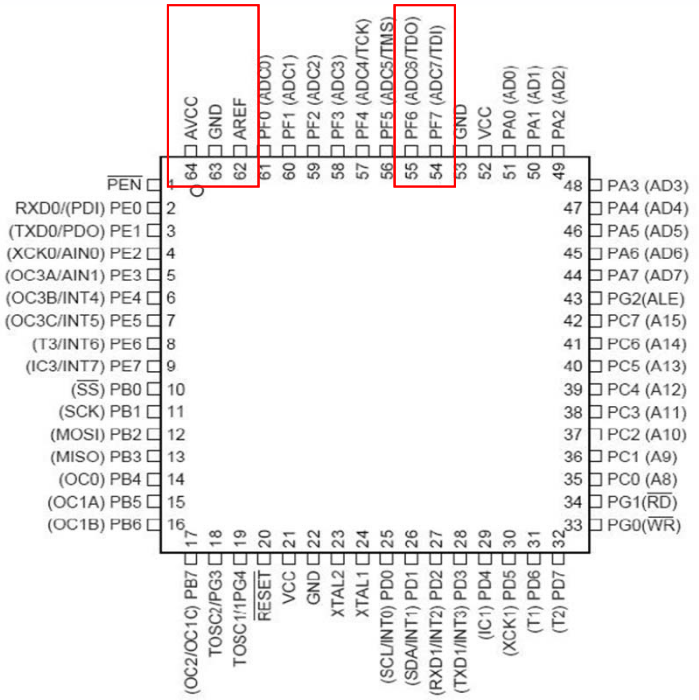
ATmega128 외부 구조(Cont)

- PDI
  - Programming Data Input(MOSI)
    - ISP 프로그래머가 ATmega128에게 보내는 입력 데이터 신호
    - ISP 케이블의 MOSI단자와 연결 함
- PDO
  - Programming Data Output(MISO)
    - ATmega128이 ISP 프로그래머에게 보내는 출력 데이터 신호
    - ISP 케이블의 MISO단자와 연결 함
- TCK
  - JTAG Test Clock
    - JTAG ICE가 ATmega128에게 보내는 클럭 신호
- TMS
  - JTAG TEST Mode Select
    - JTAG ICE가 ATmega128에게 보내는 모드 선택 신호



ATmega128 외부 구조(Cont)

- TDO (55)
  - JTAG Test Data Output
    ATmega128dl JTAG ICE에게 보내는 데이터 신호
- TDI (54)
  - JTAG Test Data Input
    JTAG ICE가 ATmega128에게 보내는 데이터 신호
- AVCC (64)
  - A/D 컨버터 및 포트 F의 전원
    (ADC를 이용하지 않더라도 외부적으로 VCC에 연결되어야 함)
- AGND (63)
  - AVCC에 대한 전원 접지
    (반드시 GND와 접속할 것)
- AREF (62)
  - A/D 컨버터의 기준 전압



ATmega128 외부 구조(Cont)

- INT0~INT7
  - External Interrupt

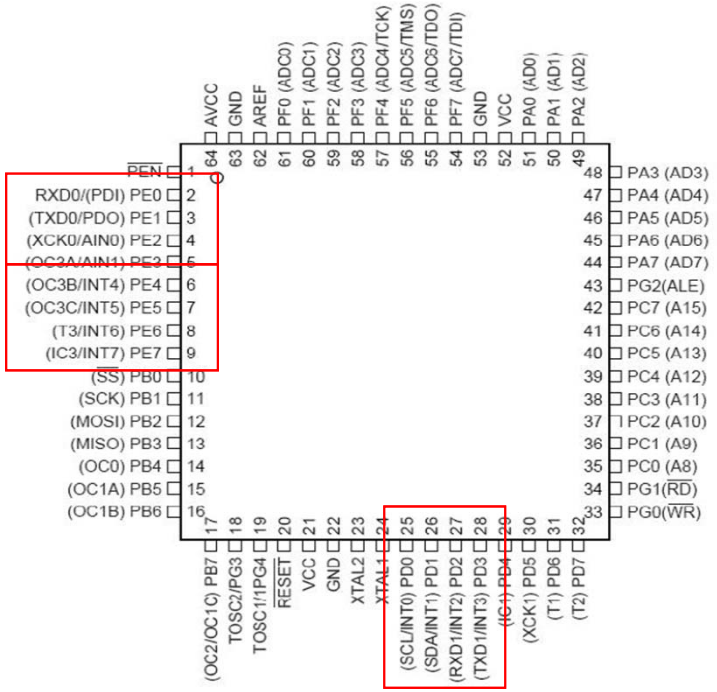
    외부 인터럽트 요청 신호

    외부 인터럽트 제어 레지스터 EICRA, EICRB를 사용하여 인터럽터 신호가 상승/하강 에지 또는 Low level 일 때 인터럽트가 발생하도록 설정 가능
- TXD0/RXD0
  - USART0 Transmit/Receive Data

    직렬통신 포트 USART0의 송수신 데이터 신호
- TXD1/RXD1
  - USART1 Transmit/Receive Data

    직렬통신 포트 USART1의 송수신 데이터 신호
- XCK0
  - USART0 Extenal Clock Input/Output

    직렬통신 포트 USART0가 동기 모드로 동작할 때의 송수신 클럭 신호



ATmega128 외부 구조(Cont)

❑ 외부 인터럽트를 이용한 예제

```
#include <mega128.h>

unsigned char led = 0xFE;

void main(void)
{
        // 포트 초기화
        DDRC = 0xFF;                                    // 포트 C 출력 설정
        DDRE = 0b00000010;          // 포트 E 입력(PE1 출력)

        PORTC = led;                                    // 포트 C에 초기값 출력

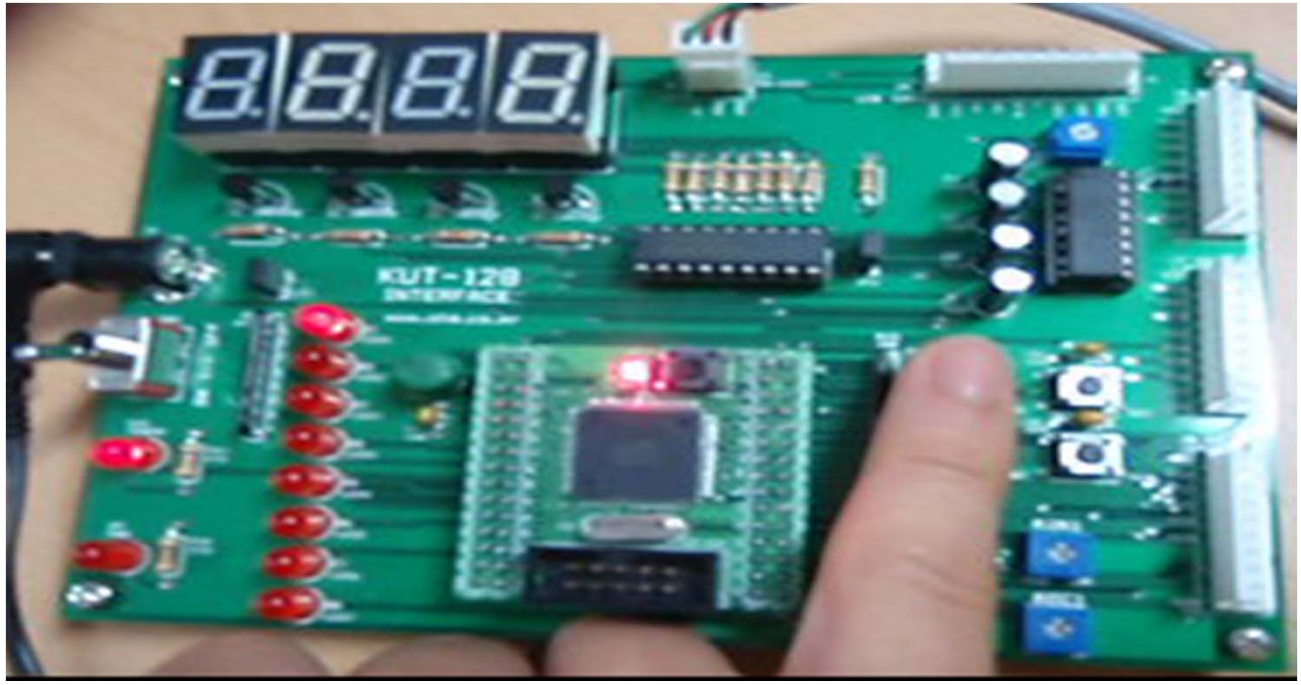
        // 인터럽트 초기화
        EICRB = 0b00000010;         // 외부 인터럽트4 : 하강 에지
        EIMSK = 0b00010000;         // 외부 인터럽트4 인에이블
        SREG = 0x80;                // 전역 인터럽트 인에이블 비트 셋

        while(1);
}

// 외부 인터럽트4 서비스 루틴
interrupt [EXT_INT4] void external_int4(void)
{
        led <<= 1;
        led |= 0b00000001;
        if(led == 0xFF) led = 0xFE;
        PORTC = led;
}
```

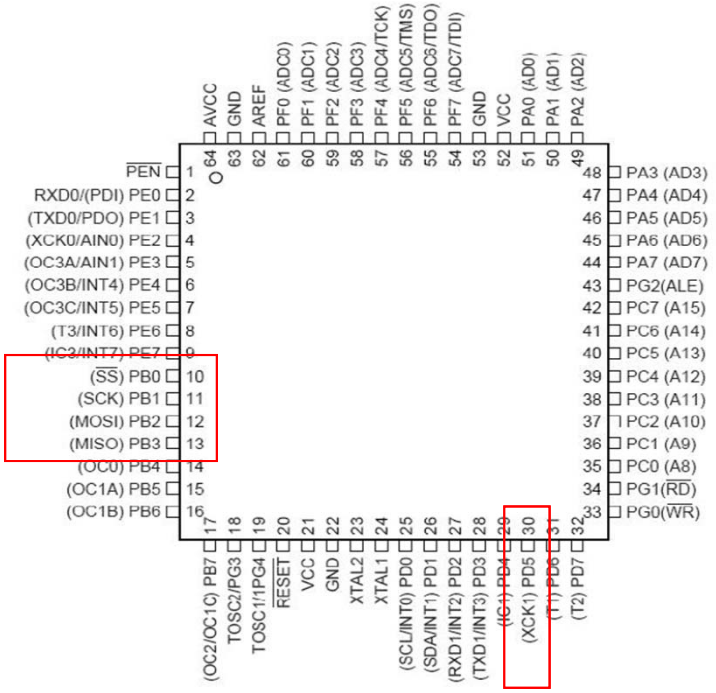
ATmega128 외부 구조(Cont)

- 외부 인터럽트를 이용한 예제



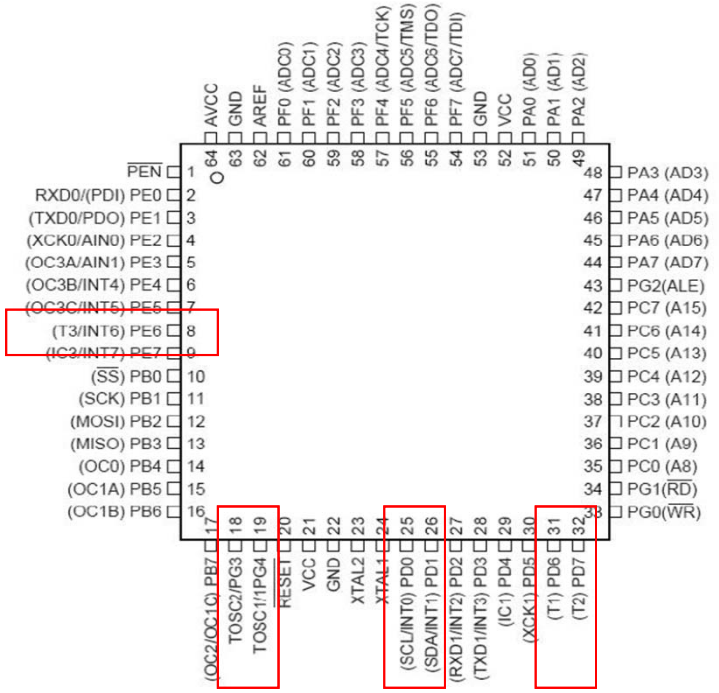
ATmega128 외부 구조(Cont)

- XCK1
  - USART1 Extenal Clock Input/Output
    직렬통신 포트 USART1가 동기 모드로 동작할 때의 송수신 클럭 신호
- SS’
  - SPI Slave Select Input
    SPI 채널의 슬레이브 선택 입력 신호
- SCK
  - SPi Serial Clock
    SPI 채널의 마스터 클록 출력 또는 슬레이브 클록 입력 신호
- MOSI
  - SPI Master Output/Slave Input
    SPI 채널의 마스터 데이터 출력 또는 슬레이브 데이터 입력 신호
- MISO
  - SPI Master Input/Slave Output
    SPI 채널의 마스터 데이터 입력 또는 슬레이브 데이터 출력 신호



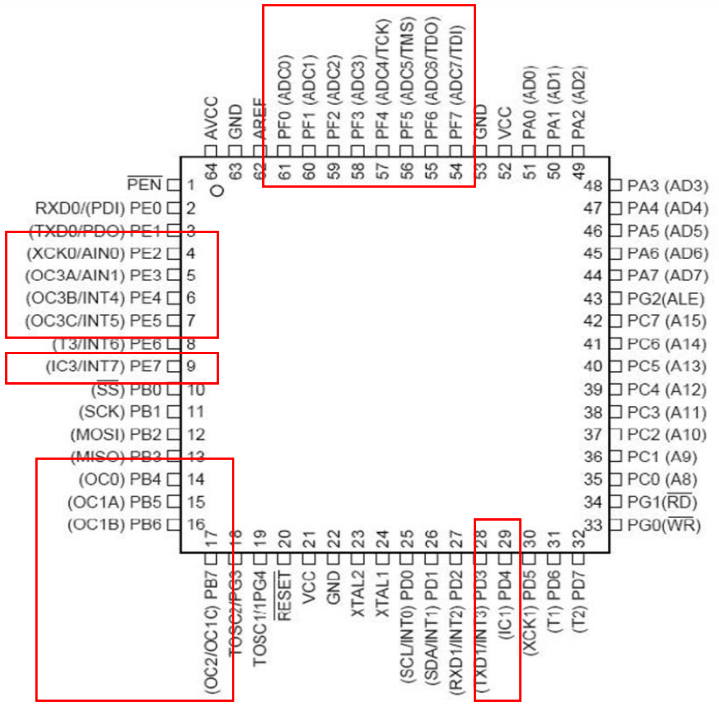
ATmega128 외부 구조(Cont)

- SCL
  - TWI(Two-Wire Interface) Serial Clock
    TWI 채널의 클럭 입출력 신호
- SDA
  - TWI Serial Data
    TWI 채널의 데이터 입출력 신호
- TOSC1/TOSC2
  - Timer/Counter0 RTC Oscillator
    타이머/카운터0의 RTC 기능을 사용할 경우 클럭 발생을 위한 수정발진자 접속 단자
- T1/T2/T3
  - Timer/counter1~3 Clock Input
    타이머/카운터1~3의 클럭 입력 신호



ATmega128 외부 구조(Cont)

- OC0, OC1A~OC1C, OC2, OC3A~OC3C
  - Timer/Counter0,1,2,3 Output Compare Match and PWM Output
    - 각각 타이머/카운터 0, 1, 2, 3의 비교 출력 신호
- ICP1/ICP3
  - Timer/Counter1, 3 Input Capture Pin
    - 각각 타이머/카운터1, 3의 캡쳐 모드에서 트리거 신호로 사용
- ADC0~7
  - ADC Input Channel 0~7
    - 각각 8채널의 10비트 A/D 컨버터에 대한 아날로그 입력 신호
- AIN0/AIN1
  - Analog Comparator Positive/Negative Input
    - 아날로그 비교기의 플러스/마이너스/ 극성 입력 신호



ATmega128 외부 구조(Cont)

❑ 타이머/카운터를 이용한 예제

```
#include <mega128.h>

unsigned char led = 0xFE;

void main(void)
{
        // 포트 초기화
        DDRC = 0xFF;                    // 포트 C 출력으로 설정
        PORTC = led;                    // 포트 C에 초기값 출력

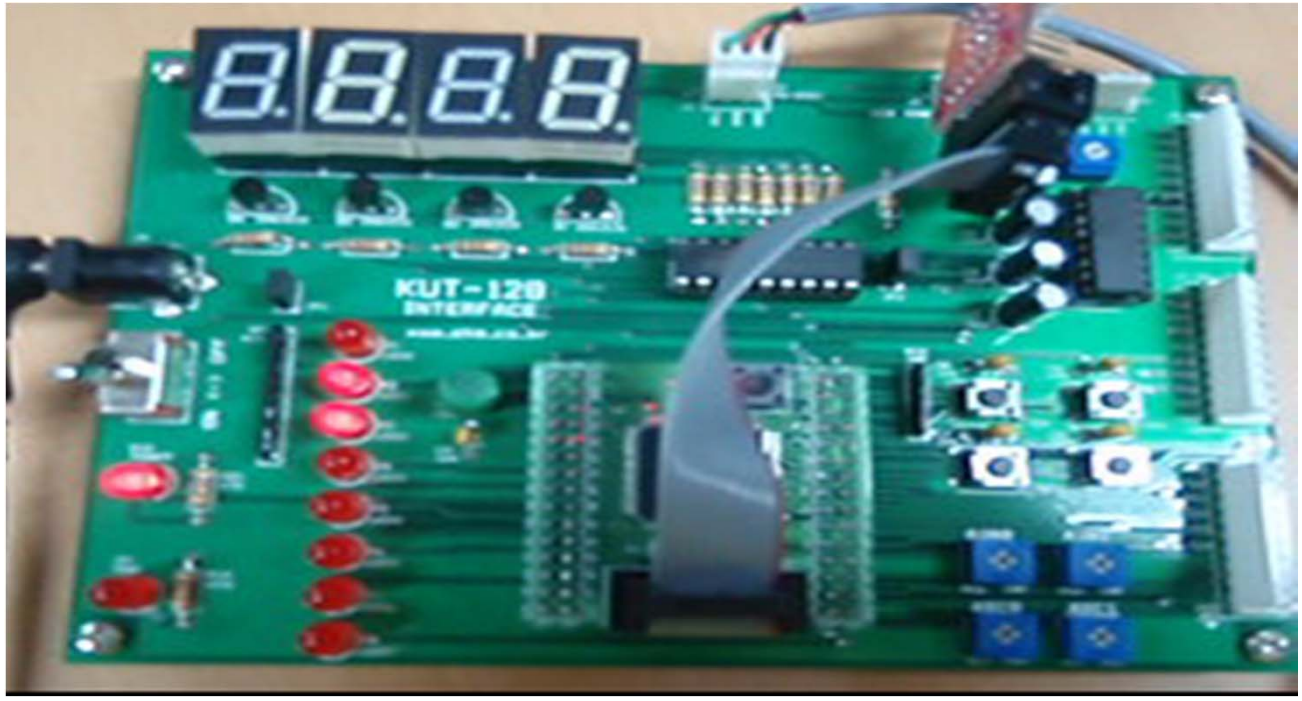
        // 인터럽트 초기화
        TIMSK = 0x01;                   // TOIE0 = 1;
        TCCR0 = 0x07;                   // 일반모드, 프리스케일 = CK/1024
        TCNT0 = 0x00;                   // 타이머/카운터0 레지스터 초기값
        SREG = 0x80;                    // 전역 인터럽트 인에이블 비트 I 셋.

        while(1);                       // 무한 루프
}

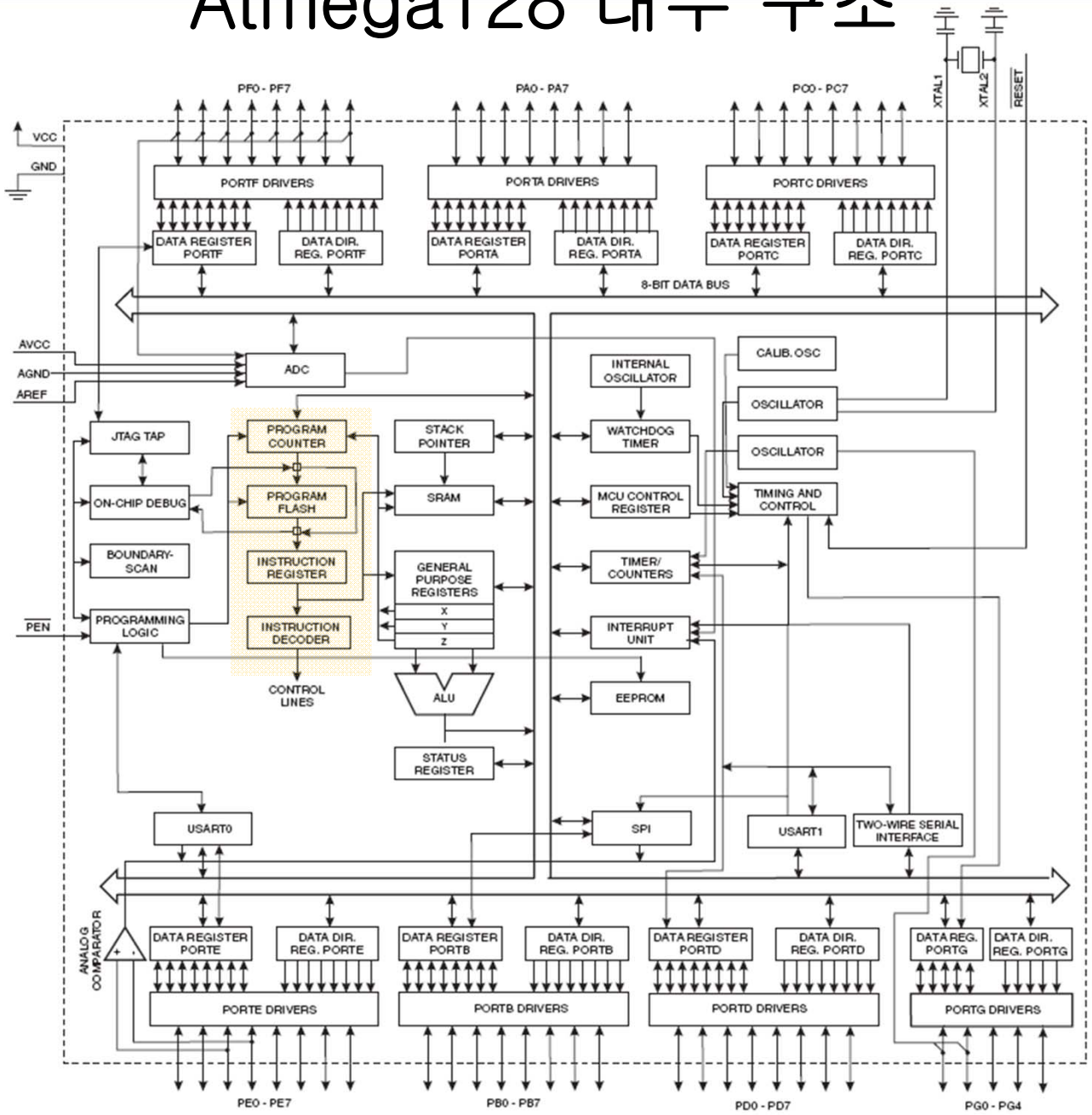
// 타이머/카운터0 오버플로우 서비스 루틴
// 인터럽트 발생 주기 1/16us * 1024분주 * 256 = 16,384us
interrupt [TIM0_OVF] void timer_int0(void)
{
        // LED 순차 점멸
        led <<= 1;
        led |= 0x01;
        if(led == 0xFF) led = 0xFE;
        PORTC = led;
}
```

ATmega128 외부 구조(Cont)

- 타이머/카운터를 이용한 예제

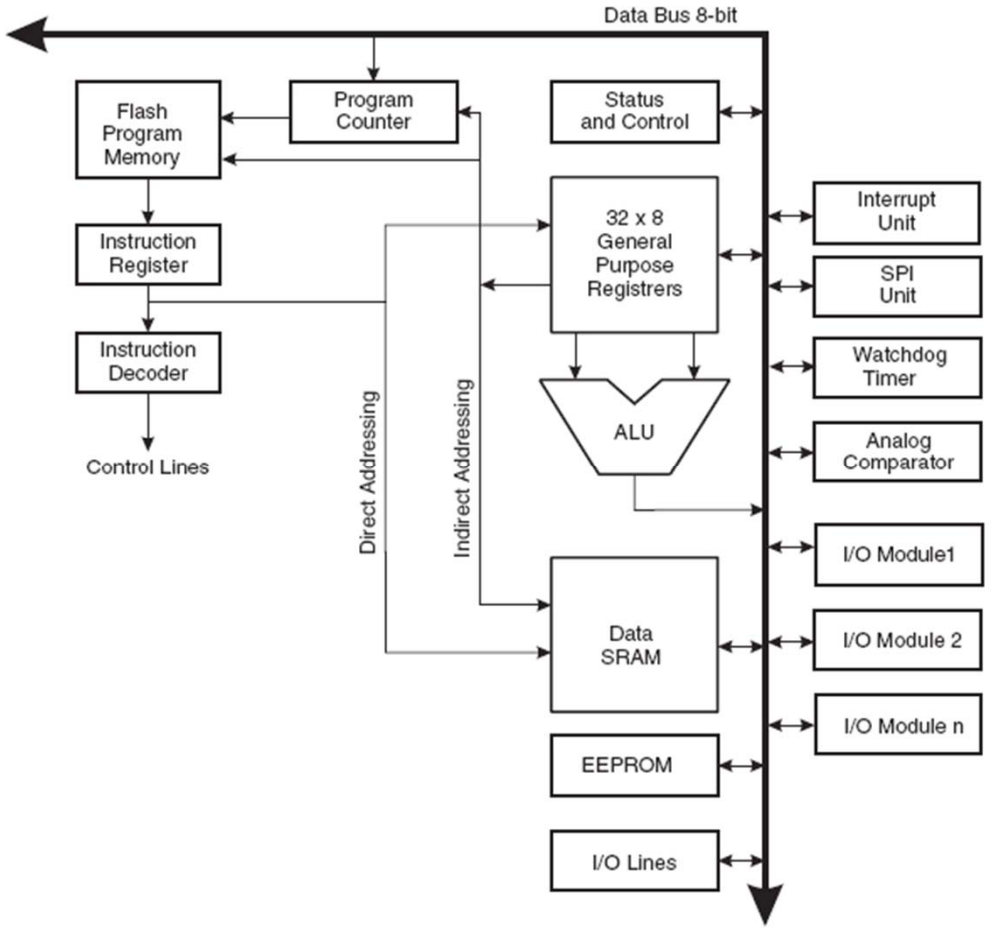


Atmega128 내부 구조



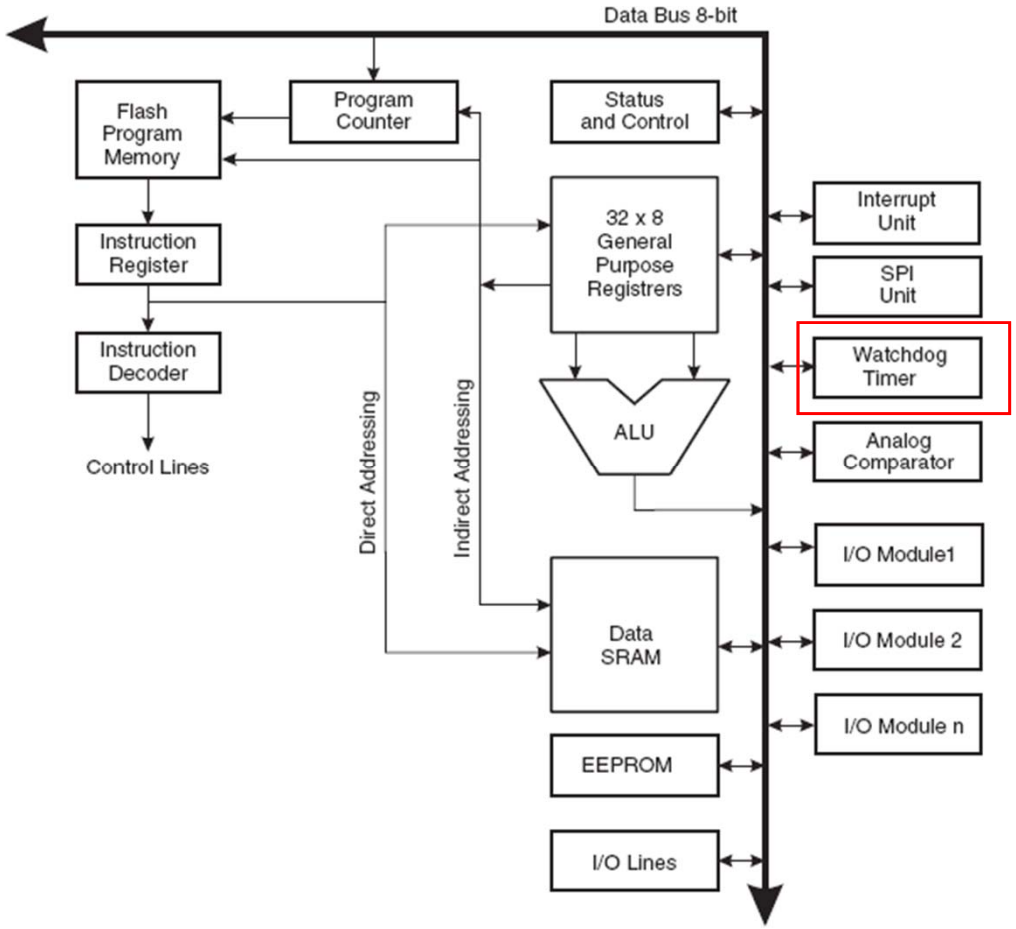
ATmega128 내부구조 – CPU 코어부

- 명령처리 관련 부분
  - 프로그램 카운터가 지시하는 플래시 메모리 번지에서 명령을 인출하고 해독하는 부분
  - ALU
  - 32개 범용 레지스터
  - 상태 레지스터
  - 스택 포인터
- 데이터 메모리 부분
- 인터럽트 처리기 부분



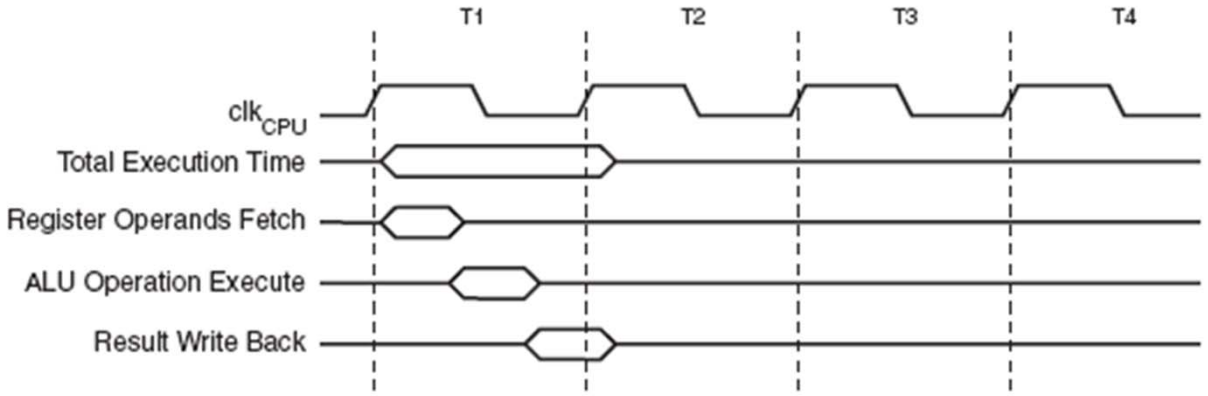
ATmega128 내부구조 – CPU 코어부

- 용어설명(Watchdog Timer)
  - 프로그래머가 실수 혹은 노이즈 등에 의해서 인터럽트 루틴에서 벗어나지 못한 경우 동일된 동작만 반복하는 등 문제가 발생하면 시스템은 다시 시작되고 정상적인 동작을 하게함

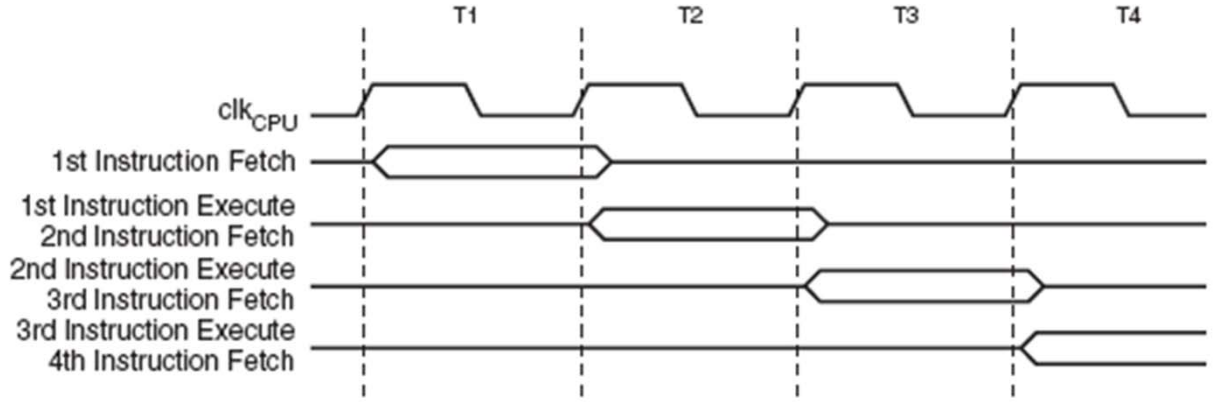


ATmega128 내부구조 – 2단계 파이프라이닝

- 클럭 주파수 1MHz당 1 MIPS 처리속도(ALU동작)



- 명령인출 & 실행 동시 처리(파이프 라인 처리방식)



ATmega128 내부구조 – 상태 레지스터(SREG)

- 가장 최근에 실행된 산술연산 명령의 결과에 따른 상태 저장
- 인터럽트 처리시 자동저장/복구 안됨 (소프트웨어적 처리필요)
  - Bit 7 – I : Global Interrupt Enable
  - Bit 6 – T : Bit Copy Storage
  - Bit 5 – H : Half Carry Flag
  - Bit 4 – S : Sign Bit
  - Bit 3 – V : Two's Complement Overflow Flag
  - Bit 2 – N : Negative Flag
  - Bit 1 – Z : Zero Flag
  - Bit 0 – C : Carry Flag

Bit	7	6	5	4	3	2	1	0	
	I	T	H	S	V	N	Z	C	SREG
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ATmega128 내부구조 – 스택 포인터(SP)

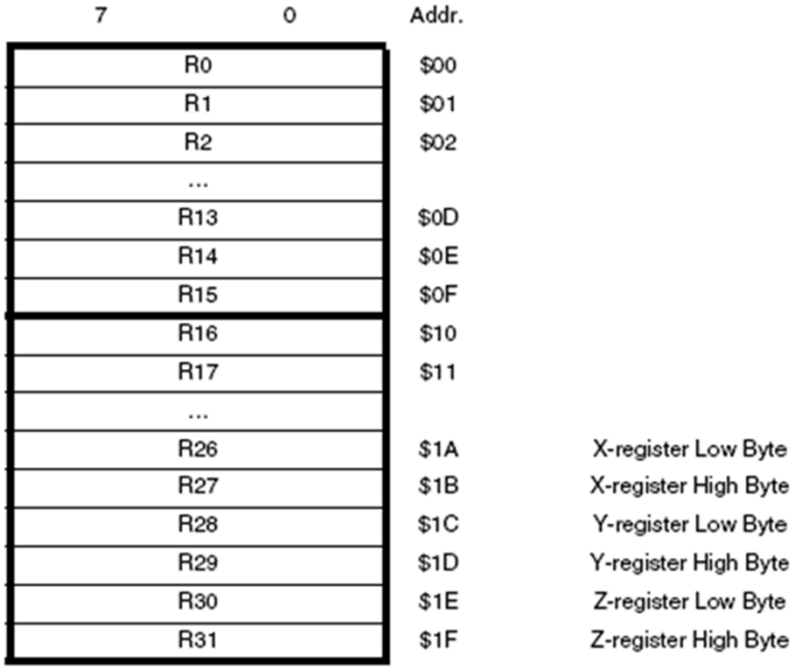
- 서브루틴 호출/인터럽트 발생시에 복귀주소(16bit) 저장
  주소저장/2감소
- RET, RETI명령 : 주소추출/2증가
- PUSH명령 : 데이터저장/1감소, POP명령 : 데이터추출/1증가
- C program/지역변수, 어셈블리 Program/임시데이터 저장 용도
- 초기값은 최소 0x0100번지 이상의 값으로 설정해야 함
  (대부분의 경우 SRAM의 끝번지(0x10FF)로 지정)

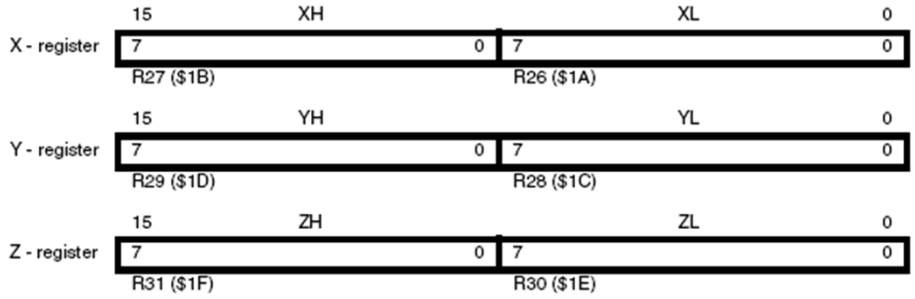
Bit	15	14	13	12	11	10	9	8	
	SP15	SP14	SP13	SP12	SP11	SP10	SP9	SP8	SPH
	SP7	SP6	SP5	SP4	SP3	SP2	SP1	SP0	SPL
	7	6	5	4	3	2	1	0	
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	0	

ATmega128 내부구조 – 범용 레지스터(R31~R0)

- 레지스터 중심의 진보된 RISC 마이크로프로세서 : 별도의 누산기 없음
- 32개 레지스터 이용 4칙 연산 수행
- 즉치 데이터를 사용하는 일부 연산 명령은 R16~R31에서만 수행

- 마지막 6개 R26~R31은 각각 2개씩 합해져서 3개의 16bit 레지스터인 X,Y,Z 레지스터로 이용 가능
  - 데이터 메모리 16bit 간접 지정 어드레스 포인터
  - 프로그램 메모리 영역 상수 액세스





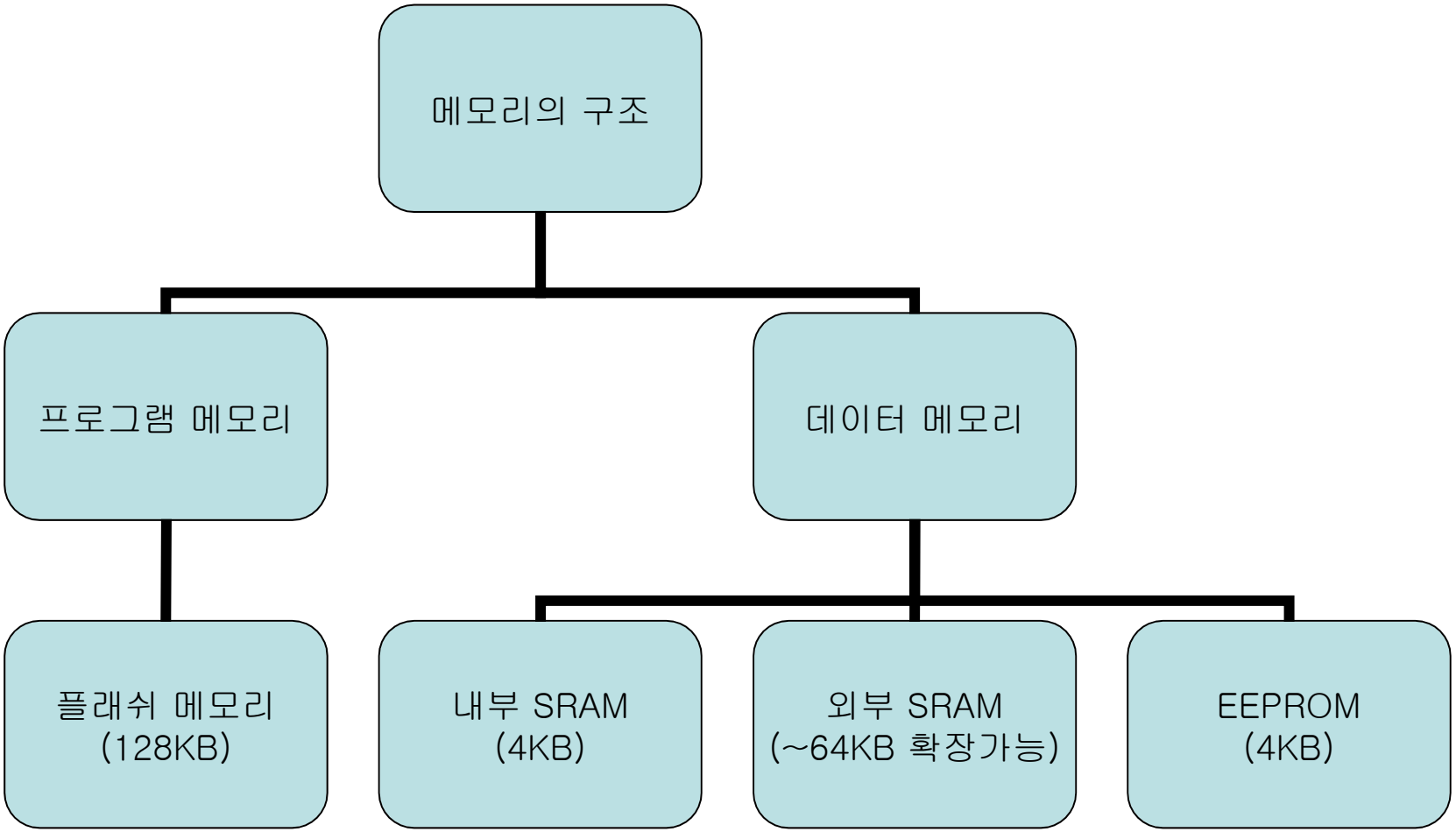
ATmega128 내부구조 – RAM 페이지 선택 레지스터 (RAMPZ)

- 내부 플래시 프로그램 메모리는 64KB 단위 2개 페이지로 구분되는 2개의 페이지 중에서 사용할 페이지를 지정하는데 사용

- 프로그램 메모리 번지의 상수를 access시, ELPM 및 SPM 명령을 통해 Z register(16bit)가 이용 → 64KB (페이지 선택필요)

Bit	7	6	5	4	3	2	1	0	
	–	––	–	–	–	–	–	RAMPZ0	RAMPZ
Read/Write	R	R	R	R	R	R	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ATmega128 메모리 구조



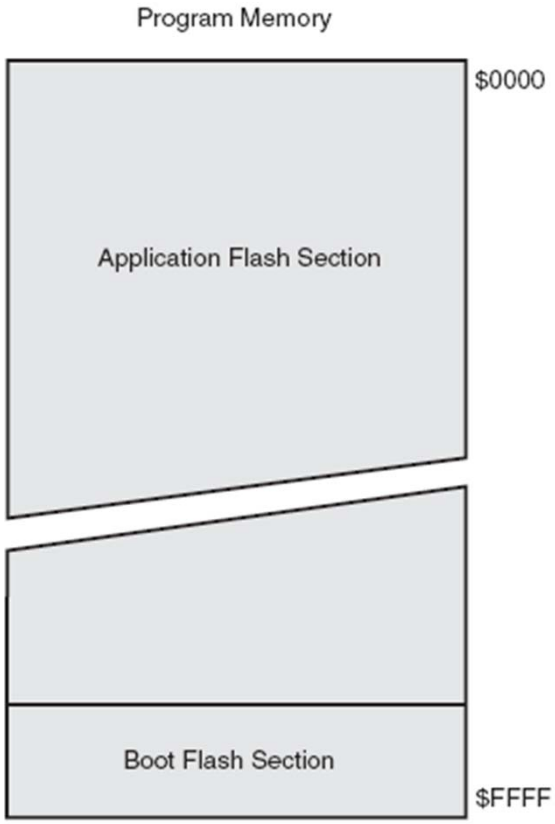
ATmega128 메모리 구조

- 프로그램 메모리 : 128KB Flash Memory

- 데이터 메모리 : 최대크기 = 64KB (0x0000 ~ 0xFFFFF)
  - 내부 메모리 : SRAM(4KB), EEPROM(4KB)
    - ✓ SRAM 일반모드 = 0x0100 ~ 0x10FF
    - ✓ SRAM ATmega103 호환모드 = 0x0060 ~ 0x0FFF
  - 외부 메모리 : 외부 확장 데이터 메모리(약 60KB)
    - ✓ 일반모드 = 0x1100 ~ 0xFFFF
    - ✓ ATmega103 호환모드 = 0x1000 ~ 0xFFFF
  - EEPROM은 특별한 방식으로 access
  - 나머지는 64KB(16bit) 데이터 메모리 영역이 맵핑

ATmega128 메모리 구조 – 프로그램 메모리

- ATmega128=8비트 마이크로컨트롤러
- 모든 명령어는 16bit or 32bit 구조
  - 메모리 1~2개의 번지 차지
- 프로그램 메모리 각 번지 16bit 구조 (2^{16} = 64K)
  - 64K X 16bit = 128Kbyte
- 부트 프로그램 섹션과 응용프로그램 섹션으로 나누어 사용 가능
- SPI 직렬통신을 이용한 ISP기능으로 프로그래밍 가능
- 약 10,000번까지 재프로그램 가능
- JTAG 에뮬레이터 및 병렬 프로그래밍 이용한 프로그램 기입
- 프로그램 메모리 영역에 정의된 상수 액세스 시 LPM/ELPM/SPM 명령 사용



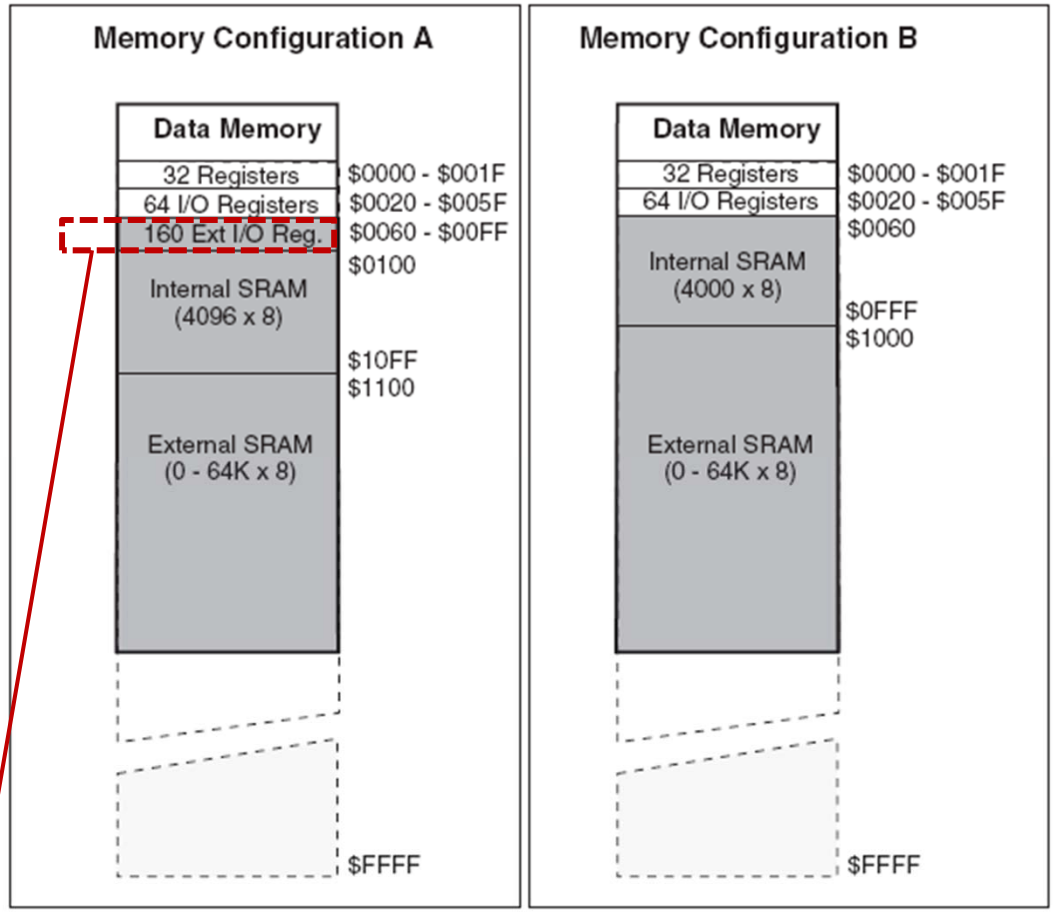
Program Memory Map

ATmega128 메모리 구조 – 데이터 메모리

- 내부 : SRAM, EEPROM
- 외부 : 확장 데이터 메모리
- EEPROM : 별도 액세스

- SRAM : 64KB 데이터 메모리 영역에 맵핑

ATmega103 호환모드에서는 **ATmega103** 에서 지원 안하는 **160**개의 확장 **I/O** 레지스터 사용 안한다.



일반모드

ATmega103 호환모드

ATmega128 메모리 구조 – I/O Registers

- I/O 레지스터
  - 내장된 I/O 디바이스 각종 제어에 사용
  - IN/OUT 명령을 이용하여 액세스
  - 0x0020~0x005F 번지 사용
  - But, IN/OUT 명령에서는 0x00~0x3F 번호로 지정

    Ex) 0x0020 번지에 있는 PINF 레지스터를 R0 레지스터로 읽는 경우

    IN R0,0x20 (X), IN R0,0x00 (O)
  - 0x00~0x1F에 위치하는 32개 레지스터는 비트단위의 접근(CBI/SBI 명령 및 SBIC/SBIS 명령을 사용)을 할 수 있으나, 나머지 번지의 레지스터는 비트 단위의 접근 불가.

- 확장 I/O 레지스터
  - ATmega128에 새로 추가된 각종 I/O 디바이스 제어
  - 0x0060~0x00FF의 160개 번지에 존재
  - LD/LDS/LDD, ST/STS/STD 명령만 이용
  - 반드시 16비트로 지정

ATmega128 메모리 구조 – 내부 SRAM

- 내부 SRAM
  - 내부 데이터 메모리로 0x0100~0x10FF 번지의 4KB를 가짐
  - 내부 데이터 메모리의 액세스에는 2 클럭 사이클이 소요
    * 외부 데이터 메모리 엑세스는 3 클럭 사이클 소요
  - 각종 사용자 변수나 스택 영역으로 사용 되기도 함

- 외부 SRAM
  - 0x1100~0xFFFF 번지의 약 60KB를 가짐
  - 각종 사용자 변수나 스택 영역으로 사용 될 수 있음

Assembly Code Example[1]

```
; OFFSET is defined to 0x2000 to ensure
; external memory access
; Configure Port C (address high byte) to
; output 0x00 when the pins are released
; for normal Port Pin operation
ldi  r16, 0xFF
out  DDRC, r16
ldi  r16, 0x00
out  PORTC, r16
; release PC7:5
ldi  r16, (1<<XMM1)|(1<<XMM0)
sts  XMCRB, r16
; write 0xAA to address 0x0001 of external
; memory
ldi  r16, 0xaa
sts  0x0001+OFFSET, r16
; re-enable PC7:5 for external memory
ldi  r16, (0<<XMM1)|(0<<XMM0)
sts  XMCRB, r16
; store 0x55 to address (OFFSET + 1) of
; external memory
ldi  r16, 0x55
sts  0x0001+OFFSET, r16
```

C Code Example[1]

```
#define OFFSET 0x2000

void XRAM_example(void)
{
unsigned char *p = (unsigned char *) (OFFSET + 1);

DDRC = 0xFF;
PORTC = 0x00;

XMCRB = (1<<XMM1) | (1<<XMM0);

*p = 0xaa;

XMCRB = 0x00;

*p = 0x55;
}
```

C:\cvavreval\inc\mega128.h

```
1  // CodeVisionAVR V2.0 C Compiler
2  // (C) 1998-2009 Pavel Haiduc, HP InfoTech S.R.L.
3
4  // I/O registers definitions for the ATmega128
5
6  #ifndef _MEGA128_INCLUDED_
7  #define _MEGA128_INCLUDED_
8
9  #pragma used+
10 sfrb PINF=0;
11 sfrb PINE=1;
12 sfrb DDRE=2;
13 sfrb PORTE=3;
14 sfrb ADCL=4;
15 sfrb ADCH=5;
16 sfrw ADCW=4;      // 16 bit access
17 sfrb ADCSRA=6;
18 sfrb ADMUX=7;
19 sfrb ACSR=8;
20 sfrb UBRR0L=9;
21 sfrb UCSR0B=0xa;
22 sfrb UCSR0A=0xb;
23 sfrb UDR0=0xc;
24 sfrb SPCR=0xd;
25 sfrb SPSR=0xe;
26 sfrb SPDR=0xf;
27 sfrb PIND=0x10;
28 sfrb DDRD=0x11;
29 sfrb PORTD=0x12;
30 sfrb PINC=0x13;
31 sfrb DDRC=0x14;
32 sfrb PORTC=0x15;
33 sfrb PINB=0x16;
34 sfrb DDRB=0x17;
```

ATmega128 메모리 구조 – EEPROM

- 4KB, 100,000번 재프로그램, 다른 메모리 어드레스와 별개로 0x0000~0x0FFF 번지에 할당, 1바이트씩 access
- EEPROM 어드레스 레지스터(EEAR), EEPROM 데이터 레지스터(EEDR), EEPROM 제어 레지스터(EECR)

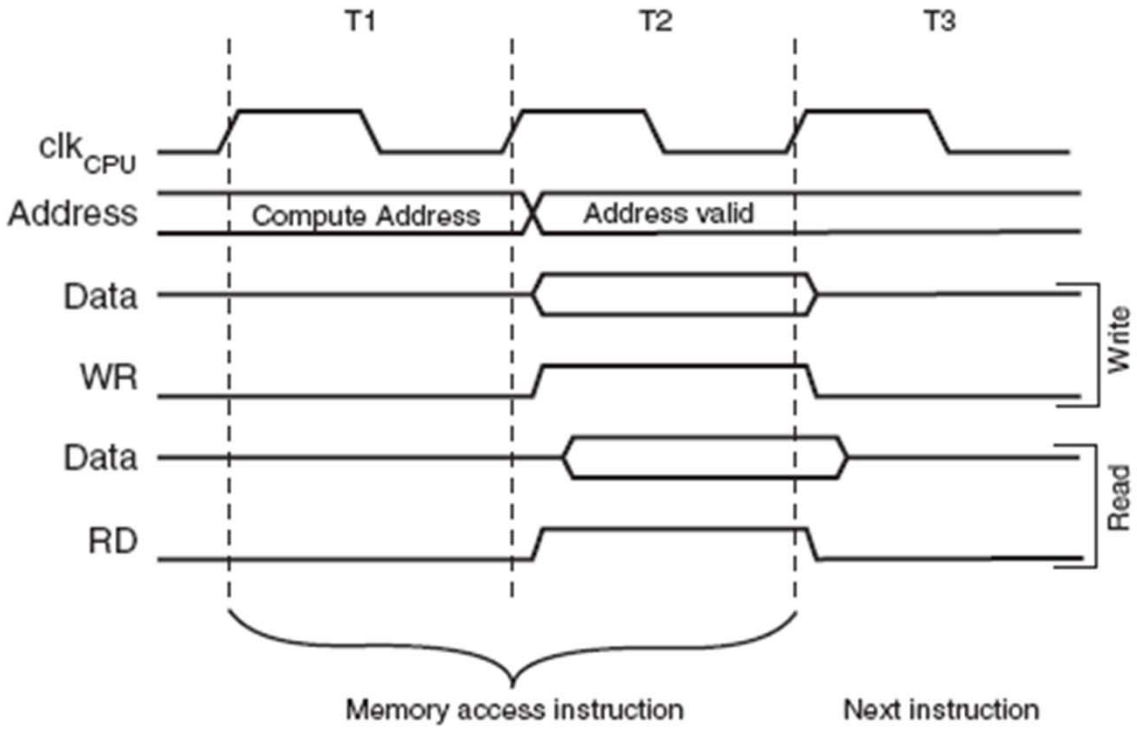
Bit	15	14	13	12	11	10	9	8	
	–	–	–	–	EEAR11	EEAR10	EEAR9	EEAR8	EEARH
	EEAR7	EEAR6	EEAR5	EEAR4	EEAR3	EEAR2	EEAR1	EEAR0	EEARL
	7	6	5	4	3	2	1	0	
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	X	X	X	X	
	X	X	X	X	X	X	X	X	

Bit	7	6	5	4	3	2	1	0	
	MSB							LSB	EEDR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	–	–	–	–	EERIE	EEMWE	EEWE	EERE	EECR
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	X	0	

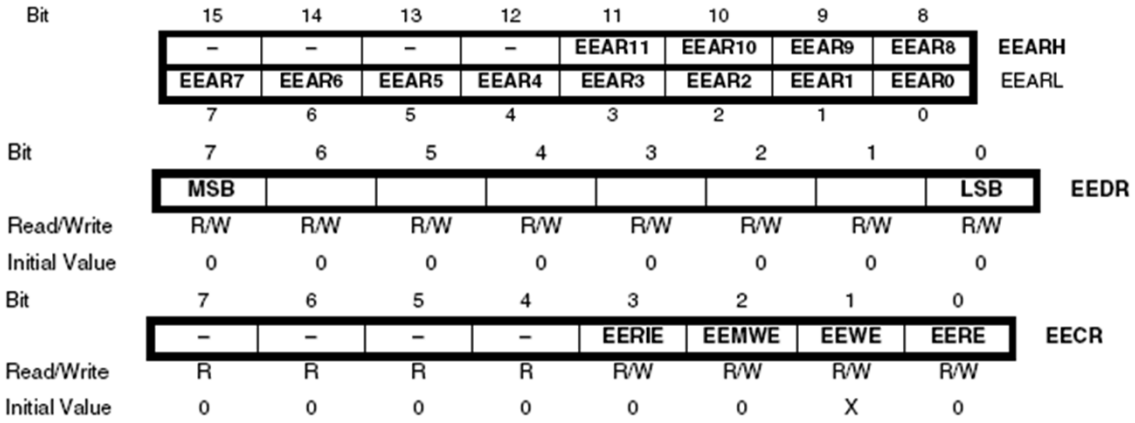
ATmega128 메모리 구조 – EEPROM

- Internal Data SRAM Access in two CLK cycles



ATmega128 메모리 구조 – EEPROM

- Write 동작
  1. 이전 write 동작이 완료되어 EEWE가 Clear 될 때까지 대기
  2. SPMCSR 레지스터에서 SPMEN이 Clear 될 때까지 대기
  3. 새로 write할 EEPROM 어드레스를 EEAR 레지스터에 저장
  4. 새로 write할 EEPROM 데이터를 EEDR 레지스터에 저장
  5. EECR 레지스터에 EEMWE를 1로, EEWE를 0으로 설정
  6. 4사이클 이내에 EEWE를 1로 설정



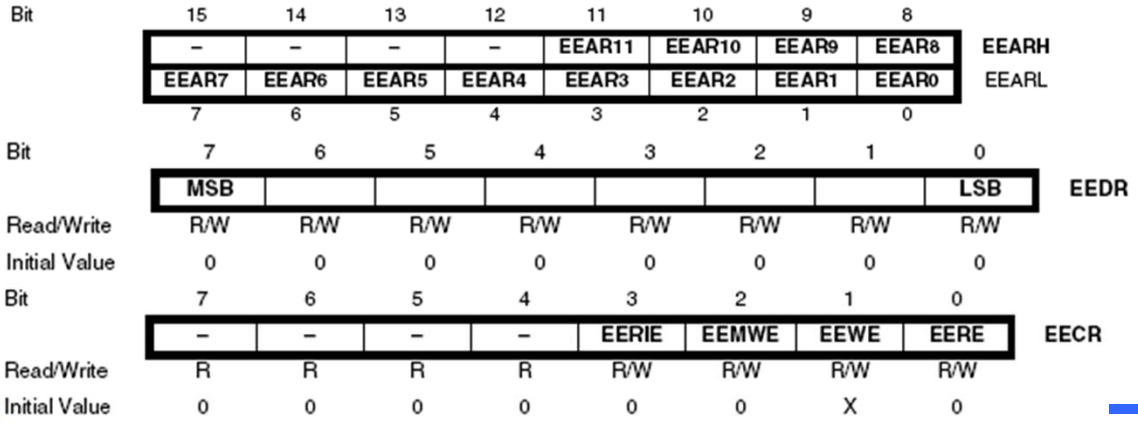
Bit	15	14	13	12	11	10	9	8	
	–	–	–	–	EEAR11	EEAR10	EEAR9	EEAR8	EEARH
	EEAR7	EEAR6	EEAR5	EEAR4	EEAR3	EEAR2	EEAR1	EEAR0	EEARL
	7	6	5	4	3	2	1	0	

Bit	7	6	5	4	3	2	1	0	
	MSB							LSB	EEDR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	–	–	–	–	EERIE	EEMWE	EEWE	EERE	EECR
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	X	0	

ATmega128 메모리 구조 – EEPROM

```
void EEPROM_write(unsigned int uiAddress, unsigned char ucData)
{
/* Wait for completion of previous write */
while(EECR & (1<<EEWE)) ;
/* Set up address and data registers */
EEAR = uiAddress;
EEDR = ucData;
/* Write logical one to EEMWE */
EECR |= (1<<EEMWE);
/* Start eeprom write by setting EEWE */
EECR |= (1<<EEWE);
}
```



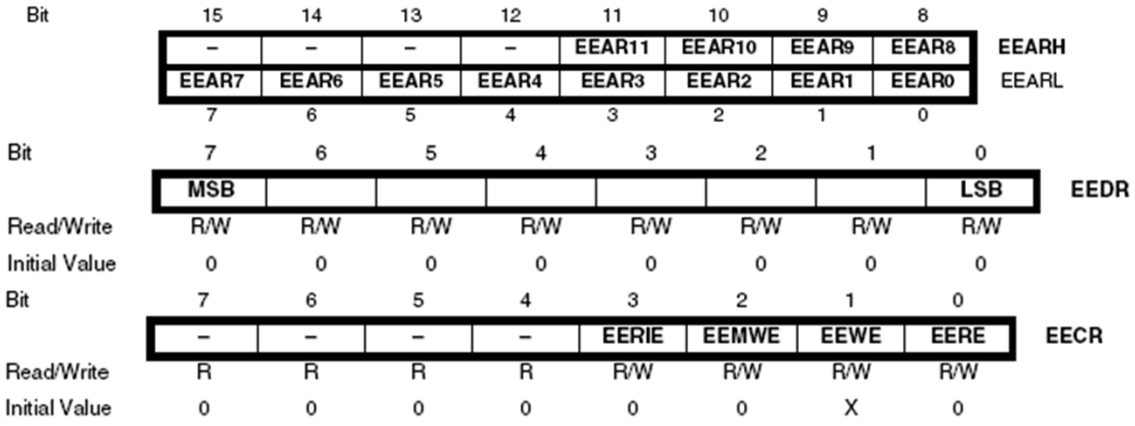
Bit	15	14	13	12	11	10	9	8	
	–	–	–	–	EEAR11	EEAR10	EEAR9	EEAR8	EEARH
	EEAR7	EEAR6	EEAR5	EEAR4	EEAR3	EEAR2	EEAR1	EEAR0	EEARL
	7	6	5	4	3	2	1	0	

Bit	7	6	5	4	3	2	1	0	
	MSB							LSB	EEDR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	–	–	–	–	EERIE	EEMWE	EEWE	EERE	EECR
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	X	0	

ATmega128 메모리 구조 – EEPROM

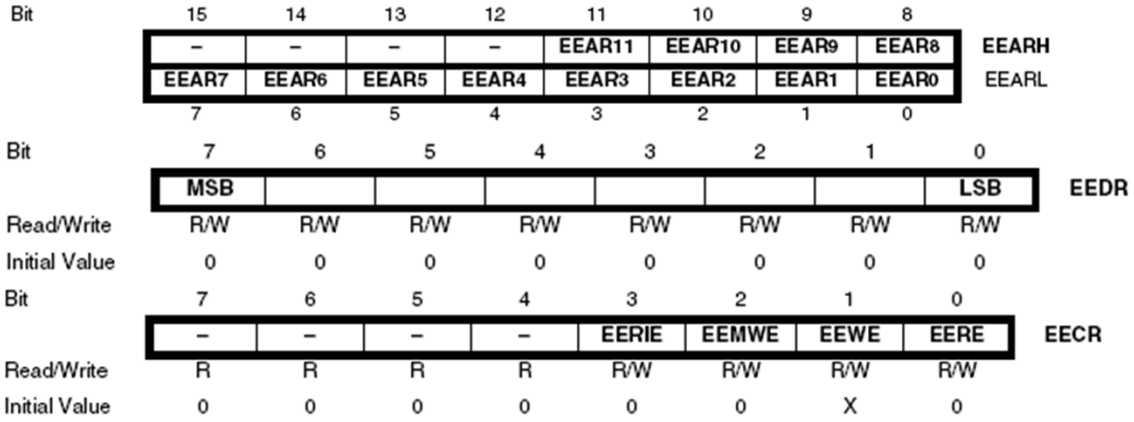
- Read 동작
  - 앞의 write 동작이 완료되어 EEWE가 Clear 될 때까지 대기
  - Read 할 EEPROM 어드레스를 EEAR 레지스터에 저장
  - EECR 레지스터에 EERE를 1로 설정
  - EEPROM 데이터를 EEDR 레지스터로부터 read (immediately)



Bit	15	14	13	12	11	10	9	8	
	–	–	–	–	EEAR11	EEAR10	EEAR9	EEAR8	EEARH
	EEAR7	EEAR6	EEAR5	EEAR4	EEAR3	EEAR2	EEAR1	EEAR0	EEARL
	7	6	5	4	3	2	1	0	
Bit	7	6	5	4	3	2	1	0	
	MSB							LSB	EEDR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
Bit	7	6	5	4	3	2	1	0	
	–	–	–	–	EERIE	EEMWE	EEWE	EERE	EECR
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	X	0	

ATmega128 메모리 구조 – EEPROM

```
unsigned char EEPROM_read(unsigned int uiAddress)
{
/* Wait for completion of previous write */
while(EECR & (1<<EEWE)) ;
/* Set up address register */
EEAR = uiAddress;
/* Start eeprom read by writing EERE */
EECR |= (1<<EERE);
/* Return data from data register */
return EEDR;
}
```



ATmega128의 외부 인터페이스

- 우수한 기능
  - 소프트웨어적으로 0~3개의 웨이트 사이클을 설정 할 수 있음 (느린 접속시간을 갖는 주변 소자와의 연결 설정가능)
  - 외부 데이터 메모리 영역을 2개의 섹터로 분할 할 수 있고 독립적으로 웨이트 사이클을 설정 할 수 있음
  - 16bit 어드레스의 상위 바이트에 있는 8bit 중에서 필요한 개수만을 어드레스 버스로 동작하게 할 수 있음
  - 데이터 버스의 신호들이 동작할 때 전류 소비량이 감소되도록 버스 키퍼 기능을 설정하여 사용할 수 있음

MCUCR레지스터

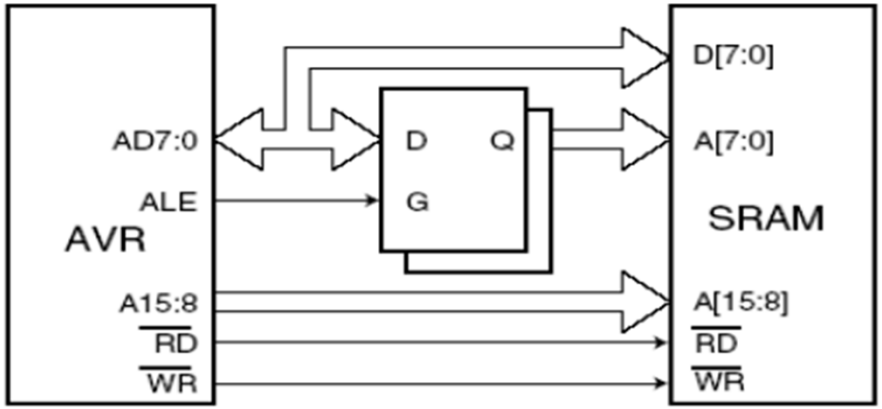
Table 4. Wait States(1)

SRWn1	SRWn0	Wait States
0	0	No wait-states
0	1	Wait one cycle during read/write strobe
1	0	Wait two cycles during read/write strobe
1	1	Wait two cycles during read/write and wait one cycle before driving out new address

Note: 1. n = 0 or 1 (lower/upper sector).
For further details of the timing and wait-states of the External Memory Interface, see Figures 13 through Figures 16 for how the setting of the SRW bits affects the timing.

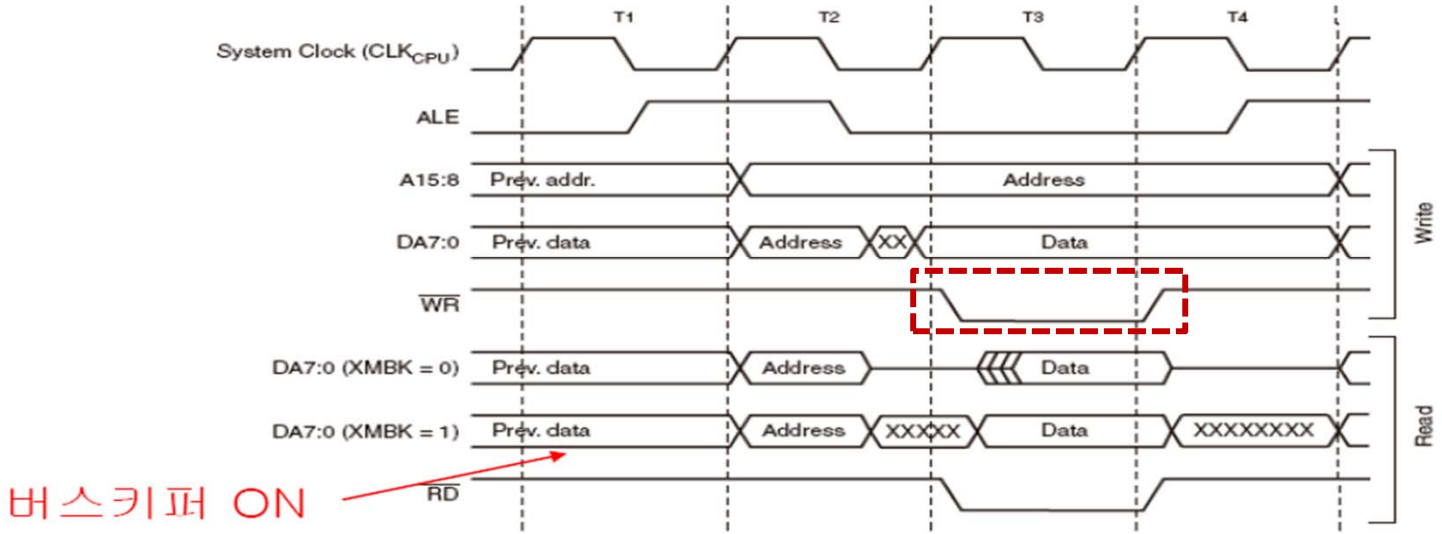
ATmega128의 외부 인터페이스 – 기본 동작

- 외부 인터페이스에 사용되는 신호
  - A15~A8, AD7~AD0, ALE, RD', WR'
  - MCUCR 레지스터에 의하여 유효하도록 설정
- AD7~AD0
  - 핀 수를 절약하기 위하여 어드레스 버스의 하위 8bit와 데이터 버스를 시분할 다중화 시킨 것
  - 74573(8-bit 3-state transparent latch) 소자에 ALE 신호를 사용하여 분리 시킴



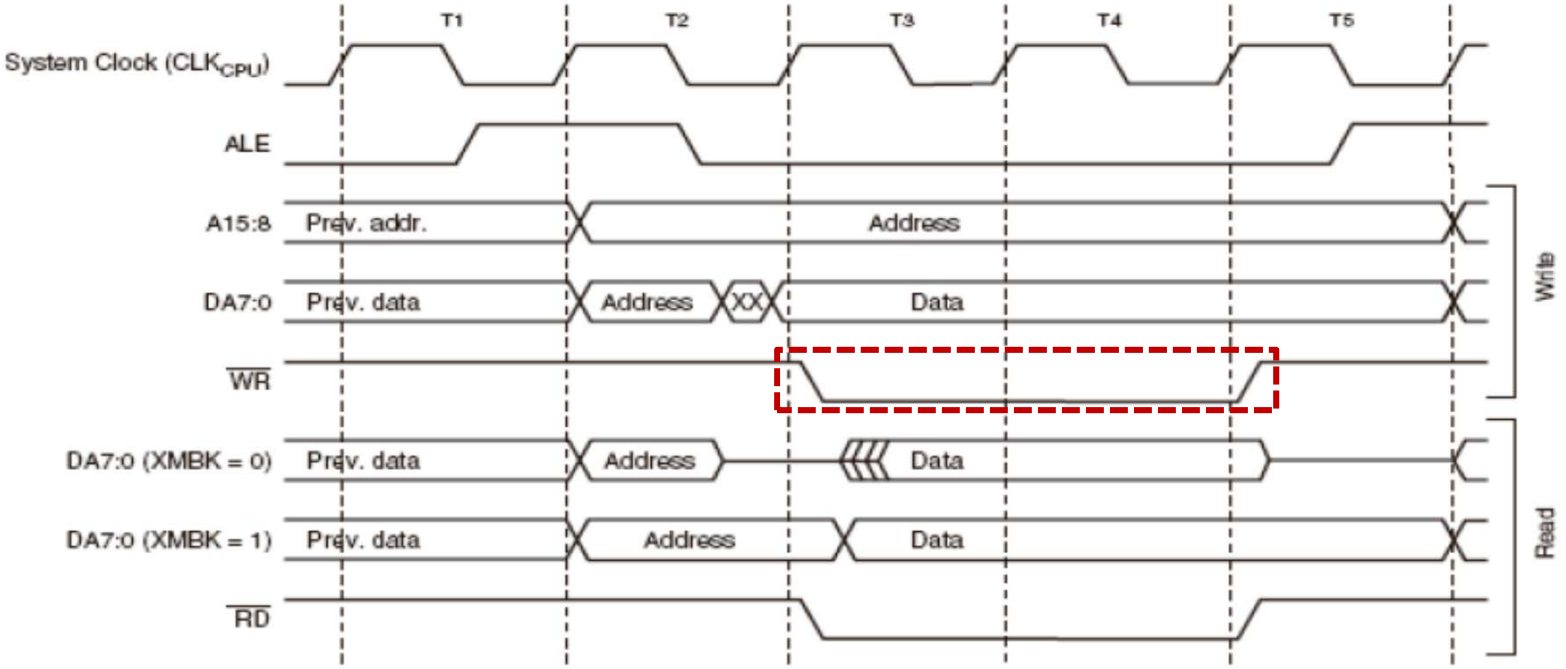
기본 외부 액세스 동작 사이클 (No wait)

- 외부 데이터 메모리 액세스 동작은 기본적으로 3클럭 사이클에 수행
- 버스 키퍼 기능: 전력소비 줄이기 위하여 리드 동작시에 이 버스가 3-스테이트 상태로 들어가야하는 순간에 이전의 상태 값을 그대로 출력하게 함

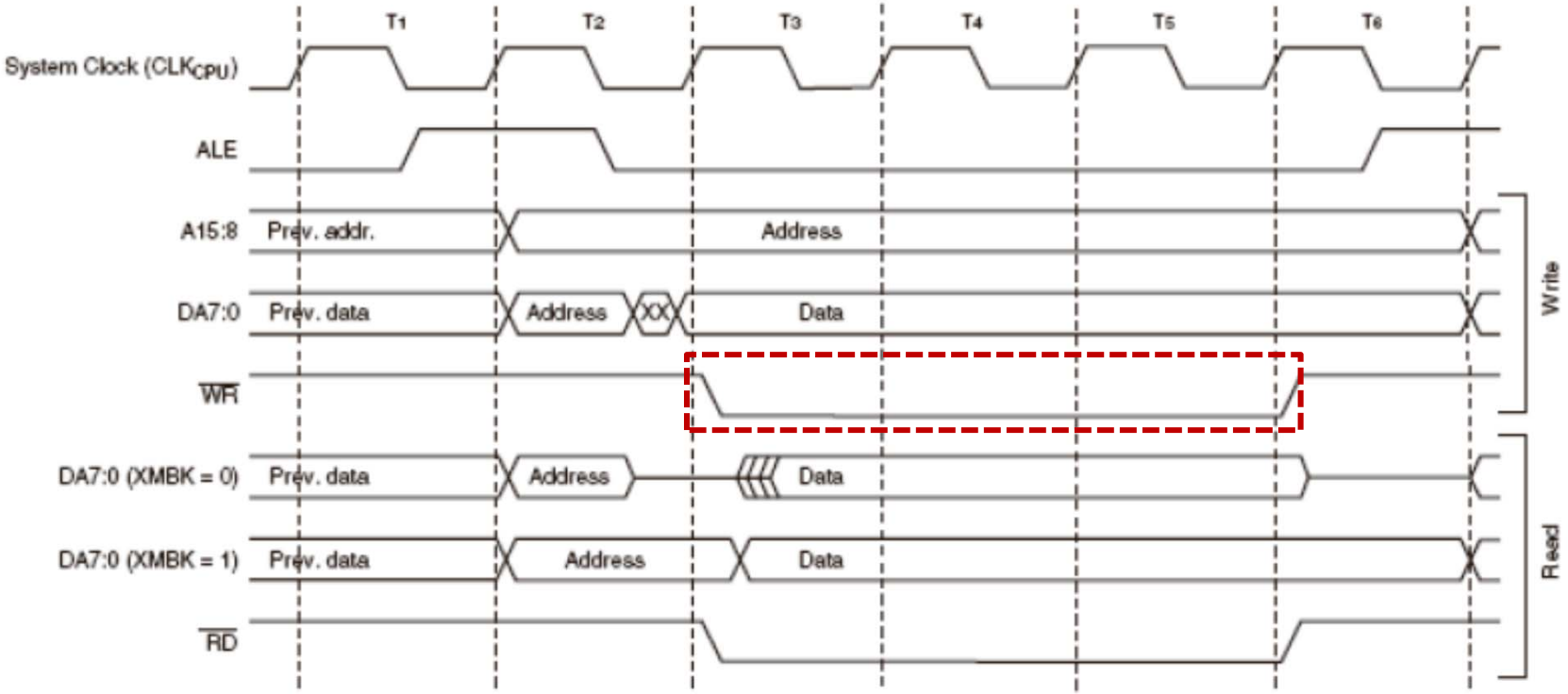


외부 액세스 동작 사이클 (1 wait-state)

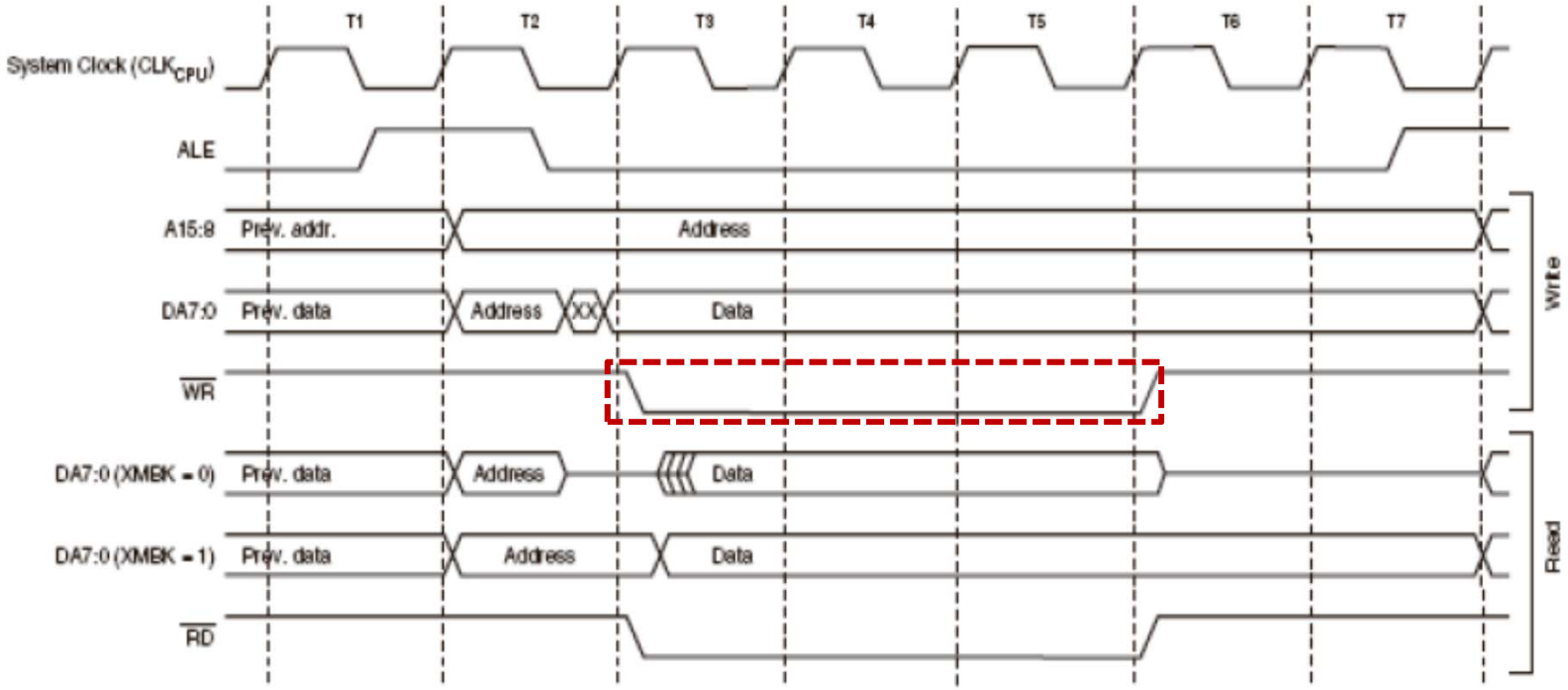
- MCUCR 및 XMCRA 레지스터를 이용해 소프트웨어적으로 1~3 개의 웨이트 사이클을 줄 수 있다.



외부 액세스 동작 사이클 (2 wait-state)



외부 액세스 동작 사이클 (3 wait-state)

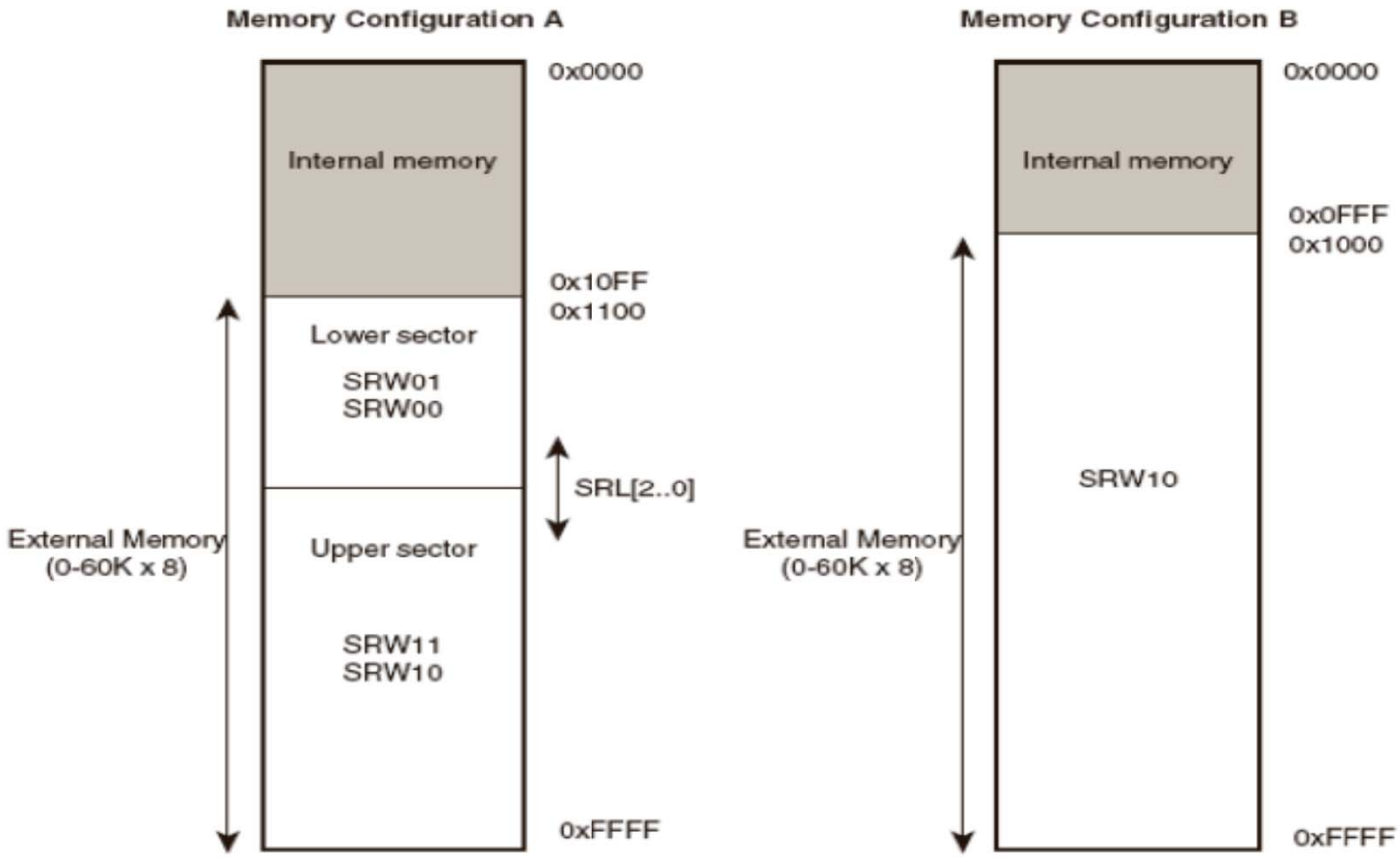


ATmega128의 외부 인터페이스 – 관련된 레지스터 설정

- 외부 데이터 메모리 영역의 섹터 분할
  - ATmega128의 외부 데이터 메모리 영역은 사용자가 임의로 경계선을 지정하여 사이즈를 설정하는 방법으로 2개의 섹터로 나눌 수 있음
  - 나누어진 각 섹터는 독립적으로 웨이트 사이클 설정 가능
  - 액세스 빠른 SRAM은 0 웨이트로 설정
  - 상대적으로 속도가 느린 I/O 디바이스들은 적절한 웨이트 부여

ATmega128의 외부 인터페이스 – 관련된 레지스터 설정

- 섹터 설정이 가능한 외부 메모리



ATmega128의 외부 인터페이스 – 관련된 레지스터 설정

- MCUCR (MCU Control Register)
  - CPU의 전체적인 시스템 기능을 설정
  - b7~b6: 외부 데이터 메모리 영역의 설정과 관련
  - SRE (External SRAM/XREM Enable)
    - A15~A8, AD0~AD7, ALE, RD, WR 등과 같은 외부 데이터 메모리 영역의 액세스 신호들 유효하게 함
    - 설정하지 않으면 기본 기능인 병렬I/O포트로 동작
  - SRW10 (Wait-state Select Bit)
    - XMCRA의 SRW11과 함께 외부 데이터 메모리 영역을 액세스할 때 부여하는 웨이트 사이클의 수를 지정

Bit	7	6	5	4	3	2	1	0	
	SRE	SRW10	SE	SM1	SM0	SM2	IVSEL	IVCE	MCUCR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ATmega128의 외부 인터페이스 – 관련된 레지스터 설정

- XMCRA (External Memory Control Register A)
  - 메모리 영역을 분할 혹은 웨이트 사이클을 설정

Bit	7	6	5	4	3	2	1	0	
	–	SRL2	SRL1	SRL0	SRW01	SRW00	SRW11	–	XMCRA
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R	
Initial Value	0	0	0	0	0	0	0	0	

SRL2	SRL1	SRL0	Sector Limits
0	0	0	Lower sector = N/A Upper sector = 0x1100 - 0xFFFF
0	0	1	Lower sector = 0x1100 - 0x1FFF Upper sector = 0x2000 - 0xFFFF
0	1	0	Lower sector = 0x1100 - 0x3FFF Upper sector = 0x4000 - 0xFFFF
0	1	1	Lower sector = 0x1100 - 0x5FFF Upper sector = 0x6000 - 0xFFFF
1	0	0	Lower sector = 0x1100 - 0x7FFF Upper sector = 0x8000 - 0xFFFF
1	0	1	Lower sector = 0x1100 - 0x9FFF Upper sector = 0xA000 - 0xFFFF
1	1	0	Lower sector = 0x1100 - 0xBFFF Upper sector = 0xC000 - 0xFFFF
1	1	1	Lower sector = 0x1100 - 0xDFFF Upper sector = 0xE000 - 0xFFFF

ATmega128의 외부 인터페이스 – 관련된 레지스터 설정

- XMCRA (External Memory Control Register A)
  - 메모리 영역을 분할 혹은 웨이트 사이클을 설정

SRWn1	SRWn0	Wait States
0	0	No wait-states
0	1	Wait one cycle during read/write strobe
1	0	Wait two cycles during read/write strobe
1	1	Wait two cycles during read/write and wait one cycle before driving out new address

ATmega128의 외부 인터페이스 – 관련된 레지스터 설정

- XMCRB (External Memory Control Register B)
  - XMBK (External Memory Bus-keeper Enable)
    - AD7~AD0 신호가 3-스테이트로 되어야 하는 동안에 이전의 값으로 출력 하도록 설정(버스 키퍼 기능)
  - 외부 데이터 메모리 어드레스의 상위 바이트 중에 어디까지 어드레스 기능으로 사용할지 설정

Bit	7	6	5	4	3	2	1	0	
	XMBK	–	–	–	–	XMM2	XMM1	XMM0	XMCRB
Read/Write	R/W	R	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

XMM2	XMM1	XMM0	# Bits for External Memory Address	Released Port Pins
0	0	0	8 (Full 60 KB space)	None
0	0	1	7	PC7
0	1	0	6	PC7 - PC6
0	1	1	5	PC7 - PC5
1	0	0	4	PC7 - PC4
1	0	1	3	PC7 - PC3
1	1	0	2	PC7 - PC2
1	1	1	No Address high bits	Full Port C

ATmega128의 명령 세트 – 데이터의 표현

- 8bit 마이크로 컨트롤러이므로 기본적으로 데이터를 8bit 단위로 처리
- 일부의 명령에서는 16bit 데이터를 사용
- C언어에서는 32bit나 64bit 데이터도 사용하지만 어셈블러에서는 이런 데이터를 지원하는 명령이 없음
- 비트, 바이트, 워드 단위를 가지며, 이 중 워드는 16bit의 길이를 가짐

ATmega128의 명령 세트 – 어드레싱 모드(1)

- 즉치 데이터 번지 지정 (Immediate Addressing)
  - 오퍼랜드(연산에 필요한 수치나 연산으로 나타나는 수치를 통틀어 이르는 말)를 명령어의 일부로 포함하고 있는 방식
  - 8bit와 6bit가 있음
  - 8bit 즉치 데이터를 사용하는 명령에서는 데스티네이션 오퍼랜드로서 범용 레지스터 R16~R31만을 사용할 수 있음
  - ADIW와 SBIW 명령은 16bit 레지스터와 6bit의 즉치 데이터를 연산하는데 16bit 데스티네이션 오퍼랜드로서 범용 레지스터 R24, R26, R28, R30만을 사용할 수 있음

ATmega128의 명령 세트 – 어드레싱 모드(1)

- 즉치 데이터 번지 지정 (Immediate Addressing)

ex) 8비트 즉치 데이터를 사용하는 경우

```
LDI     Rdx, imm8      ;   Rdx ← imm8
```

8비트 즉치데이터 0~255

레지스터 R16~R31

ex) 6비트 즉치 데이터를 사용하는 경우

```
ADIW   RDI, imm6      ;   RDI+1:RDI ← RDI+1:RDI + IMM6
```

6비트 즉치 데이터 0~65535

레지스터 R24, R26, R28, R30

ATmega128의 명령 세트 – 어드레싱 모드(2)

- 레지스터 직접 번지 지정(Register Direct Addressing)
  - 오퍼랜드로서 32개의 범용 레지스터 R0~R31 중의 하나를 사용하는 방식
  - 8bit 즉치 데이터를 사용하는 명령에서는 범용 레지스터 R16~R31만을 사용할 수 있음
  - 16bit 데이터를 사용하는 명령에서는 범용레지스터 R24~R31만을 사용할 수 있음

ATmega128의 명령 세트 – 어드레싱 모드(2)

- 레지스터 직접 번지 지정(Register Direct Addressing)

ex) 데스티네이션 오퍼랜드에 사용되는 경우

LDI imm8 : Rdx ← imm8

레지스터 R16~R31

ex) 소스 오퍼랜드에 사용되는 경우

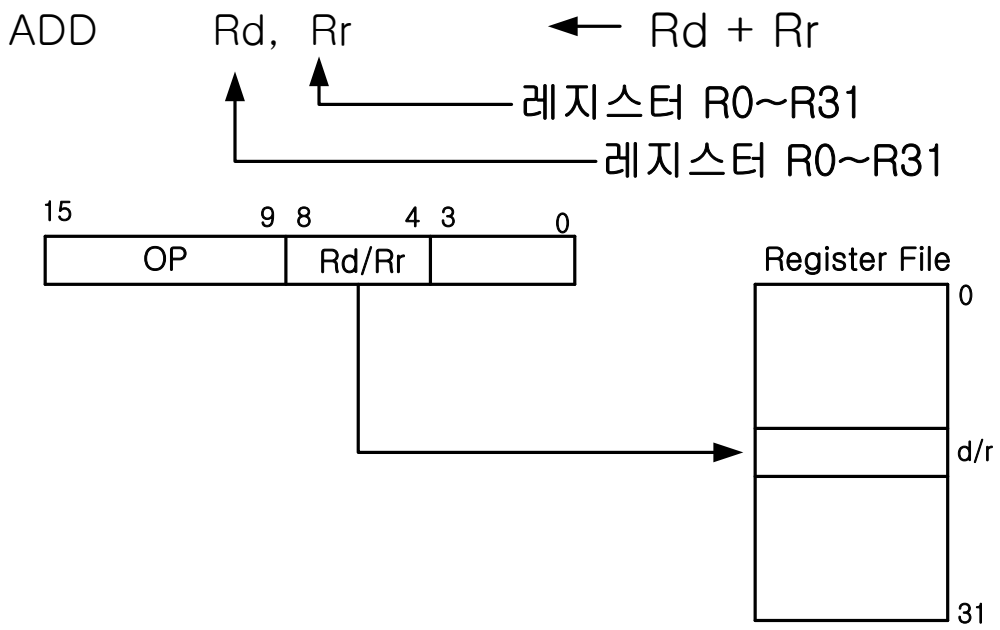
STS addr16, Rr : (addr16) ← Rr

레지스터 R0~R31

ATmega128의 명령 세트 – 어드레싱 모드(2)

- 레지스터 직접 번지 지정(Register Direct Addressing)

  ex) 소스 및 데스티네이션 오퍼랜드에 모두 사용되는 경우



<레지스터 직접 번지 지정>

ATmega128의 명령 세트 – 어드레싱 모드(3)

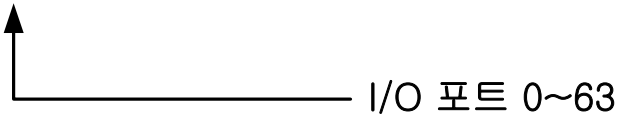
- I/O 직접 번지 지정(I/O Direct Addressing)
  - 오퍼랜드로서 64개의 I/O 레지스터중의 하나를 사용하는 방식
  - 기계어 코드에서 6bit의 어드레스로 지정
  - 실제의 I/O 레지스터를 액세스할 때는 이 6bit 값에 0x20을 더하여 처리
  - IN/OUT 명령에서 사용
  - CBI/SBI/SBIC/SBIS 명령에서는 64개의 I/O 레지스터 중에서 32개만을 사용할 수 있음

ATmega128의 명령 세트 – 어드레싱 모드(3)

- I/O 직접 번지 지정(I/O Direct Addressing)

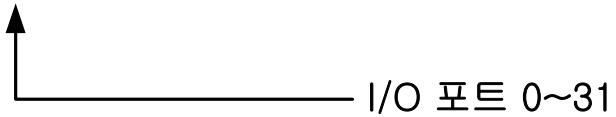
ex) 입출력 명령

```
IN      Rd, port        ;   Rd ← (port)
OUT     port, Rr        ;   (port) ←Rr
```



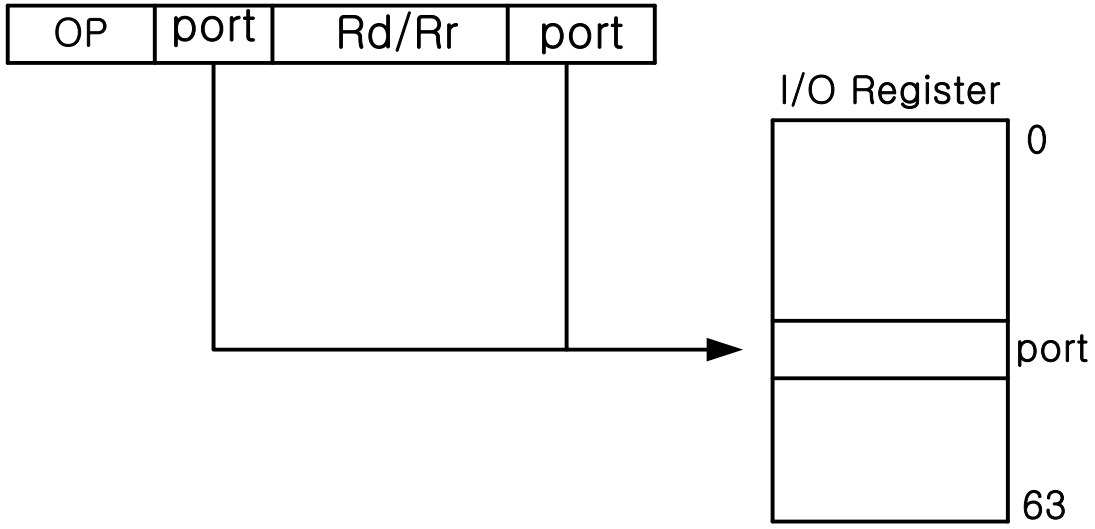
ex) 비트 clear/set 명령

```
CBI     port32, b       ;   (port32)b ← 0
SBI     port32, b       ;   (port32)b ← 1
```



ATmega128의 명령 세트 – 어드레싱 모드(3)

- I/O 직접 번지 지정(I/O Direct Addressing)



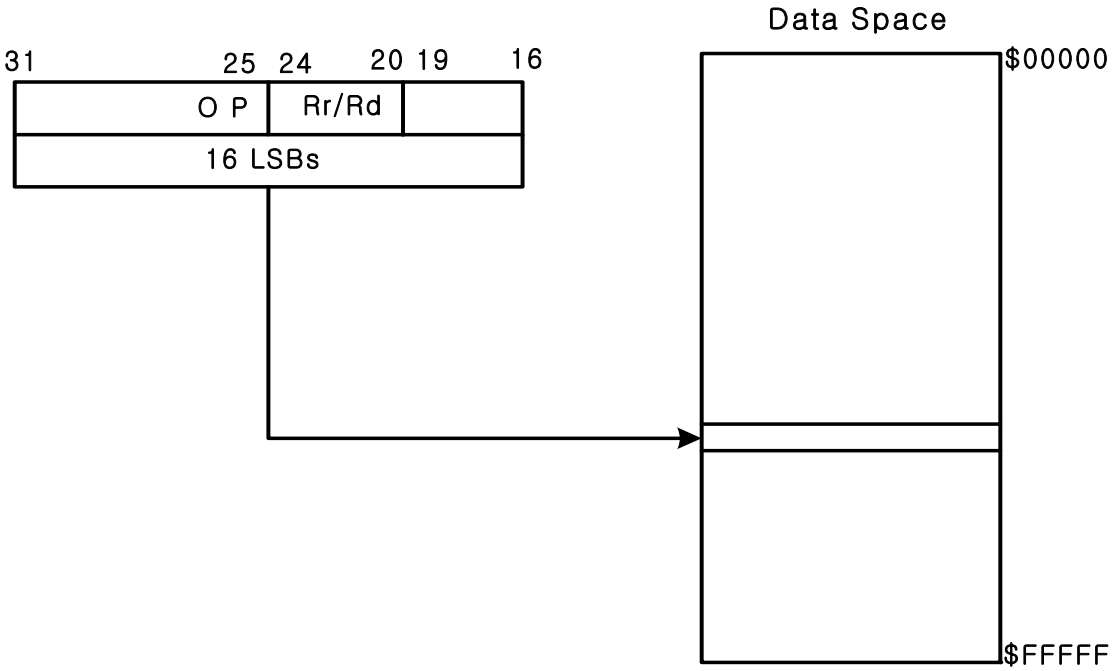
<I/O 직접 번지 지정>

ATmega128의 명령 세트 – 어드레싱 모드(4)

- 데이터 직접 번지 지정(Data Direct Addressing)
  - 오퍼랜드로서 데이터 메모리 영역을 16bit 어드레스 값으로 직접 지정하는 방식
  - 16bit 데이터 메모리 번지는 32개의 범용 레지스터 영역, 64개의 I/O 레지스터 영역, 160개의 확장 I/O 레지스터 영역, 4KB의 내부 SRAM 영역, 외부 데이터 메모리 영역을 모두 포함
  - LDS/STS 명령에서만 사용(16bit 직접 번지는 이밖에 JMP/CALL 명령에서도 사용)

ATmega128의 명령 세트 – 어드레싱 모드(4)

- 데이터 직접 번지 지정(Data Direct Addressing)



Direct Data Addressing

ATmega128의 명령 세트 – 어드레싱 모드(4)

- 데이터 직접 번지 지정(Data Direct Addressing)

ex) 소스 오퍼랜드에 사용되는 경우

```
LDS     Rd, addr16          ;  Rd ←— (addr16)
                  ↑
                  └———— 16비트 어드레스 0~65535
```

ex) 데스티네이션 오퍼랜드에 사용되는 경우

```
STS     addr16, Rr          ;  (addr16) ←— Rr
                  ↑
                  └———— 16비트 어드레스 0~65535
```

ATmega128의 명령 세트 – 어드레싱 모드(5)

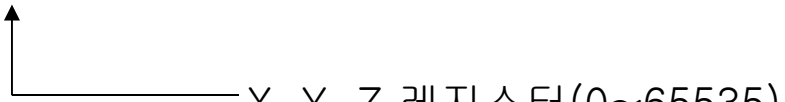
- 데이터 간접 번지 지정(Data Indirect Addressing)
  - 오퍼랜드로서 데이터 메모리 영역을 16bit 어드레스 포인터인 X, Y, Z 레지스터를 사용하여 간접 지정하는 방법
  - 16bit 데이터 메모리 번지는 32개의 범용 레지스터 영역, 64개의 I/O 레지스터 영역, 160개의 확장 I/O 레지스터 영역, 4KB의 내부 SRAM 영역, 외부 데이터 메모리 영역을 모두 포함
  - LD/ST 명령에서만 사용(LPM/ELPM/SPM 명령에서는 Z 레지스터를 사용하여 프로그램 메모리를 이와 유사하게 액세스)

ATmega128의 명령 세트 – 어드레싱 모드(5)

- 데이터 간접 번지 지정(Data Indirect Addressing)

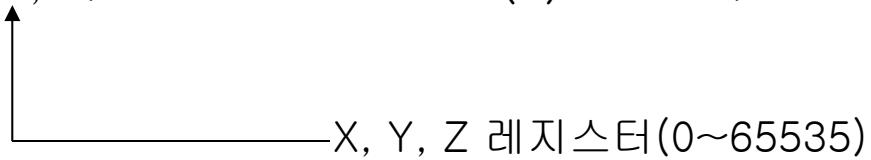
ex) 소스 오퍼랜드에 사용되는 경우

```
LD      Rd, X                   ;   Rd ←—— (X)
```



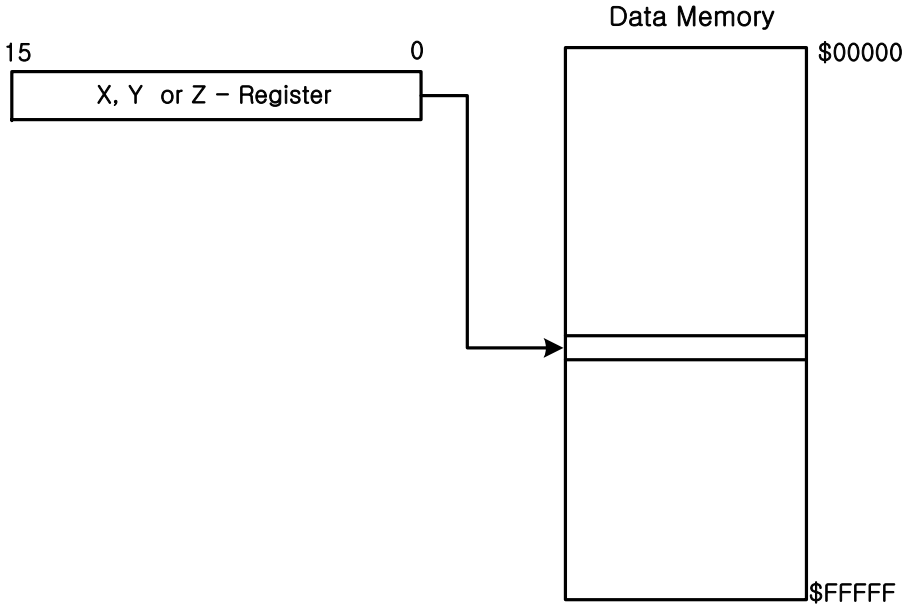
ex) 데스티네이션 오퍼랜드에 사용되는 경우

```
ST      X, Rr                   ;   (X) ←—— Rr
```



ATmega128의 명령 세트 – 어드레싱 모드(5)

- 데이터 간접 번지 지정(Data Indirect Addressing)



Data Indirect Addressing

ATmega128의 명령 세트 – 어드레싱 모드(6)

- 동작 후에 자동증가 기능이 있는 데이터 간접 번지 지정 (Data Indirect Addressing with Post-increment)
  - 데이터 간접 번지 지정과 동일하지만 동작을 수행한 후에 이 명령에서 지정된 X, Y, Z 레지스터를 1만큼 증가시켜 두는 기능을 추가적으로 가지는 방법
  - 번지를 증가시키면서 데이터 메모리 영역을 연속적으로 읽거나 쓰는 동작에 유용
  - LD/ST 명령에서만 사용(LPM/ELPM 명령에서는 Z 레지스터를 사용하여 프로그램 메모리를 이와 유사하게 액세스)

ATmega128의 명령 세트 – 어드레싱 모드(6)

- 동작 후에 자동증가 기능이 있는 데이터 간접 번지 지정 (Data Indirect Addressing with Post-increment)

ex) 소스 오퍼랜드에 사용되는 경우

```
LD      Rd, X+          ;  Rd ←— (X),   X ←— X + 1
```

X, Y, Z 레지스터

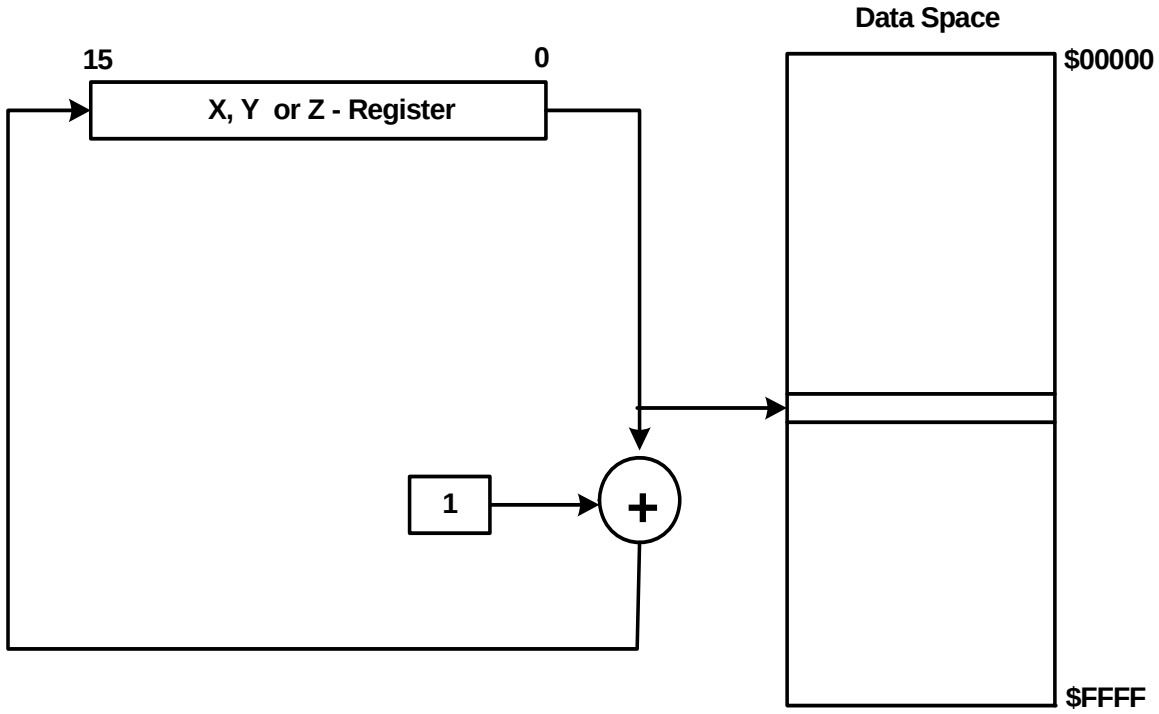
ex) 데스티네이션 오퍼랜드에 사용되는 경우

```
ST      X+, Rr          ;  (X) ←— Rr,   X ←— X + 1
```

X, Y, Z 레지스터

ATmega128의 명령 세트 – 어드레싱 모드(6)

- 동작 후에 자동증가 기능이 있는 데이터 간접 번지 지정 (Data Indirect Addressing with Post-increment)



자동증가 기능이 있는 데이터 간접 번지 지정

ATmega128의 명령 세트 – 어드레싱 모드(7)

- 동작 전에 자동감소 기능이 있는 데이터 간접 번지 지정 (Data Indirect Addressing with Pre-decrement)
  - 데이터 간접 번지 지정과 동일하지만 동작을 수행하기 전에 이 명령에서 지정된 X, Y, Z 레지스터를 1만큼 감소시키는 기능을 추가적으로 가지는 방법
  - 번지를 감소시키면서 데이터 메모리 영역을 연속적으로 읽거나 쓰는 동작에 유용
  - LD/ST 명령에서만 사용

ATmega128의 명령 세트 – 어드레싱 모드(7)

- 동작 전에 자동감소 기능이 있는 데이터 간접 번지 지정 (Data Indirect Addressing with Pre-decrement)

ex) 소스 오퍼랜드에 사용되는 경우

```
LD      Rd, -X          ;  X ←— X - 1,   Rd ←— (X)
```

X, Y, Z 레지스터

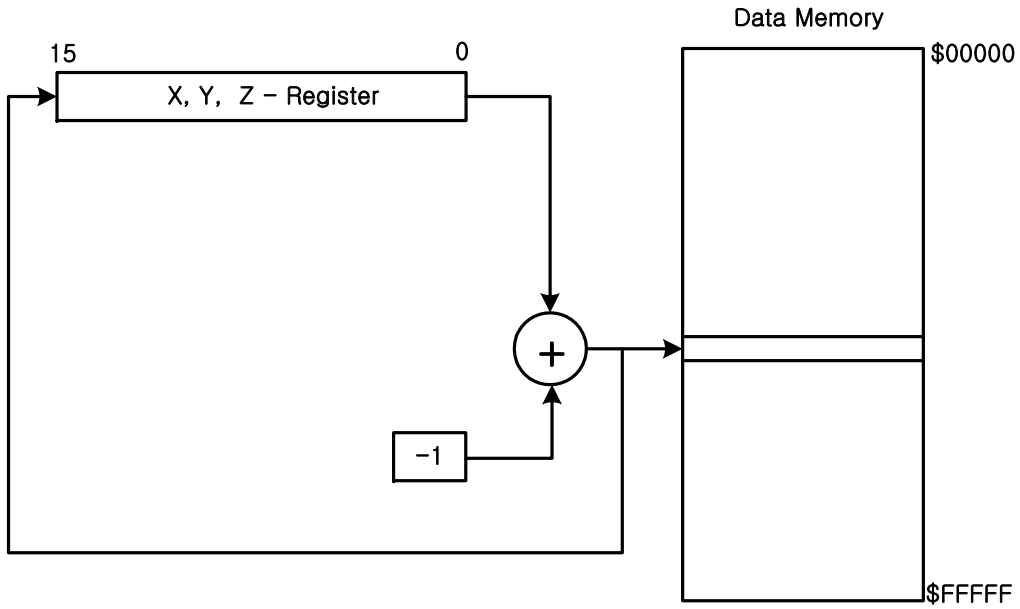
ex) 데스티네이션 오퍼랜드에 사용되는 경우

```
ST      -X, Rr          ;  X ←— X - 1,   (X) ←— Rr
```

X, Y, Z 레지스터

ATmega128의 명령 세트 – 어드레싱 모드(7)

- 동작 전에 자동감소 기능이 있는 데이터 간접 번지 지정 (Data Indirect Addressing with Pre-decrement)



Data Indirect Addressing with Pre-decrement

ATmega128의 명령 세트 – 어드레싱 모드(8)

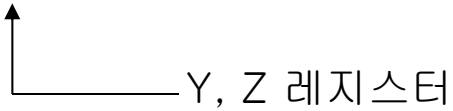
- 변위가 있는 데이터 간접 번지 지정 (Data Indirect Addressing with Displacement)
  - 데이터 간접 번지 지정과 유사하지만 어드레스 포인터로서 Y, Z 레지스터에 변위를 합하여 사용하는 방법
  - 베이스 레지스터로는 Y 레지스터와 Z 레지스터만을 사용할 수 있음
  - 변위는 6bit의 양의 정수로 표현되어 0~63 범위를 사용
  - 베이스 번지를 기준으로 하는 데이터 테이블을 처리하는 동작에 유용
  - LDD/STD 명령에서만 사용

ATmega128의 명령 세트 – 어드레싱 모드(8)

- 변위가 있는 데이터 간접 번지 지정 (Data Indirect Addressing with Displacement)

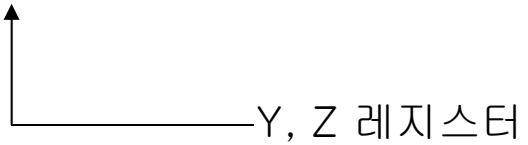
ex) 소스 오퍼랜드에 사용되는 경우

```
LDD     Rd, Y+q          ;  Rd ←── ( Y + q )
```



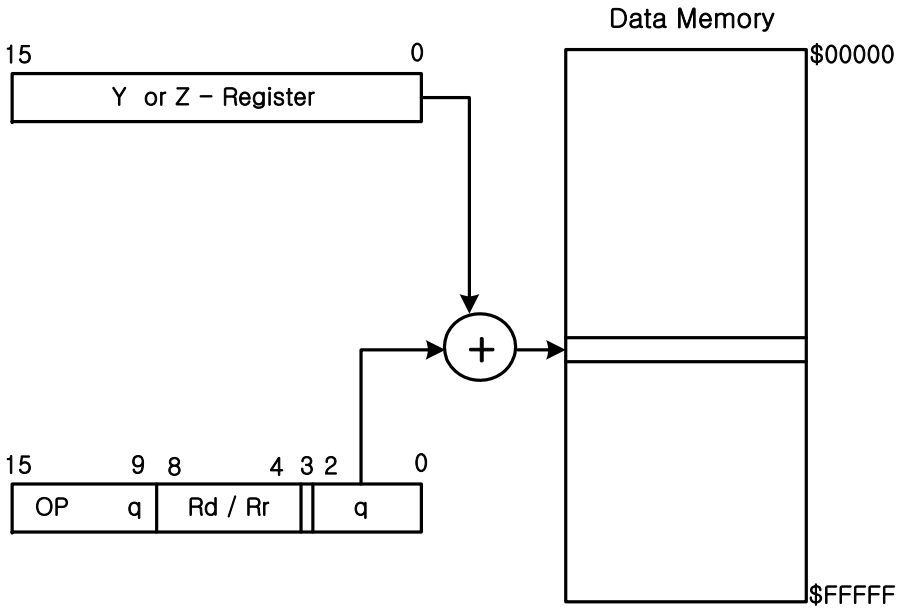
ex) 데스티네이션 오퍼랜드에 사용되는 경우

```
STD     Y+q, Rr          ;  ( Y + q ) ←── Rr
```



ATmega128의 명령 세트 – 어드레싱 모드(8)

- 변위가 있는 데이터 간접 번지 지정 (Data Indirect Addressing with Displacement)



Data Indirect with Displacement

ATmega128의 명령 세트 – 어드레싱 모드(9)

- 프로그램 메모리 상수 번지 지정 (Program Memory Constant Addressing)
  - 어드레스 포인터로서 Z 레지스터를 사용하여 프로그램 메모리 영역의 상수를 액세스하는 방법
  - 프로그램 메모리는 64K * 16bit의 구조를 가지고 있지만, 어드레싱 모드에서는 상수를 바이트 단위로만 액세스하므로 프로그램 메모리를 128K * 8bit의 구조로 간주
  - 16bit 어드레스 포인터로는 모두 지정하지 못하므로 프로그램 메모리를 64K * 8bit 용량의 2개 페이지로 구분하여 사용
  - LPM 명령에서는 무조건 첫번째 페이지만을 사용
  - ELPM/SPM 명령에서는 RAMPZ 레지스터를 이용하여 사용하려는 페이지 지정할 수 있음
  - LPM/ELPM/SPM 명령에서만 사용

ATmega128의 명령 세트 – 어드레싱 모드(9)

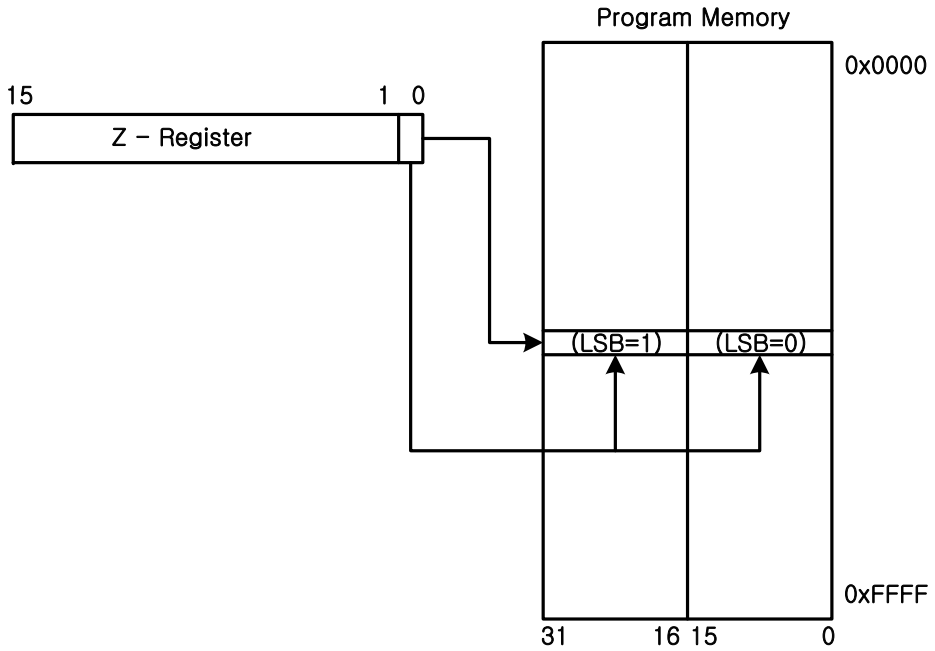
- 프로그램 메모리 상수 번지 지정 (Program Memory Constant Addressing)

  ex) LPM 명령의 경우

```
LPM                 ; R0 ←——— (Z)
LPM     Rd, Z       ; Rd ←——— (Z)
```

ATmega128의 명령 세트 – 어드레싱 모드(9)

- 프로그램 메모리 상수 번지 지정 (Program Memory Constant Addressing)



Code Memory Constant Addressing

ATmega128의 명령 세트 – 어드레싱 모드(10)

- 자동증가 기능이 있는 프로그램 메모리 상수 번지 지정 (Program Memory Constant Addressing with Post-increment)
  - 프로그램 메모리 상수 번지 지정과 동일하지만 동작 후에 Z 레지스터를 1만큼 증가시켜 두는 기능을 추가로 가지고 있는 방법
  - LPM/ELPM 명령에서만 사용

ATmega128의 명령 세트 – 어드레싱 모드(10)

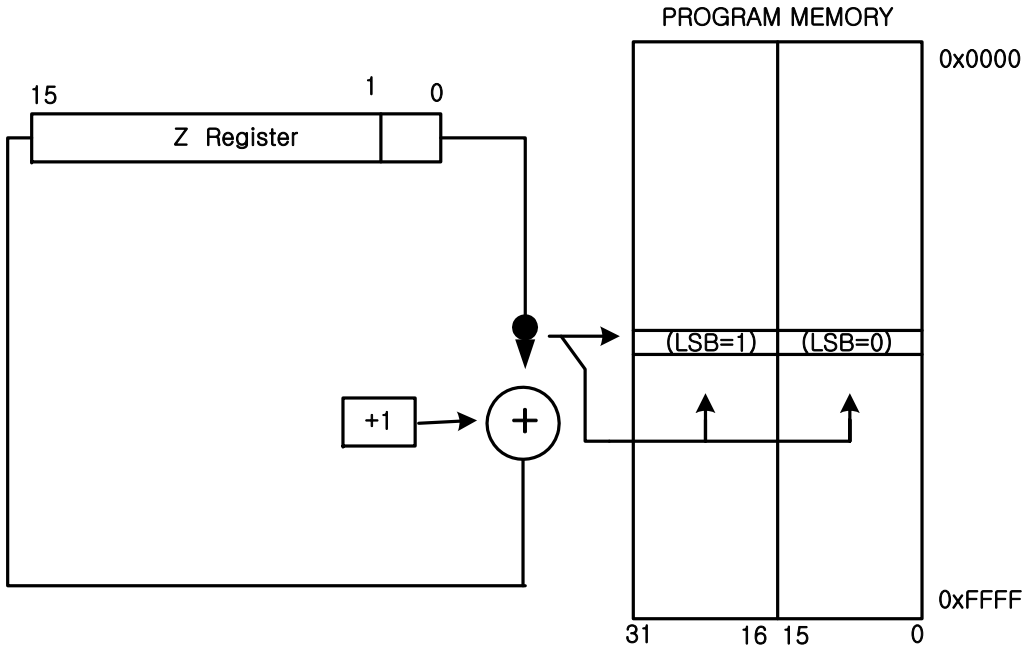
- 자동증가 기능이 있는 프로그램 메모리 상수 번지 지정 (Program Memory Constant Addressing with Post-increment)

  ex) LPM 명령의 경우

```
LPM     Rd, Z+          ; Rd ←——— (Z),   Z ←——— Z + 1
```

ATmega128의 명령 세트 – 어드레싱 모드(10)

- 자동증가 기능이 있는 프로그램 메모리 상수 번지 지정 (Program Memory Constant Addressing with Post-increment)



Relative Program Memory Addressing

ATmega128의 명령 세트 – 어드레싱 모드

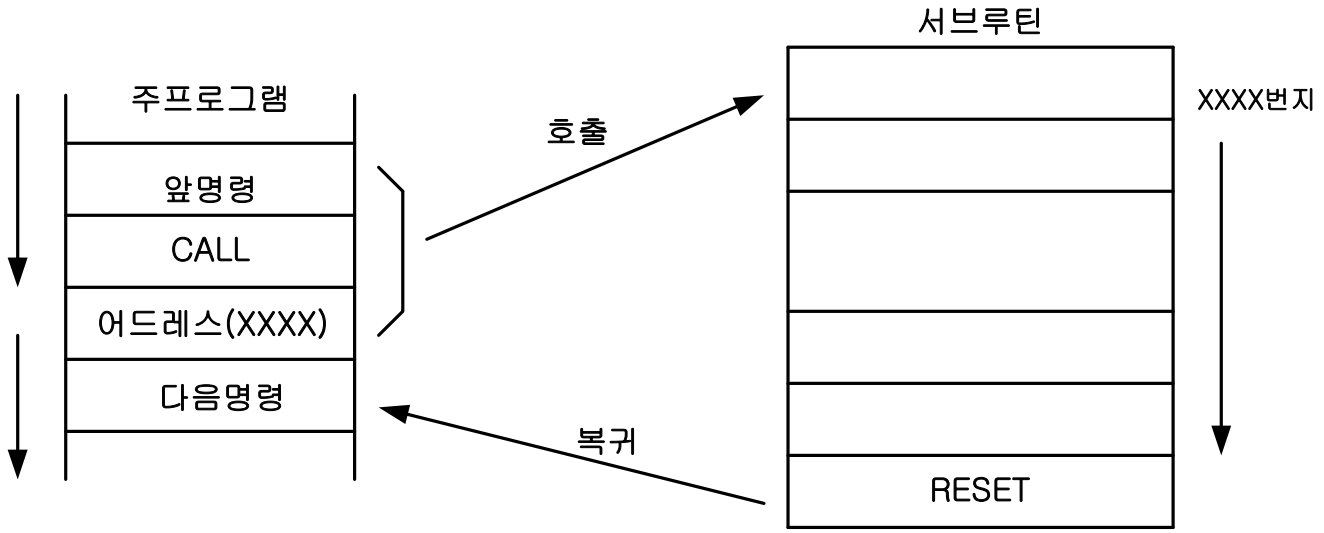
DATA TRANSFER INSTRUCTIONS					
MOV	Rd, Rr	Move Between Registers	Rd ← Rr	None	1
MOVW	Rd, Rr	Copy Register Word	Rd+1:Rd ← Rr+1:Rr	None	1
LDI	Rd, K	Load Immediate	Rd ← K	None	1
LD	Rd, X	Load Indirect	Rd ← (X)	None	2
LD	Rd, X+	Load Indirect and Post-Inc.	Rd ← (X), X ← X + 1	None	2
LD	Rd, - X	Load Indirect and Pre-Dec.	X ← X - 1, Rd ← (X)	None	2
LD	Rd, Y	Load Indirect	Rd ← (Y)	None	2
LD	Rd, Y+	Load Indirect and Post-Inc.	Rd ← (Y), Y ← Y + 1	None	2
LD	Rd, - Y	Load Indirect and Pre-Dec.	Y ← Y - 1, Rd ← (Y)	None	2
LDD	Rd,Y+q	Load Indirect with Displacement	Rd ← (Y + q)	None	2
LD	Rd, Z	Load Indirect	Rd ← (Z)	None	2
LD	Rd, Z+	Load Indirect and Post-Inc.	Rd ← (Z), Z ← Z+1	None	2
LD	Rd, -Z	Load Indirect and Pre-Dec.	Z ← Z - 1, Rd ← (Z)	None	2
LDD	Rd, Z+q	Load Indirect with Displacement	Rd ← (Z + q)	None	2
LDS	Rd, k	Load Direct from SRAM	Rd ← (k)	None	2
ST	X, Rr	Store Indirect	(X) ← Rr	None	2
ST	X+, Rr	Store Indirect and Post-Inc.	(X) ← Rr, X ← X + 1	None	2
ST	- X, Rr	Store Indirect and Pre-Dec.	X ← X - 1, (X) ← Rr	None	2
ST	Y, Rr	Store Indirect	(Y) ← Rr	None	2
ST	Y+, Rr	Store Indirect and Post-Inc.	(Y) ← Rr, Y ← Y + 1	None	2
ST	- Y, Rr	Store Indirect and Pre-Dec.	Y ← Y - 1, (Y) ← Rr	None	2
STD	Y+q,Rr	Store Indirect with Displacement	(Y + q) ← Rr	None	2
ST	Z, Rr	Store Indirect	(Z) ← Rr	None	2
ST	Z+, Rr	Store Indirect and Post-Inc.	(Z) ← Rr, Z ← Z + 1	None	2
ST	-Z, Rr	Store Indirect and Pre-Dec.	Z ← Z - 1, (Z) ← Rr	None	2
STD	Z+q,Rr	Store Indirect with Displacement	(Z + q) ← Rr	None	2
STS	k, Rr	Store Direct to SRAM	(k) ← Rr	None	2
LPM		Load Program Memory	R0 ← (Z)	None	3
LPM	Rd, Z	Load Program Memory	Rd ← (Z)	None	3
LPM	Rd, Z+	Load Program Memory and Post-Inc	Rd ← (Z), Z ← Z+1	None	3
ELPM		Extended Load Program Memory	R0 ← (RAMPZ:Z)	None	3
ELPM	Rd, Z	Extended Load Program Memory	Rd ← (RAMPZ:Z)	None	3
ELPM	Rd, Z+	Extended Load Program Memory and Post-Inc	Rd ← (RAMPZ:Z), RAMPZ:Z ← RAMPZ:Z+1	None	3
SPM		Store Program Memory	(Z) ← R1:R0	None	-
IN	Rd, P	In Port	Rd ← P	None	1
OUT	P, Rr	Out Port	P ← Rr	None	1
PUSH	Rr	Push Register on Stack	STACK ← Rr	None	2
POP	Rd	Pop Register from Stack	Rd ← STACK	None	2

ATmega128의 명령 세트 – 서브루틴과 스택

- 서브루틴(subroutine)이란?
  - 동일한 처리루틴이 한 프로그램내의 여러 곳에서 사용할 경우에 이용
  - 별도의 독립된 루틴으로 작성하여 두고 필요할 때마다 주 루틴(main routine)에서 이를 호출(call)하여 사용
  - 하나의 프로그램이 쓸데없이 길어지는 것을 방지할 수 있음
  - 소스 프로그램의 입력이 간단해지며 나중에 프로그램을 수정할 경우에도 서브루틴 한 곳만 수정하면 되므로 관리가 간편
  - 모듈식 프로그램의 작성에 매우 편리

ATmega128의 명령 세트 – 서브루틴과 스택

❑ 서브루틴



<서브루틴 호출에서 리턴까지의 실행 과정>

ATmega128의 명령 세트 – 서브루틴과 스택

❑ 서브루틴 구조

- 주프로그램에서의 호출에 의해 실행이 개시되고, 나중에 복귀될 주소가 스택에 저장
- 실행 종료는 복귀명령에 의해 인식되고, 스택에 저장되었던 복귀주소가 되찾아져서 서브루틴 호출이 이루어졌던 바로 다음의 명령으로 제어가 넘겨짐
- 주프로그램의 앞이나 뒤 어느 곳에 위치해도 관계없음
- 주프로그램에 의한 호출 및 복귀에 의해서만 수행되어야 하며, 정상적으로 메모리의 번지순에 의하여 프로그램을 수행하다가 이 부분이 수행되어서는 안됨
- 특히 주요프로그램에서와 서브루틴에서 각각 레지스터의 사용이 중복되지 않도록 주의

ATmega128의 명령 세트 – 서브루틴과 스택

❑ 서브루틴 구조

- 입력 파라미터와 출력 파라미터를 사용하면 범용성을 높일 수 있음
- 점프 명령이나 서브루틴 호출 명령은 프로그램의 실행 순서를 변경하여 지정된 번지로 점프한다는 점에서 유사
- 점프명령에서는 다시 리턴하는 것이 전제되지 않는 것에 비하여 서브루틴 명령에서는 점프명령 전에 원래의 위치로 리턴할 수 있도록 복귀 주소를 스택에 저장
- 인터럽트와 매우 유사하지만, 발생 원인이나 처리 방법에 있어서 다른 점이 있음

ATmega128의 명령 세트 – 서브루틴과 스택

❑ 서브루틴 구조

서 브 루 틴	인 터 럽 트
▸ 동일한 처리를 프로그램내의 여러 곳에서 필요로 할 때 사용 ▸ **CPU**의 서브루틴 호출 명령에 의해 동작이 시작 ▸ 점프하기 전에 복귀주소를 스택에 자동으로 저장 ▸ 별도의 처리 루틴으로 제어가 옮겨져서 프로그램을 수용 ▸ 복귀 명령(RET)에 의해 원래의 위치로 복귀 ▸ 대부분 입력 및 출력 파라미터가 있음	▸ 주변장치가 CPU에게 긴급한 처리를 요구할 때 사용 ▸ 주변장치가 CPU에게 하드웨어적인 인터럽트 요구신호를 보냄으로써 시작 ▸ 점프하기 전에 복귀주소를 스택에 자동으로 저장 ▸ 별도의 처리 루틴으로 제어가 옮겨져서 프로그램을 수행 ▸ 복귀명령(RETI)에 의해 원래의 위치로 복귀 ▸ 입력 및 출력 파라미터가 없음.

<서브루틴과 인터럽트의 비교>

ATmega128의 명령 세트 – 서브루틴과 스택

- 서브루틴 종류 (닫힌 서브루틴과 열린 서브루틴)
  - 닫힌 서브루틴
    - 일반적으로 그냥 서브루틴이라고도 함
    - 주프로그램에서의 호출명령에 의해서 CPU의 실행제어가 서브루틴으로 넘겨져서 처리를 수행
    - 서브루틴의 실행이 종료되어 복귀 명령을 만나면 다시 주루틴으로 실행제어가 되돌아가는 방법

ATmega128의 명령 세트 – 서브루틴과 스택

- 서브루틴 종류 (닫힌 서브루틴과 열린 서브루틴)
  - 열린 서브루틴
    - 별도로 분리되어 작성된 서브루틴 전체가 프로그램이 어셈블될 때 주프로그램의 일부로서 확장되어 삽입되는 방식
    - 열린 서브루틴으로 작성된 소스는 서브루틴이 따로 남아있지 않고 주프로그램의 일부가 되어 주프로그램의 실행파일이 길어짐
    - CPU가 서브루틴을 호출하거나 리턴하는 절차가 없으므로 실행시간의 지연이 없음

ATmega128의 명령 세트 – 서브루틴과 스택

❑ 스택

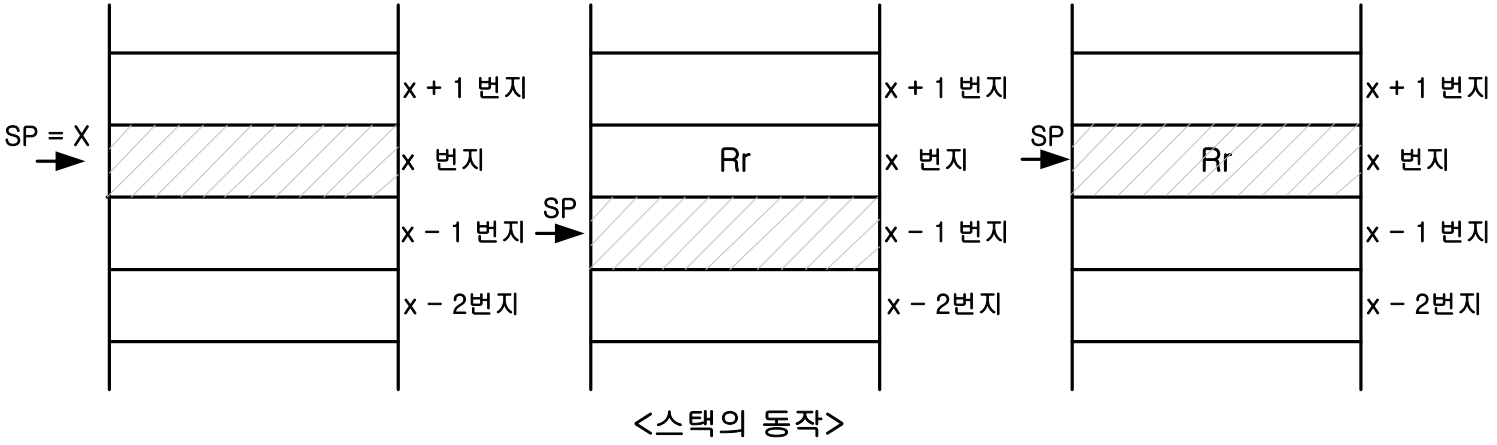
- 데이터 메모리인 SRAM의 일부를 특별한 데이터 저장 영역으로 할당하여 두고 LIFO(Last-Input Fast-Output) 방식으로만 액세스하여 사용하는 메모리

- 데이터를 원하는 순서대로 저장하였다가 나중에 다시 순서대로 꺼낼 수만 있으면 되는 경우에 사용

- ATmega128에서 스택 포인터는 스택에 1바이트의 데이터를 저장하고 나서 1만큼 값이 감소하고, 스택에서 1바이트의 데이터를 꺼내기 전에 1만큼 값이 증가하는 방식으로 동작하므로 스택에 데이터를 저장할수록 스택 포인터가 낮은 번지로 감소하고, 데이터를 꺼내면 원래의 위치로 올라감

ATmega128의 명령 세트 – 서브루틴과 스택

❑ 스택



<스택의 동작>

ATmega128의 기본 하드웨어 – 메모리 록 비트와 퓨즈 비트

❑ 메모리 록 비트

- 메모리 록 비트는 메모리 프로그래밍에 관련된 보호기능을 설정하는데 사용되며, 1바이트 중에서 6개의 비트만 사용
- 이 비트들은 모두 디폴트 되어 있으며, 프로그램하면 0으로 설정되지만 칩삭제 명령을 수행하면 이 비트들은 다시 1로 환원

Lock Bit Byte	Bit No.	Description	Default Value
	7	–	1 (unprogrammed)
	6	–	1 (unprogrammed)
BLB12	5	Boot lock bit	1 (unprogrammed)
BLB11	4	Boot lock bit	1 (unprogrammed)
BLB02	3	Boot lock bit	1 (unprogrammed)
BLB01	2	Boot lock bit	1 (unprogrammed)
LB2	1	Lock bit	1 (unprogrammed)
LB1	0	Lock bit	1 (unprogrammed)

ATmega128의 기본 하드웨어 – 메모리 록 비트와 퓨즈 비트

메모리 록 비트			메모리 보호기능
LB 모드	**LB2**	**LB1**	외부에서 플래시 메모리를 프로그래밍 하는 기능에 대한 보호 설정
1	**1**	**1**	아무 메모리 록 기능도 설정되지 않는다
2	**1**	**0**	플래시나 EEPROM을 병렬모드나 SPI/JTAG 직렬모드로 더 이상 프로그래밍을 금지시킨다. 퓨즈비트도 마찬가지다.
3	**0**	**0**	플래시나 EEPROM을 병렬모드나 SPI/JTAG 직렬모드로 더 이상 프로그래밍하거나 verify하는 것을 금지 시킨다.
BLB0모드	**BLB02**	**BLB01**	실행 프로그램으로 응용 프로그램 섹션을 액세스하는데 대한 보호설정
1	**1**	**1**	**LPM/ELPM/SPM** 명령을 사용하여 프로그램의 응용 프로그램 섹션을 액세스하는데 아무 제한이 설정되지 않는다.
2	**1**	**0**	**SPM** 명령으로 프로그램 메모리의 응용 프로그램 섹션을 **write**하는 것이 금지된다.
3	**0**	**0**	**SPM** 명령으로 프로그램 메모리의 응용 프로그램 섹션을 라이트 하는 것이 금지되고 만약 부트로더 섹션에 인터럽트 벡터가 위치한다면 응용 프로그램 섹션이 실행되는 동안에 모든 인터럽트는 금지된다
4	**0**	**1**	부트로더 섹션에서 실행하는 LPM/ELPM 명령으로 프로그램 메모리의 응용 프로그램 섹션을 read하는것이 금지된다
BLB1모드	**BLB12**	**BLB11**	실행 프로그램으로 부트로더 섹션을 액세스하는데 대한 보호설정
1	**1**	**1**	**LPM/ELPM/SPM** 명령을 사용하여 프로그램의 부트로더 섹션을 액세스하는데 아무 제한이 설정되지 않는다.
2	**1**	**0**	**SPM** 명령으로 프로그램 메모리의 부트로더 섹션을 **write**하는 것이 금지된다.
3	**0**	**0**	**SPM** 명령으로 프로그램 메모리의 부트로더 섹션을 라이트 하는 것이 금지되고 만약 응용 프로그램 섹션에 인터럽트 벡터가 위치한다면 부트로더 섹션이 실행되는 동안에 모든 인터럽트는 금지된다
4	**0**	**1**	응용 프로그램 섹션에서 실행하는 LPM/ELPM 명령으로 프로그램 메모리의 부트로더 섹션을 read하는 것이 금지 된다

ATmega128의 기본 하드웨어 – 메모리 록 비트와 퓨즈 비트

- 퓨즈비트
  - 퓨즈 비트는 MCU의 기본적인 시스템 설정용으로 사용되며 모두 3바이트로 구성
  - 이 비트들은 프로그램하지 않으면 1(unprogrammed)로 설정되며, 프로그램하면 0(programmed)으로 설정
  - 칩삭제 명령을 설정하더라도 퓨즈 비트는 영향을 받지 않으며, 메모리 록 비트의 LB1을 사용하여 퓨즈비트를 변경할 수 없도록 보호(lock) 기능을 설정할 수 있음
  - 퓨즈 비트를 먼저 설정하고 메모리 록 비트를 나중에 설정해야 함
  - 퓨즈 비트를 설정하는 첫 번째 바이트는 Extended Fuse Bit로서 1바이트 중에서 2개의 비트만 사용

ATmega128의 기본 하드웨어 – 메모리 록 비트와 퓨즈 비트

- 퓨즈비트 (제 1바이트)

<Extended Fuse Byte의 비트구성>

비트 번호	비트 이름	기 능	디폴트 값
7	-	-	1
6	-	-	1
5	-	-	1
4	-	-	1
3	-	-	1
2	-	-	1
1	M103C	ATmega103 호환모드	0(programmed)
0	WDTON	Watchdog Timer 동작	1(unprogrammed)

ATmega128의 기본 하드웨어 – 메모리 록 비트와 퓨즈 비트

❑ 퓨즈비트 (제 2바이트)

- 퓨즈 비트를 설정하는 두 번째 바이트는 Fuse High Byte로서 ATmega128은 디폴트 상태에서 JTAG 과 SPI에 의한 프로그램이 가능한 상태에 있으며, 칩 삭제에서 EEPROM은 보호되지 않음

<Fuse High Byte의 비트구성>

비트 번호	비트 이름	기 능	디폴트 값
7	OCDEN	OCD 사용가능	1(unprogrammed)
6	JTAGEN	JTAG 사용가능	0(programmed)
5	SPIEN	SPI를 통한 직렬 프로그래밍 가능	0(programmed)
4	CKOPT	오실레이터 옵션	1(unprogrammed)
3	EESAVE	Chip Erase시에 EEPROM 내용 보호	1(unprogrammed)
2	BOOTSZ1	Boot Size 선택	0(programmed)
1	BOOTSZ0	Boot Size 선택	0(programmed)
0	BOOTRST	Reset Vector 선택	1(unprogrammed)

ATmega128의 기본 하드웨어 – 메모리 록 비트와 퓨즈 비트

- 퓨즈비트 (제 2바이트)
  - 부트 사이즈는 0xF000~0xFFF영역의 4096워드가 됨

<Boot Size 설정>

BOOTSZ1	BOOTSZ0	Boot Size	응용프로그램 섹션	부트로더 섹션
1	1	512워드(4페이지)	0x0000~0xFDFF	0xFE00~0xFFFF
1	0	1024워드(8페이지)	0x0000~0xFBFF	0xFC00~0xFFFF
0	1	2048워드(16페이지)	0x0000~0xF7FF	0xF800~0xFFFF
0	0	4096워드(32페이지)	0x0000~0xEFFF	0xF000~0xFFFF

ATmega128의 기본 하드웨어 – 메모리 록 비트와 퓨즈 비트

❑ 퓨즈비트 (제 3바이트)

- 퓨즈 비트를 설정하는 세 번째 바이트는 Fuse Low Byte로서 디폴트 상태에서 BOD 기능이 사용되지 않는 상태에 있으며, Start-up 시간은 최대값으로 되고 시스템 클럭은 내부 RC오실레이터 의하여 1MHz로 발진됨

<Fuse Low Byte의 비트구성>

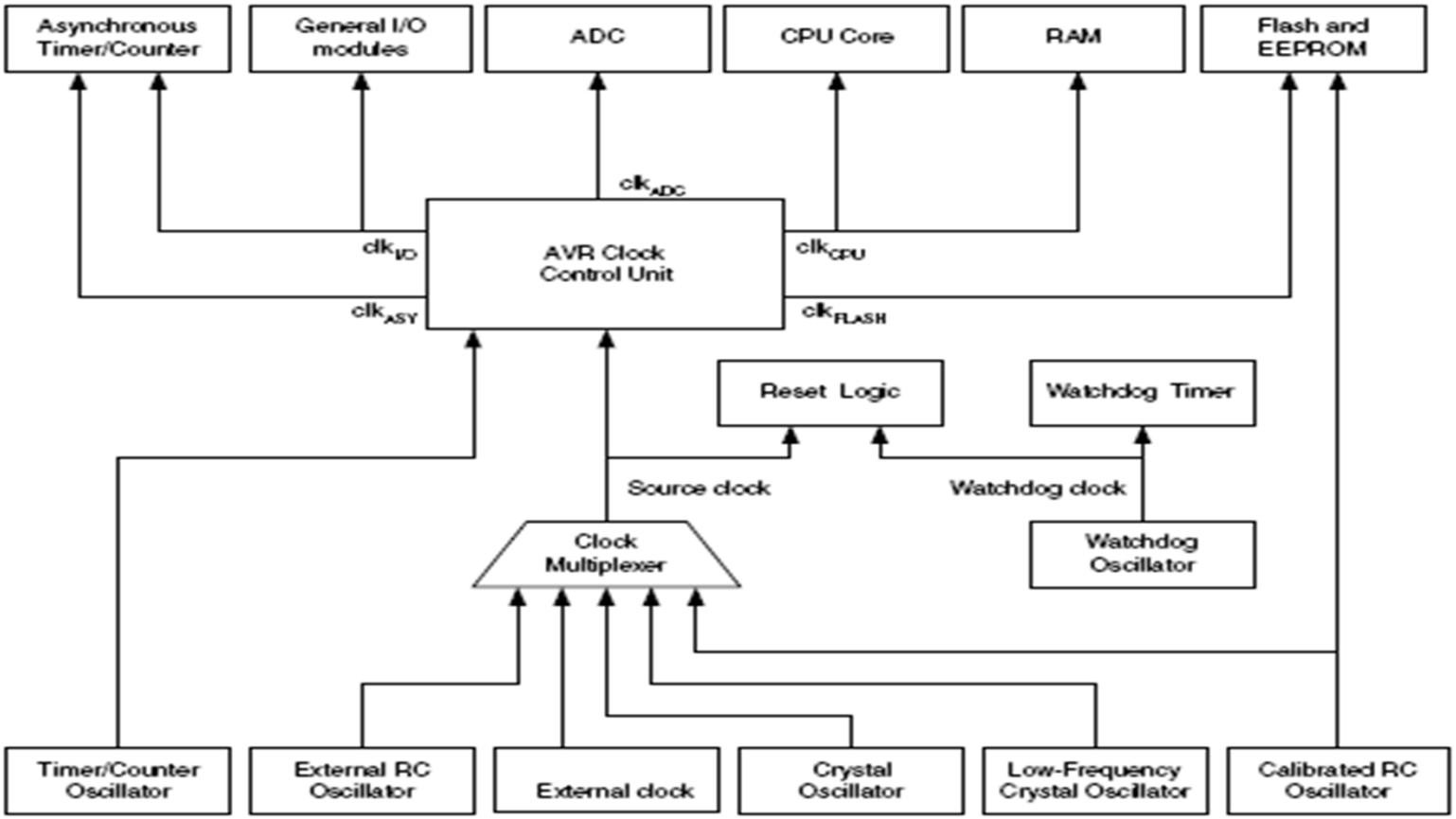
비트 번호	비트 이름	기 능	디폴트 값
7	BODEVEL	BOD(Brown Out Detector)	1(unprogrammed)
6	BODEN	BOD 기능을 사용하도록 허용	1(unprogrammed)
5	SUT1	기동 시간을 선택	1(unprogrammed)
4	SUT0		0(programmed)
3	CKSEL3	클록 소스를 선택	0(programmed)
2	CKSEL2		0(programmed)
1	CKSEL1		0(programmed)
0	CKSEL0		1(unprogrammed)

ATmega128의 기본 하드웨어 – 시스템 클럭

- 시스템 클럭
  - ATmega128에서는 시스템 클럭을 발생할 수 있는 방법이 여러 가지가 있음
  - MCU 내부의 각 구성 부분들은 모두 동시에 클럭을 필요로 하지는 않음
  - 소비전력을 감소시키기 위하여 각 부분에 별도의 클럭을 공급하고 개별적으로 슬립 모드를 사용하여 이들 클럭의 공급을 차단할 수 있음

ATmega128의 기본 하드웨어 – 시스템 클럭

<ATmega128의 클럭 발생 및 배분 블록도>



ATmega128의 기본 하드웨어 – 시스템 클럭

❑ 클럭 소스의 선택

- ATmega128에서 사용할 수 있는 클럭 소스는 모두 5가지가 있는데, 이것들은 퓨즈 비트 설정용의 3번째 바이트(Fuse Low Byte)에 있는 4개의 비트 CKSEL3~0을 사용함

<클럭 소스의 선택>

클럭 소스		CKSEL3~0
External Crystal/Ceramic Reasonator		1111~1010
External Low-frequency Crystal		1001
External RC Oscillator	8.0~12.0MHz	1000
	3.0~8.0MHz	0111
	0.9~3.0MHz	0110
	0.0~0.9MHz	0101
Calibrated Internal RC Oscillator	8.0MHz	0100
	4.0MHz	0011
	2.0MHz	0010
	2.0MHz	0001
External Clock		0000

ATmega128의 기본 하드웨어 – 시스템 클럭

1) 외부 크리스탈 사용

- 외부에 크리스탈 또는 세라믹 레조네이터를 2개의 콘덴서와 함께 MCU의 XTAL1 입력단자 및 XTAL2 출력단자에 연결하여 구성
- 플래시 퓨즈 비트 CKSEL0은 SUT1~0과 함께 기동시간(start-up time)을 선택하기 위해 사용
- 스타트 업 시간이란 MCU가 파워 다운모드 또는 파워 세이브모드부터 벗어나 정지해 있던 클럭이 안정되게 발생하여 MCU가 명령을 실행하기 시작할 때까지의 지연시간을 말함

ATmega128의 기본 하드웨어 – 시스템 클럭

2) 외부 저주파 크리스탈 사용

- 32.768KHz의 낮은 주파수를 시스템 클럭으로 사용하는 경우 CKSEL3~0를 1001로 설정

- CKOPT를 0으로 설정하는 경우에는 XTAL1과 XTAL2단자에 내부 콘덴서가 연결되므로 외부에 있는 콘덴서는 제거해야 함

ATmega128의 기본 하드웨어 – 시스템 클럭

3) 외부 RC 사용

- 외부 RC발진기는 정밀한 타이밍이 요구되지 않는 용도에 사용
- 외부저항 R과 콘덴서 C를 MCU의 XTAL1 단자에 연결
- 발진주파수는 대략 f = 1/(3RC)로 주어지며, C는 적어도 22pF 이상의 콘덴서를 사용
- CKOPT 퓨즈 비트를 0으로 하면 36pF의 내부 콘덴서가 XTAL1과 GND에 연결되므로 외부 콘덴서는 제거해야 함
- 발진기는 주파수 범위에 따라 4개의 다른 모드로 사용되며, SUT1~0 퓨즈 비트에 따라 스타트 업 시간과 지연시간이 정해짐

ATmega128의 기본 하드웨어 – 시스템 클럭

4) 내부 RC 사용

- 내부 RC 발진기는 1.0, 2.0, 4.0, 8.0MHz 중에서 퓨즈 비트 CKSEL3~0의 값에 따라 하나의 클럭이 선택되어 공급
- 이 때 CKOPT 퓨즈 비트값은 반드시 1이어야 하며, SUT1~0 퓨즈 비트에 따라 스타트 업 시간과 지연시간이 정해짐
- RC 발진기에서는 XTAL1 단자와 XTAL2 단자는 아무것도 연결하지 않고 오픈 상태로 둠
- 내부 RC발진기에서 출력되는 클럭은 시스템 클럭으로도 사용할 수 있지만, 플래시 메모리와 EEPROM을 액세스하는 클럭으로도 사용
- 내부 RC발진기에서 출력되는 클럭의 주파수는 8bit OSCCAL 레지스터를 이용하여 조정할 수 있음

#OSCCAL(발진기 조정 레지스터) : 클럭 주파수를 조정하는데 사용

ATmega128의 기본 하드웨어 – 시스템 클럭

5) 외부 클럭 사용

- 외부로부터 클럭이 XTAL1 단자로 직접 공급
- XTAL2 단자는 사용하지 않고 비워둬야함
- SUT1~0퓨즈 비트에 따라 스타트 업 시간과 지연시간이 정해짐

ATmega128의 기본 하드웨어 – 시스템 클럭

6) 크리스탈 분주 제어 레지스터(XDIV)

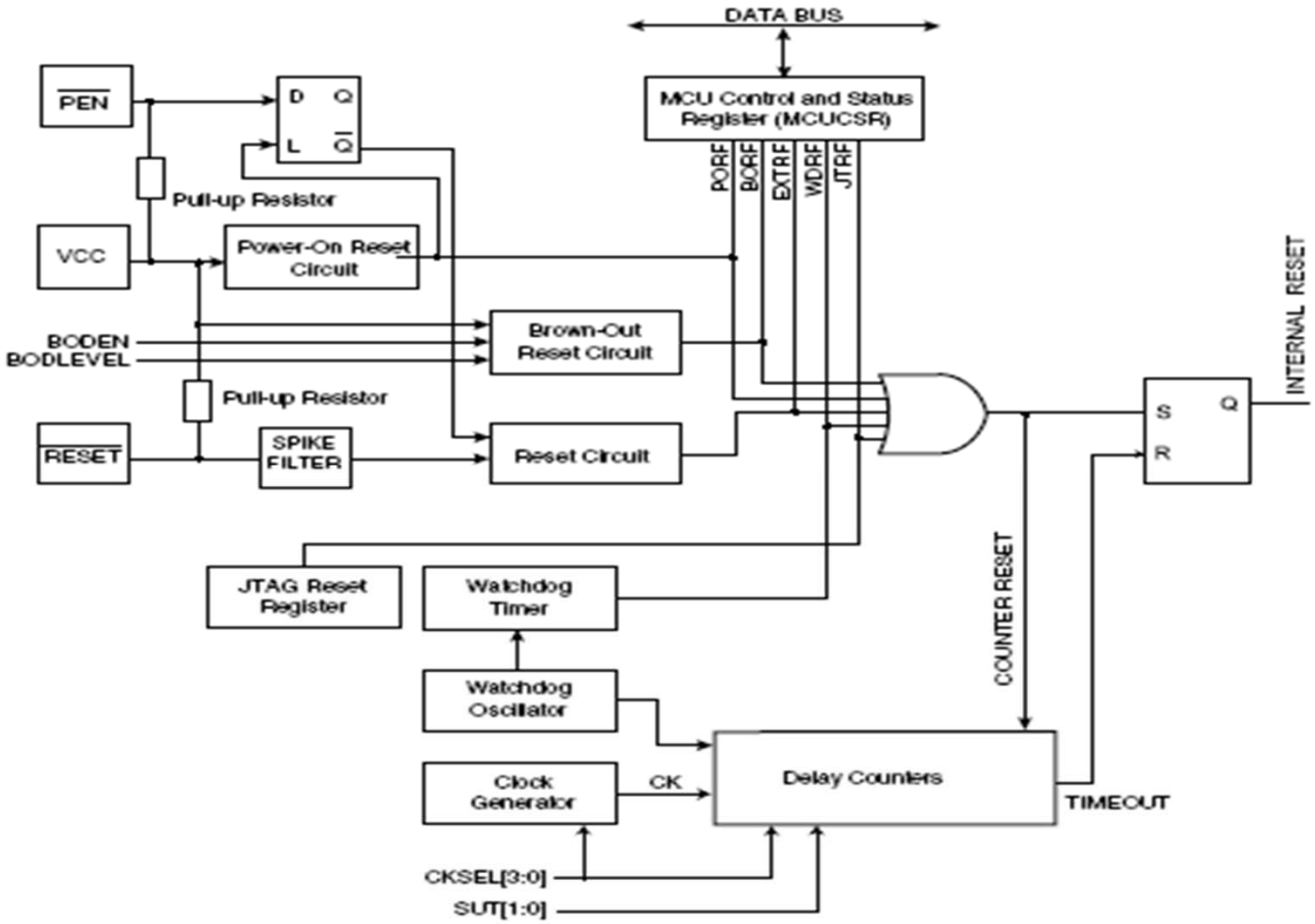
- XDIV 레지스터를 사용하면 소스 클럭을 2~129로 나누어 CPU와 모든 주변장치에 공급되는 클럭의 주파수를 낮출 수 있음
- 소비전력을 절감하는데도 유용

ATmega128의 기본 하드웨어 – 시스템 리셋

- 파워-온 리셋 : 전원이 공급될 때 MCU에 걸리는 리셋
- 워치-독 리셋 : 워치-독 타이머에 의해 MCU에 걸리는 리셋
- 브라운 아웃 검출 리셋 : 전원전압 Vcc가 지정된 최소시간 2us 이상 동안 브라운 아웃 리셋 임계 전압(2.7V 또는 4V)보다 낮은 경우 발생
- JTAG AVR 리셋 : JTAG 시스템에서 리셋 레지스터가 논리값 1인 동안 MCU에 걸리는 리셋

ATmega128의 기본 하드웨어 – 시스템 리셋

<리셋 내부 회로>



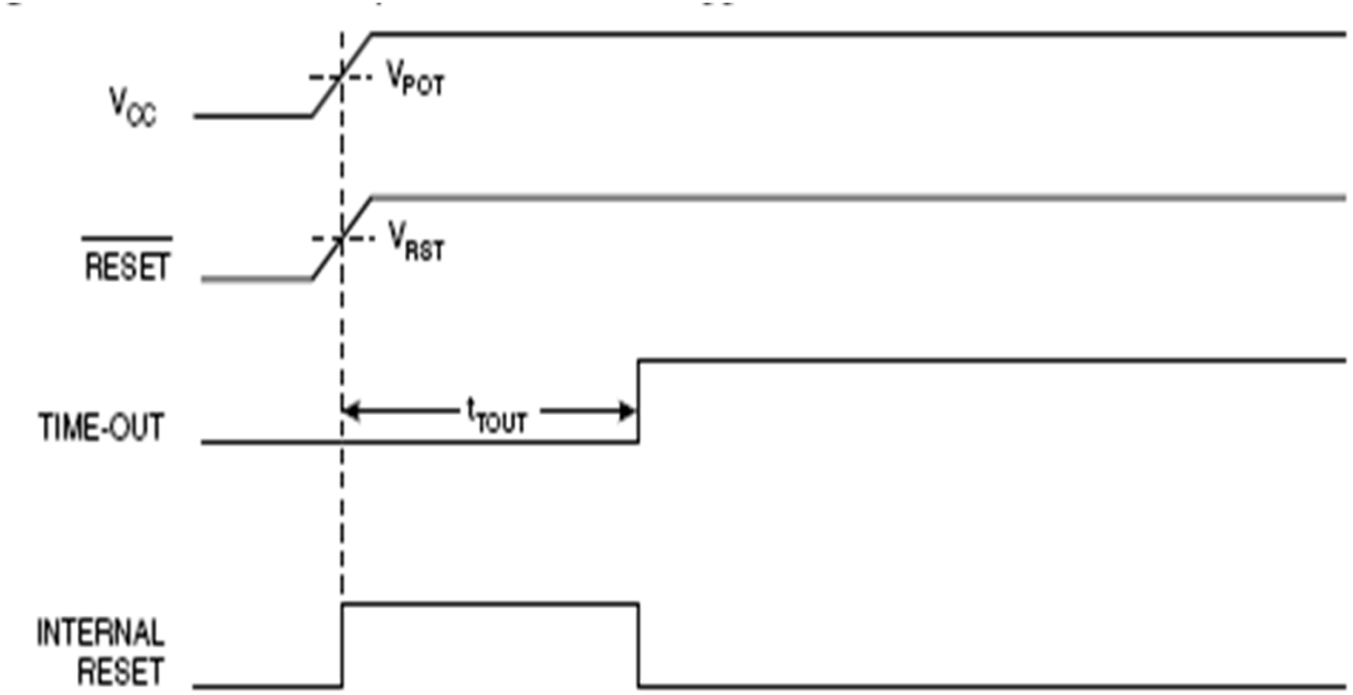
ATmega128의 기본 하드웨어 – 시스템 리셋

- 파워-온 리셋
  - 전원 공급 시 칩의 동작을 안전하게 함
  - Vcc가 인가된 후에 한 주기 동안 low로 유지
  - 내부 타이머는 워치-독 타이머로 카운트
  - 타이머는 Vcc가 파워-온 임계전압 Vpot도달한 후 일정시간 후까지 MCU가 동작하는 것을 제한
  - 플래시 내의 FSTRT 퓨즈 비트는 MCU 클럭으로 세라믹 레조네이터나 고속-스타트 오실레이터를 사용한다면 좀 더 짧은 스타트-업 시간으로 프로그램 가능
  - 스타트-업 지연이 충분하지 못하다면, RESET'은 풀-업 저항이나 Vcc에 직접 연결될 수 있음

ATmega128의 기본 하드웨어 – 시스템 리셋

- 파워-온 리셋

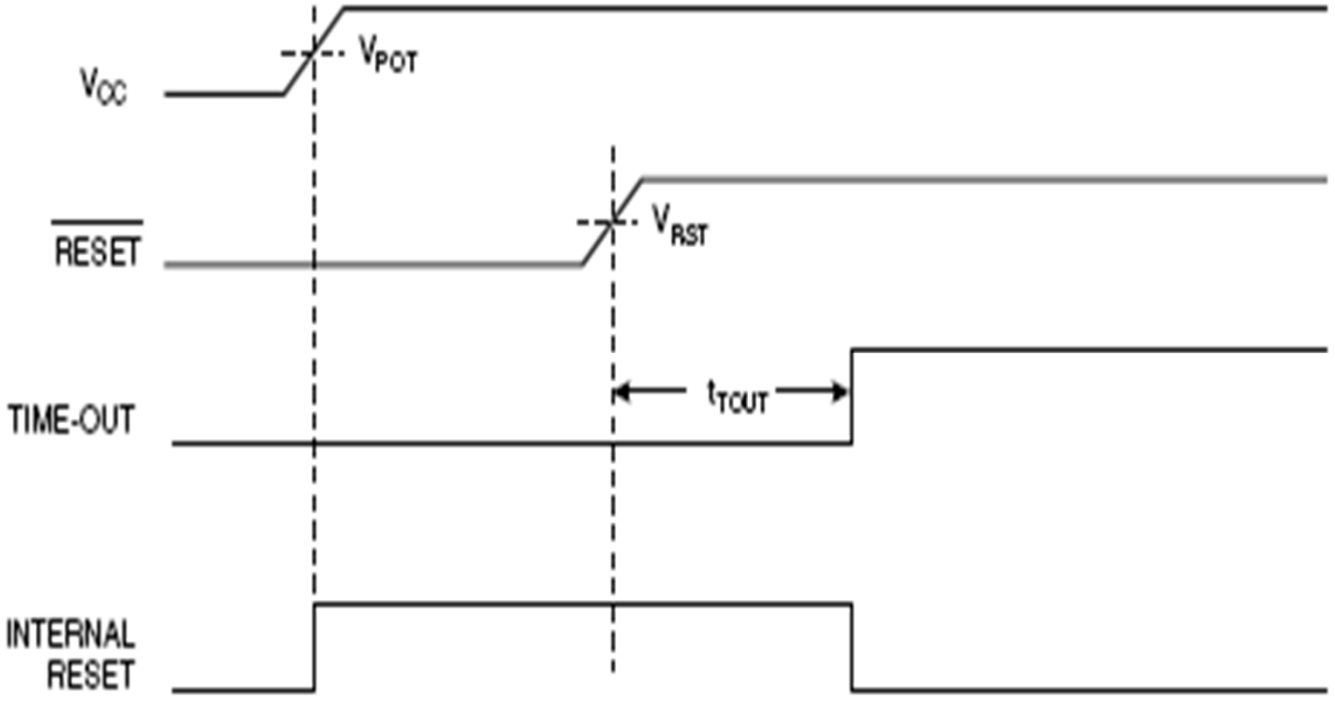
<MCU Start-up, Vcc와 RESET' 연결>



ATmega128의 기본 하드웨어 – 시스템 리셋

- 파워-온 리셋

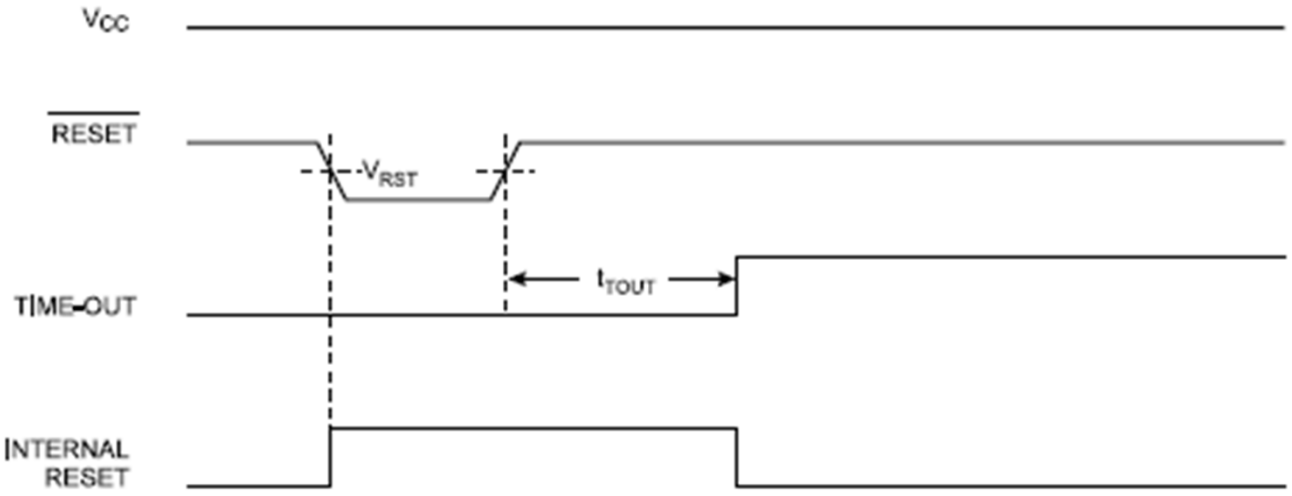
<MCU Start-up, Vcc와 RESET'의 외부제어>



ATmega128의 기본 하드웨어 – 시스템 리셋

- 외부 리셋
  - RESET'핀이 low신호가 입력되면 발생 1.5us이상의 리셋 펄스가 입력되어야만 리셋 신호가 발생
  - 짧은 리셋 펄스는 리셋 신호의 발생을 보증할 수 없음
  - 인가된 신호가 상승 에지에서 리셋 임계 전압에 이르면 타임-아웃 주기 발생 후에 MCU가 동작을 시작

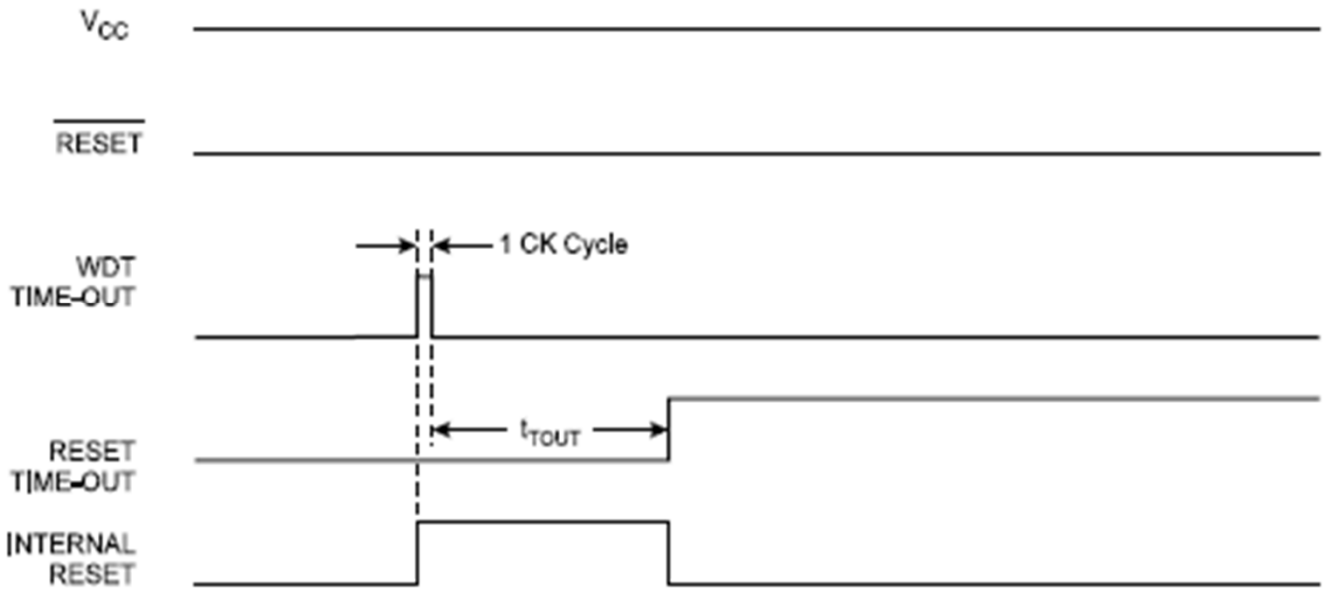
<외부 리셋 동작>



ATmega128의 기본 하드웨어 – 시스템 리셋

- 워치-독 리셋
  - 워치-독 타이머가 타임-아웃이 될 때, 1클럭 사이클 동안 리셋 펄스가 발생.
  - 이 펄스의 상승 에지에서 지연 타이머가 타임-아웃 주기 동안 유지

<워치-독 리셋 동작>



ATmega128의 기본 하드웨어 – 슬립 모드

- 슬립 모드(Sleep Mode)
  - MCU 내부에서 사용하지 않는 모듈들의 동작을 정지시켜(shut down) 소비전력을 절감하기 위한 동작
  - ATmega128에서는 6가지의 다양한 슬립 모드를 제공
  - MCUCR 레지스터에서 SE 비트를 1로 설정하고 SM2~0비트를 이용하여 슬립 모드를 6가지 중의 하나로 지정한 상태에서 SLEEP 명령을 실행
  - MCU가 슬립 모드에 있을 때 허용된 인터럽트가 발생되면 슬립 모드는 해제

ATmega128의 기본 하드웨어 – 슬립 모드

- 슬립 모드(Sleep Mode)
  - MCU는 기동 시간에 4사이클이 더 경과한 후에 동작이 개시되는데, 먼저 인터럽트 서비스 루틴이 실행되고 나서 SLEEP 명령의 바로 뒤에 이어지는 명령으로 복귀
  - 모든 레지스터와 SRAM에 있는 내용은 슬립 모드에 들어갔다가 복귀된 후에도 변경되지 않고 원래대로 유지
  - 슬립모드에서 리셋이 발생하면 슬립모드가 해제되고 정상적인 리셋 동작이 수행

ATmega128의 기본 하드웨어 – 슬립 모드

Bit	7	6	5	4	3	2	1	0	
	SRE	SRW10	SE	SM1	SM0	SM2	IVSEL	IVCE	MCUCR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

<MCUCR(MCU Control Register)>

SM2	SM1	SM0	슬립 모드
0	0	0	Idle
0	0	1	ADC Noise Register
0	1	0	Power-down
0	1	1	Power-save
1	0	0	(reserved)
1	0	1	(reserved)
1	1	0	Standby
1	1	1	Extended Standby

<슬립 모드의 지정>

ATmega128의 기본 하드웨어 – 슬립 모드

	Active Clock Domains					Oscillators		Wake Up Sources					
Sleep Mode	clk_{CPU}	clk_{FLASH}	clk_{IO}	clk_{ADC}	clk_{ASY}	Main Clock Source Enabled	Timer Osc Enabled	INT7:0	TWI Address Match	Timer 0	SPM/ EEPROM Ready	ADC	Other I/O
Idle			X	X	X	X	X(2)	X	X	X	X	X	X
ADC Noise Reduction				X	X	X	X(2)	X(3)	X	X	X	X	
Power-down								X(3)	X				
Power-save					X(2)		X(2)	X(3)	X	X(2)			
Standby(1)						X		X(3)	X				
Extended Standby(1)					X(2)	X	X(2)	X(3)	X	X(2)			

※1)외부 크리스탈 또는 레조네이터를 사용하는 경우에만 유효함

2)ASSR 레지스터의 AS0 비트를 1로 설정한 경우에만 유효함

3)INT7~4는 레벨 트리거 모드일 경우에만 유효함

<6가지 슬립 모드의 동작 요약>

ATmega128의 기본 하드웨어 – 슬립 모드

- Idle
  - CPU의 동작을 멈추지만, SPI, USART, 아날로그 비교기, ADC, TWI, 타이머/카운터, 워치독, 인터럽트 시스템 등의 동작은 계속됨
  - 이 모드에서는 CPU 클럭과 FLASH 클럭은 차단되지만 나머지 클럭은 모두 공급

ATmega128의 기본 하드웨어 – 슬립 모드

- ADC Noise Reduction
  - CPU의 동작을 멈추지만,ADC, TWI, 타이머/카운터, 워치독, 외부 인터럽트 등의 동작은 계속 됨
  - 이 모드에서는 CPU 클럭과 FLASH 클럭, I/O 클럭은 차단되지만 나머지 클럭은 모두 공급
  - 이 모드는 ADC가 정밀한 동작을 위하여 잡음을 덜 받도록 하기 위한 모드
  - ADC가 인에이블 되어 있다면 이 모드로 들어갈 때 변환이 자동으로 시작

ATmega128의 기본 하드웨어 – 슬립 모드

- Power-down
  - 외부 크리스탈 또는 세라믹 레조네이터 발진이 정지되며, TWI, 워치독, 외부 인터럽트 등의 동작은 계속됨
  - 이 모드에서는 모든 클럭이 차단되어 비동기 모듈의 동작만이 가능
  - 이 모드가 해제될 때는 클럭 발진회로가 안정 상태에 들어가는 충분한 시간이 요구되므로 기동시간이 필요한데 이는 사용자가 지정

ATmega128의 기본 하드웨어 – 슬립 모드

- Power-save
  - Power-down 모드와 유사하지만 1가지가 다른데 ASSR 레지스터에서 AS0 비트를 1로 설정하여 타이머/카운터0이 TOSC1과 TOSC2 단자로 입력되는 외부 클럭에 의하여 비동기로 동작할 때 이 모드를 사용
  - 이 모드에서 ASY 클럭을 제외한 모든 클럭이 차단
  - 그러나, 만약 ASSR 레지스터에서 AS0 비트를 0으로 설정하였다면 Power-save 모드 보다는 Power-down 모드를 사용하는 것이 좋음

ATmega128의 기본 하드웨어 – 슬립 모드

- Standby
  - 슬립 모드에서도 클럭 발생회로인 오실레이터가 동작하고 있다는 것을 제외하면 Power-down 모드와 유사
  - 이 모드는 외부 크리스탈 또는 레조네이터를 사용하는 경우에만 유효
  - 오실레이터가 동작하고 있으므로 Standby 모드가 해제되어 정상동작이 개시되는데 6개의 사이클만이 소요

ATmega128의 기본 하드웨어 – 슬립 모드

- Extended Standby
  - 슬립 모드에서도 클럭 발생회로인 오실레이터가 동작하고 있다는 것을 제외하면 Power-save 모드와 유사
  - 이 모드는 외부 크리스탈 또는 레조네이터를 사용하는 경우에만 유효
  - 오실레이터가 동작하고 있으므로 Standby 모드가 해제되어 정상 동작이 개시되는데 6개의 사이클만이 소요

ATmega128의 기본 하드웨어 – JTAG 인터페이스

- JTAG
  - JTAG(Joint Test Action Group)이라는 데이터 전송 포맷
  - AVR 8비트 RISC 마이크로 컨트롤러 칩상에서 디버깅 하는 툴
  - 컴퓨터와 반도체 소자를 연결하여 디버깅(on-chip debugging), 불휘발성 메모리의 프로그래밍(ISP : In-System Programming), PCB 테스트 등을 수행하는 인터페이스 기법
  - IEEE Standard 1149.1로 표준화되어 있음
  - MCUCR 레지스터의 JTD 비트를 사용하여 인터페이스 기능을 허용 또는 금지시킬 수 있음
  - Atmel AVR 디바이스는 이 기능을 프로그래밍과 온칩 디버거로 범위를 확장